



Patient Monitoring System (PMS)

Phase Number

Architectural extension

ALASTNAME1-BLASTNAME2-LASTNAME3

ACADEMIC YEAR 2025 – 2026

H09B5B: SOFTWAREARCHITECTUUR

H07Z9B: SOFTWARE ARCHITECTURE

FirstName1 A LASTNAME1 (m000000)

FirstName2 B LASTNAME2 (s000000)

FirstName3 LASTNAME3 (r000000)

Contents

1	Introduction	5
2	QAS Decisions	6
2.1	Qual1: name of the quality	6
3	Other decisions	7
4	Discussion	8
A	Client-Server View	9
B	Decomposition View	14
C	Deployment view	16
D	Process View	20
E	Element catalog	24
E.1	Components	24
E.1.1	Av2AcknowledgementService	24
E.1.2	Av2AdminPortal	24
E.1.3	Av2AuthenticationValidator	24
E.1.4	Av2CommunicationGapLog	24
E.1.5	Av2CommunicationGateway	25
E.1.6	Av2DataIngestionService	25
E.1.7	Av2EmergencyBackupReceiver	25
E.1.8	Av2FailureClassifier	25
E.1.9	Av2FailureDetector	26
E.1.10	Av2GatewayRegistry	26
E.1.11	Av2HeartbeatTracker	26
E.1.12	Av2InternalFailureDetector	27
E.1.13	Av2MessageBroker	27
E.1.14	Av2MissingDataDetector	27
E.1.15	Av2MonitoringService	27
E.1.16	Av2PatientGateway	28
E.1.17	Av2RecoverySyncService	28
E.1.18	Av2RecoveryTracker	28
E.1.19	Av2RedeploymentCoordinator	28
E.1.20	Av2RequestReceiver	29
E.1.21	Av2RetryCoordinator	29
E.1.22	Av2RoutingManager	29
E.1.23	Av2SensorDataRepository	29
E.1.24	Av2SLAMonitor	30
E.1.25	Av2StakeholderNotifier	30
E.1.26	ClinicalJobCreator	30
E.1.27	ClinicalModelCache	30
E.1.28	ClinicalModelDB	31
E.1.29	eHealth Platform	31
E.1.30	HIS	31
E.1.31	KVStorageService	31
E.1.32	MLaaS	32
E.1.33	MLModelManager	32
E.1.34	OtherFunctionality	32

E.1.35	P1CommandRouter	32
E.1.36	P1ConfigurationHandler	32
E.1.37	P1CorrelationTracker	33
E.1.38	P1DurableNotificationLog	33
E.1.39	P1EHRProxyModule	33
E.1.40	P1HISAdapter	33
E.1.41	P1NotificationDispatcher	33
E.1.42	P1NotificationInboxModule	34
E.1.43	P1OnDemandConsultationHandler	34
E.1.44	P1PatientDataService	34
E.1.45	P1PatientGatewayCommander	34
E.1.46	P1PatientQueryService	35
E.1.47	P1PatientStatusCache	35
E.1.48	P1PatientStatusModule	35
E.1.49	P1PhysicianCommandService	35
E.1.50	P1PhysicianGateway	35
E.1.51	P1PriorityBuffer	36
E.1.52	P1RecipientRegistry	36
E.1.53	P1RiskEventSubscriber	36
E.1.54	P1RiskLevelHandler	36
E.1.55	P1SensorIngestModule	36
E.1.56	PatientManagementService	37
E.1.57	PatientRecordRepository	37
E.1.58	PatientStatusService	37
E.1.59	RiskEstimationCombiner	37
E.1.60	RiskEstimationProcessor	38
E.1.61	RiskEstimationScheduler	38
E.1.62	UserAuthenticationService	38
E.2	Modules	38
E.2.1	Av2BackupChannelManager	38
E.2.2	Av2BatteryMonitor	39
E.2.3	Av2DegradedModeManager	39
E.2.4	Av2EmergencyNotificationService	39
E.2.5	Av2LocalBufferingRepository	39
E.2.6	Av2NetworkConnectivityMonitor	39
E.2.7	Av2PatientAlertService	40
E.2.8	Av2RetryAndSynchronizationService	40
E.2.9	Av2SensorCommunicator	40
E.2.10	Av2SensorDataTransmitter	40
E.2.11	Av2ServerConnectivityMonitor	41
E.2.12	JobMLModelSelector	41
E.2.13	JobProcessor	41
E.2.14	MLaaSLibrary	41
E.2.15	MLLibrary	41
E.2.16	MLModelRepository	42
E.2.17	MLModelRetrieval	42
E.2.18	MLModelStorageManager	42
E.2.19	MLModelUpdateProcessor	42
E.2.20	ModelUpdater	42
E.2.21	SensorDataRiskAssessment	43
E.3	Interfaces	43
E.3.1	ApplyClinicalModel	43
E.3.2	Av2AuthFailureSignals	43
E.3.3	Av2AvailabilityMonitoring	43
E.3.4	Av2BackupChannelDispatch	44
E.3.5	Av2BackupEmergencyIngress	44
E.3.6	Av2BatteryStatus	45
E.3.7	Av2BrokerCommitNotification	45

E.3.8	Av2BufferedDataDispatch	45
E.3.9	Av2BufferedSensorData	46
E.3.10	Av2CommunicationGapHistory	46
E.3.11	Av2CommunicationGatewayHealthCheck	47
E.3.12	Av2ConnectivityStatus	47
E.3.13	Av2DegradeModeStatus	47
E.3.14	Av2DeliveryAck	48
E.3.15	Av2DetectedFailures	48
E.3.16	Av2EmergencyDispatch	49
E.3.17	Av2EscalationEvents	49
E.3.18	Av2FailureEvents	49
E.3.19	Av2GatewayDataIngress	50
E.3.20	Av2GatewayResyncCommand	50
E.3.21	Av2HeartbeatUpdate	50
E.3.22	Av2InboundGatewayData	51
E.3.23	Av2MessageBrokerHealthCheck	51
E.3.24	Av2NotificationDispatch	51
E.3.25	Av2PatientContactLookup	52
E.3.26	Av2PatientStatusOverview	52
E.3.27	Av2QueuedSensorDataMgmt	52
E.3.28	Av2RecoveryCoordination	53
E.3.29	Av2RecoveryStatus	53
E.3.30	Av2RedeploymentControl	53
E.3.31	Av2RetryDispatch	54
E.3.32	Av2RoutFailureSignals	54
E.3.33	Av2SensorDataAcquisition	54
E.3.34	Av2ServerConnectionStatus	55
E.3.35	Av2SLAReporting	55
E.3.36	Av2ValidatedTransmission	55
E.3.37	ClinicalJobMgmt	56
E.3.38	ClinicalModelCacheMgmt	56
E.3.39	ClinicalModelStorage	56
E.3.40	DataBuffer	56
E.3.41	FetchClinicalModels	57
E.3.42	FetchJobs	57
E.3.43	FetchMLModel	57
E.3.44	healthAPI	58
E.3.45	JobMgmt	59
E.3.46	KVStore	59
E.3.47	LaunchRiskEstimation	59
E.3.48	MLaaSAPI	60
E.3.49	MLaaSService	60
E.3.50	MLAPI	61
E.3.51	MLModelMgmt	61
E.3.52	MLModelStorage	62
E.3.53	MLModelSync	62
E.3.54	MLModelUpdateMgmt	62
E.3.55	OtherDataMgmt	62
E.3.56	P1ClinicalConfigMgmt	63
E.3.57	P1IConfigCommand	63
E.3.58	P1ICorrelationTracking	63
E.3.59	P1IHISAccess	63
E.3.60	P1INotificationInbox	64
E.3.61	P1INotificationLog	64
E.3.62	P1IOnDemandCommand	65
E.3.63	P1IOnDemandSensorFetch	65
E.3.64	P1IPatientQuery	65
E.3.65	P1IPatientStatusRead	65

E.3.66	P1IPhysicianAPI	65
E.3.67	P1IPhysicianCommand	66
E.3.68	P1IPriorityBuffer	66
E.3.69	P1IRecipientRegistry	67
E.3.70	P1IRiskEvents	67
E.3.71	P1IRiskLevelCommand	67
E.3.72	PatientRecordMgmt	67
E.3.73	Results	68
E.3.74	SensorDataAnalysis	68
E.3.75	SensorDataMgmt	68
E.3.76	UserAuthentication	68
E.4	Nodes	69
E.4.1	AdminNode	69
E.4.2	AdminNode (Pilot Deployment)	69
E.4.3	AdminWorkstation	69
E.4.4	BackendGatewayNode	69
E.4.5	BackendGatewayNode (Pilot Deployment)	69
E.4.6	DevBackendNode (Dev Deployment)	70
E.4.7	DevGatewayNode (Dev Deployment)	70
E.4.8	DevPatientRecordsNode (Dev Deployment)	70
E.4.9	eHealthPlatform	70
E.4.10	GatewayNode	70
E.4.11	GatewayNode (Pilot Deployment)	70
E.4.12	HISServer	70
E.4.13	MLaaS Conn (Pilot Deployment)	70
E.4.14	MLaaS Service (Dev Deployment)	70
E.4.15	MLaaS Service (Pilot Deployment)	71
E.4.16	MLaaSCloud	71
E.4.17	MLaaSConnectorNode	71
E.4.18	MonitoringNode	71
E.4.19	MonitoringNode (Pilot Deployment)	71
E.4.20	PatientDataNode	71
E.4.21	PatientDataNode (Dev Deployment)	71
E.4.22	PatientDataNode (Pilot Deployment)	71
E.4.23	PatientGatewayDevice	71
E.4.24	PatientRecordsNode	71
E.4.25	PatientRecordsNode (Pilot Deployment)	72
E.4.26	PhysicianNode	72
E.4.27	PhysicianNode (Pilot Deployment)	72
E.4.28	PhysicianWorkstation	72
E.4.29	RiskEstimationManagementNode (Pilot Deployment)	72
E.4.30	RiskEstimationManagmentNode (Dev Deployment)	72
E.4.31	RiskEstimationMgmtNode	72
E.4.32	RiskEstimationProcessorNode	72
E.4.33	RiskEstimationProcessorNode (Dev Deployment)	73
E.4.34	RiskEstimationProcessorNode (Pilot Deployment)	73
E.4.35	SMSTGatewayProvider	73
E.4.36	TelecomNetwork	73
E.4.37	TODO Node (Pilot Deployment)	73
E.4.38	TODONode	73
E.5	Exceptions	73
E.6	Data types	74
E.7	Unresolved issues	80

1. Introduction

2. QAS Decisions

2.1 Qual1: name of the quality

Key Decisions

(AD1) Key decision 1

(AD2) Key decision 2

Architectural Decision 1: name of the decision

Description and motivation of the decision.

Employed tactics and patterns

name of tactic1, name of tactic2, name of tactic3

Sensitivity points

Description of the sensitivity points.

Relation to other architectural decisions

How this decision relates to other architectural decisions.

Trade-off justification

Justification of the trade-offs.

Considered alternatives

Name of alternative 1. Describe alternative and why it was not chosen.

Architectural Decision 2: name of the decision

Description and motivation of the decision.

Employed tactics and patterns

name of tactic1, name of tactic2

Sensitivity points

Description of the sensitivity points.

Relation to other architectural decisions

How this decision relates to other architectural decisions.

Trade-off justification

Justification of the trade-offs.

Considered alternatives

Name of alternative 1. Describe alternative and why it was not chosen.

Name of alternative 2. Describe alternative and why it was not chosen.

3. Other decisions

4. Discussion

A. Client-Server View

Figures

A.1	Context diagram for the client-server view	9
A.2	Primary diagram of the client-server view	10
A.3	Decomposition of the Av2CommunicationGateway	11
A.4	Decomposition of the Av2MonitoringService	11
A.5	Decomposition of the P1NotificationDispatcher	12
A.6	Decomposition of the P1PatientDataService	13
A.7	Decomposition of the P1PhysicianCommandService	13

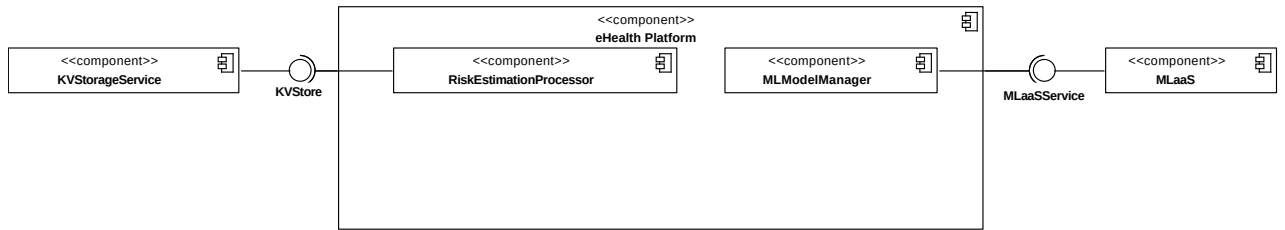


Figure A.1: Context diagram for the client-server view

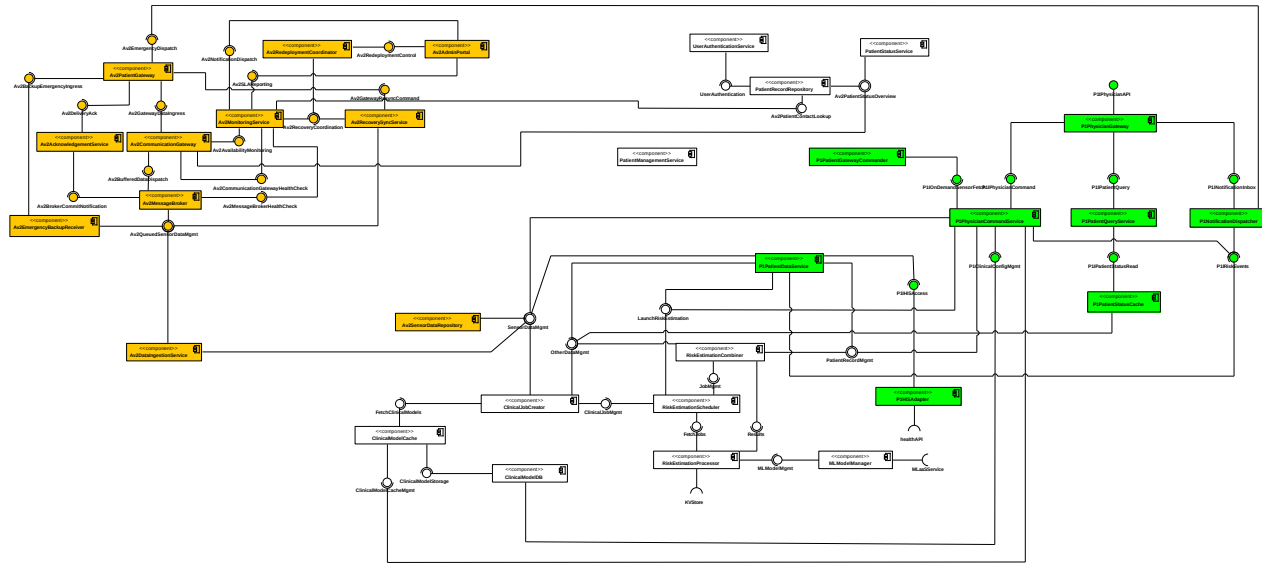


Figure A.2: Primary diagram of the client-server view

This architecture is not yet complete. Functionality that still has to be worked out in more detail is captured by the `OtherFunctionality` component in gray.

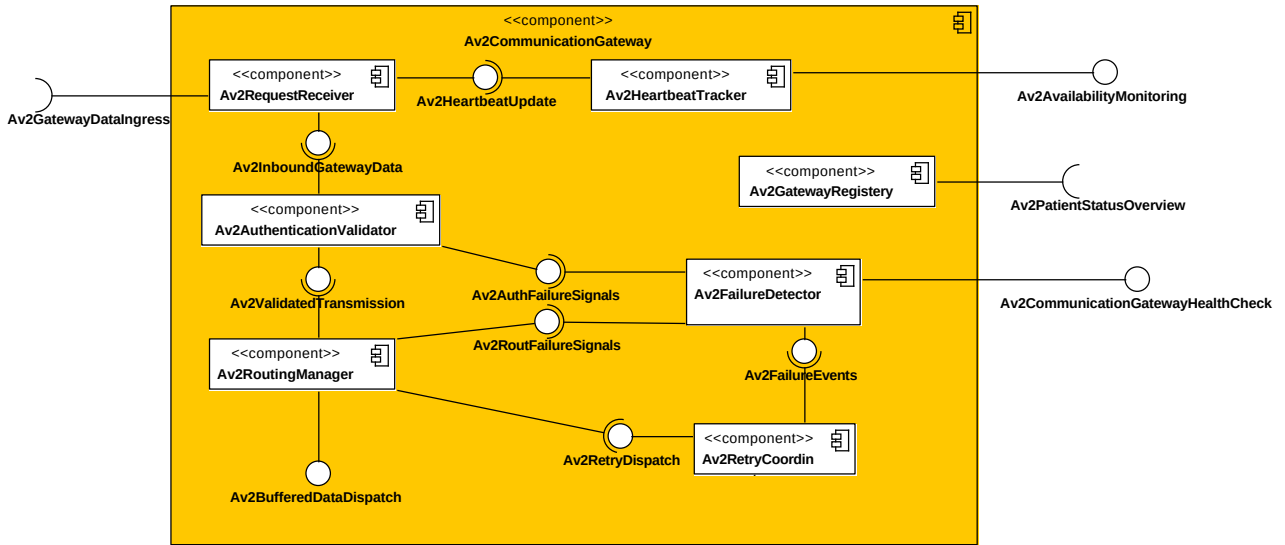


Figure A.3: Decomposition of the Av2CommunicationGateway

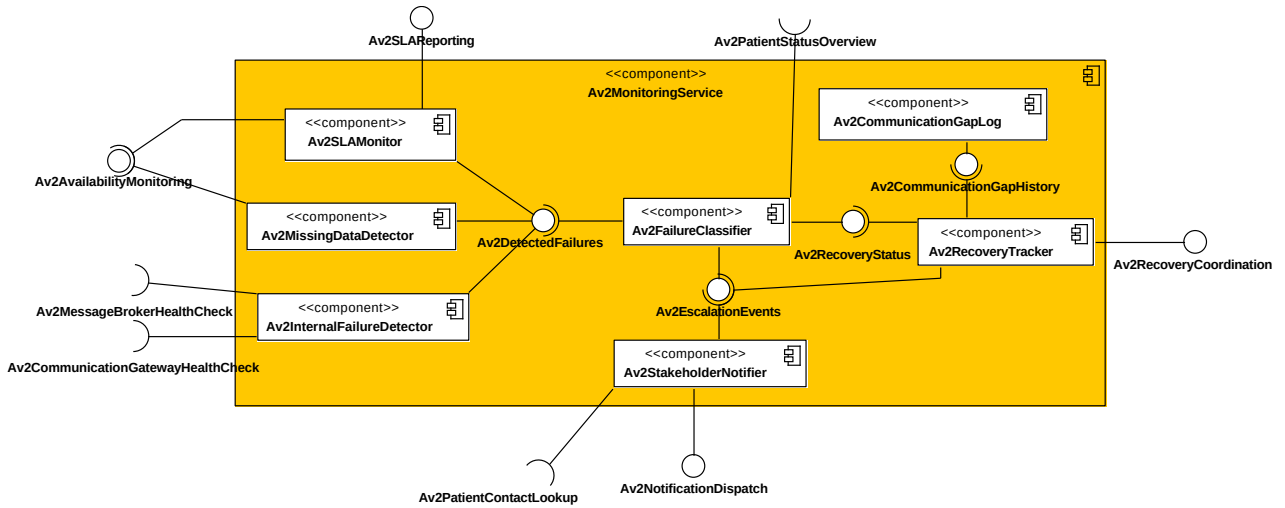


Figure A.4: Decomposition of the Av2MonitoringService

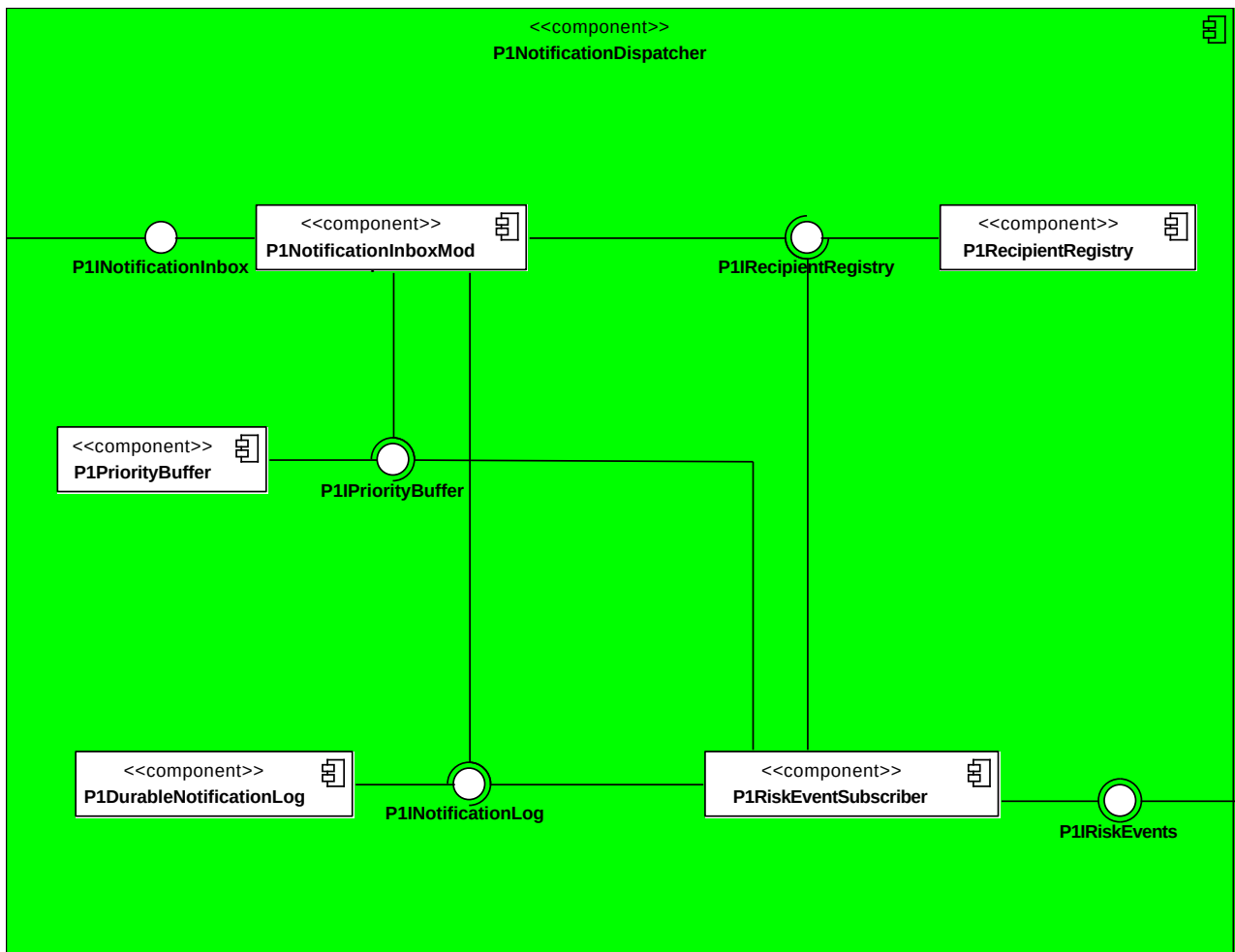


Figure A.5: Decomposition of the P1NotificationDispatcher

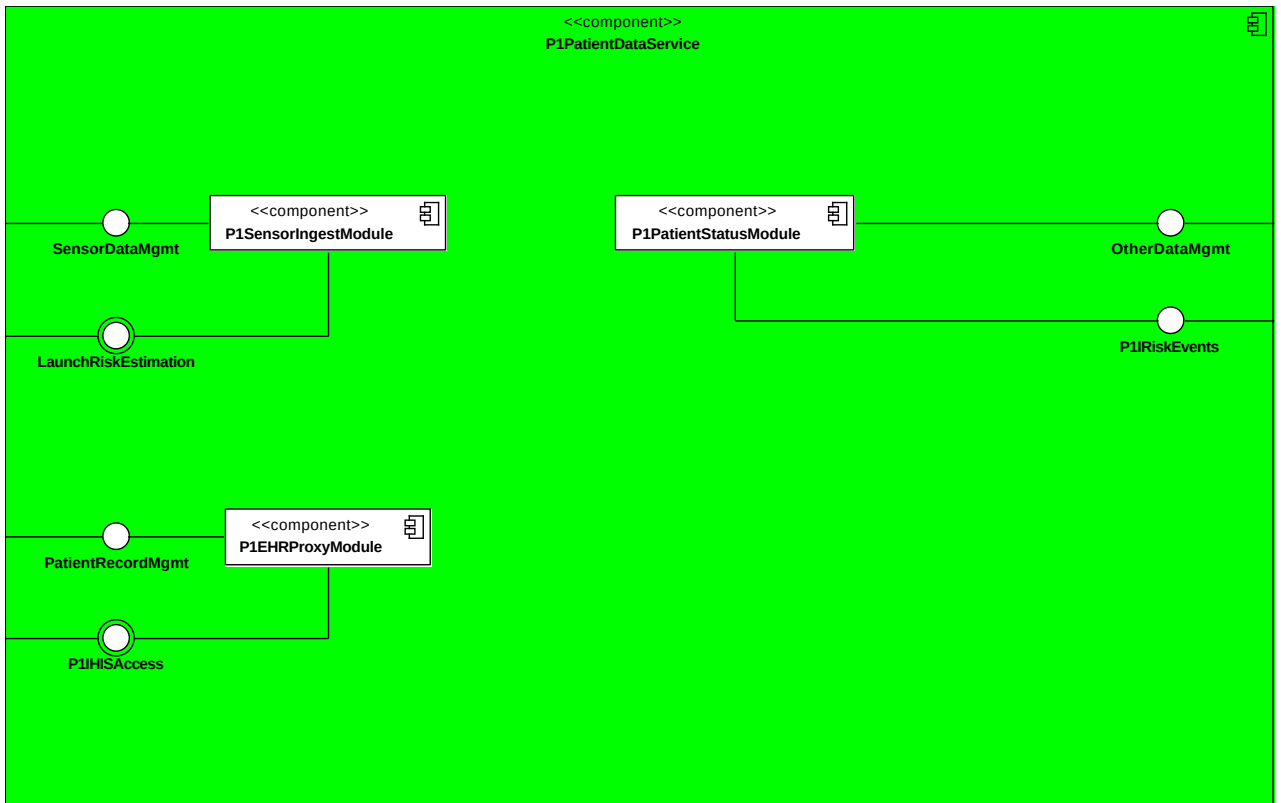


Figure A.6: Decomposition of the P1PatientDataService

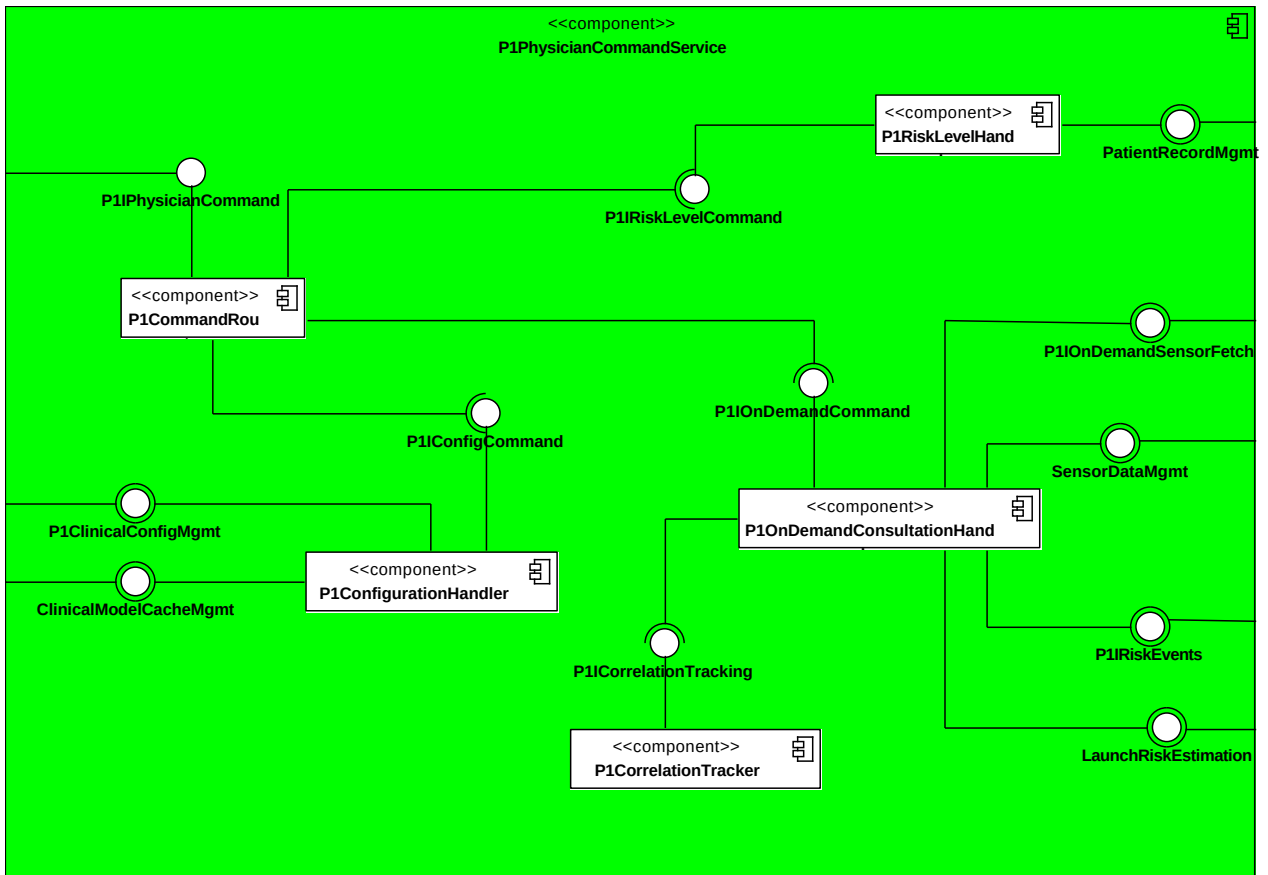


Figure A.7: Decomposition of the P1PhysicianCommandService

B. Decomposition View

Figures

B.1	Context diagram for the decomposition view	14
B.2	Decomposition of the Av2PatientGateway	14
B.3	Decomposition of the MLModelManager	15
B.4	Decomposition of the RiskEstimationProcessor	15

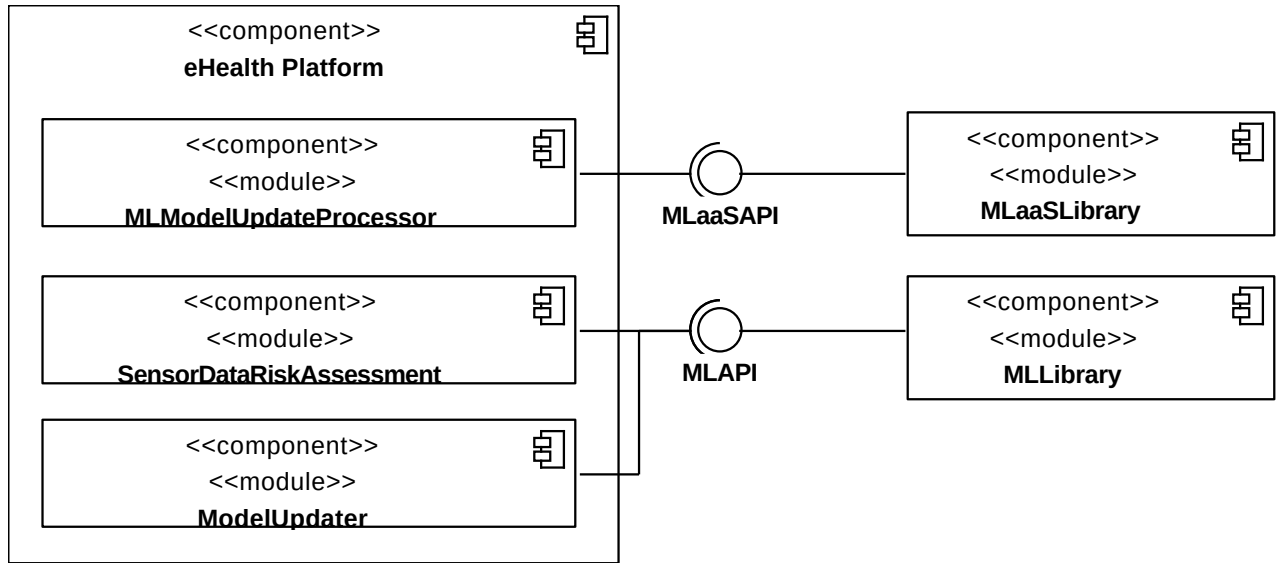


Figure B.1: Context diagram for the decomposition view

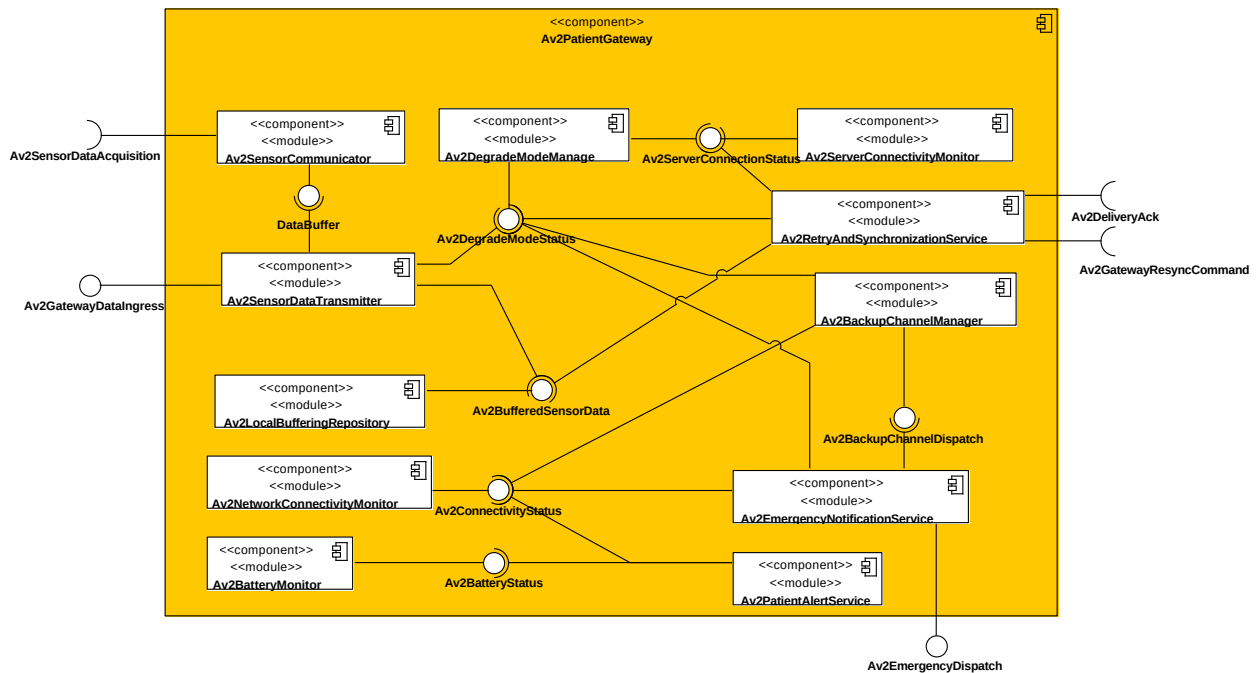


Figure B.2: Decomposition of the Av2PatientGateway

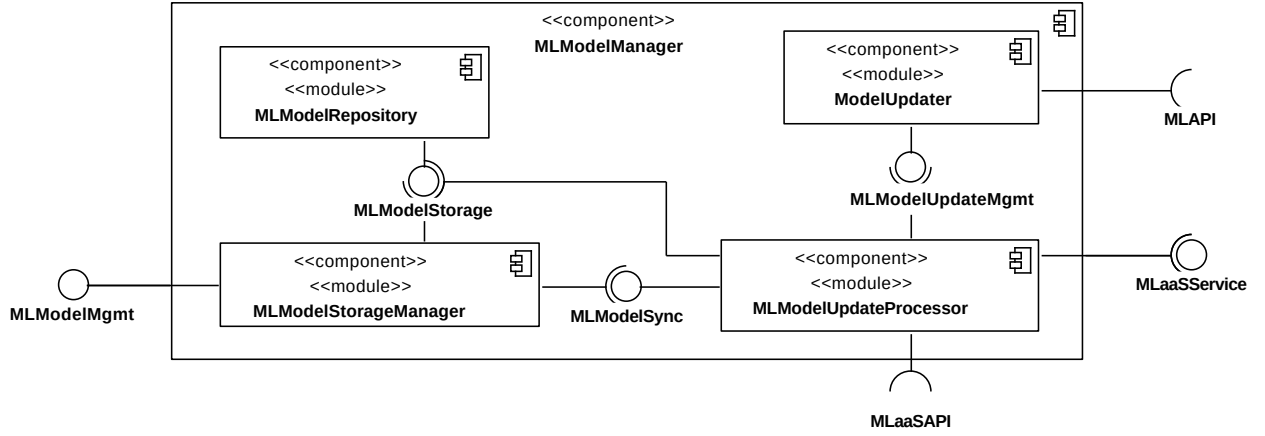


Figure B.3: Decomposition of the MLModelManager

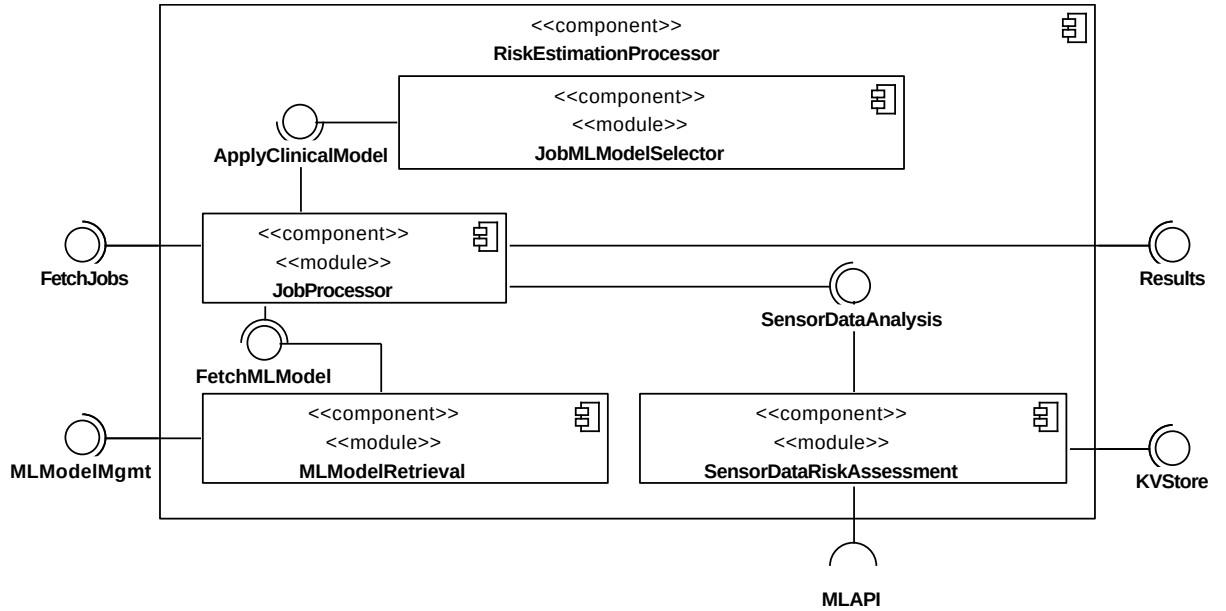


Figure B.4: Decomposition of the RiskEstimationProcessor

C. Deployment view

Figures

C.1	Context diagram for the deployment view	16
C.2	Primary deployment diagram	17
C.3	Pilot Deployment (200 patients, 5 clin models)	18
C.4	Development Test Deployment (20 patients, 3 clin models)	19

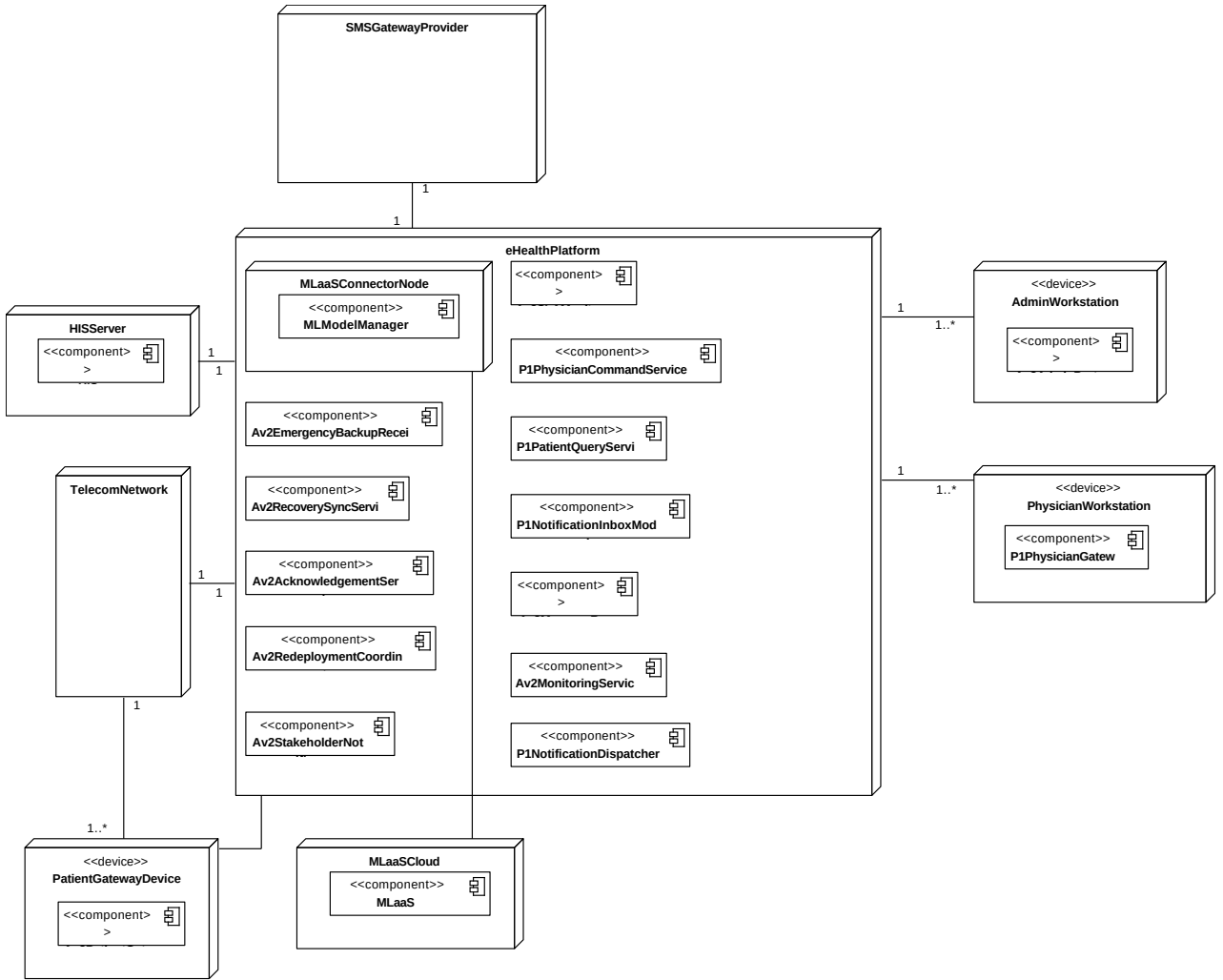


Figure C.1: Context diagram for the deployment view

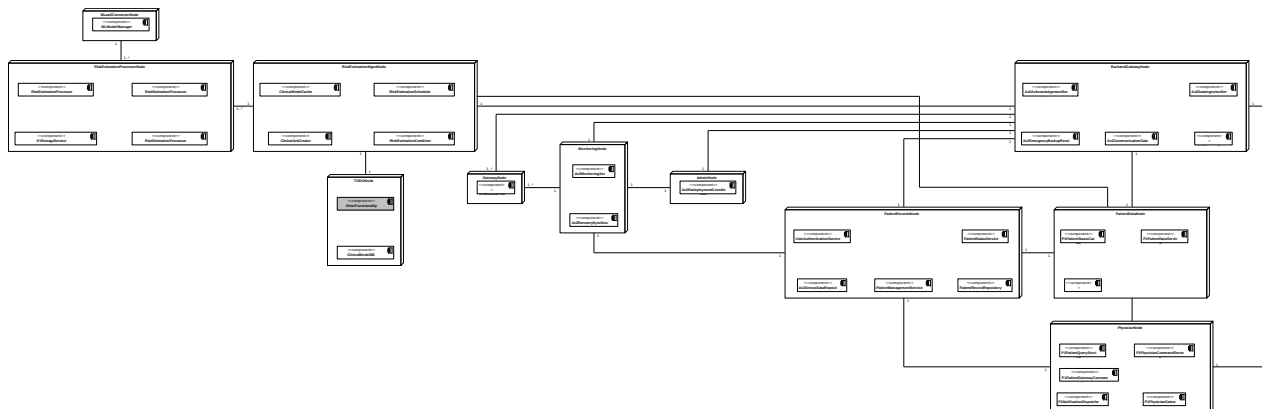


Figure C.2: Primary deployment diagram

This deployment is necessarily incomplete as the architecture is incomplete. The functionality that is not worked out yet is captured by **OtherFunctionality** component, currently deployed on **TODONode**. This **TODONode** will also have to be replaced with the final nodes are required by the replacement of **OtherFunctionality**.

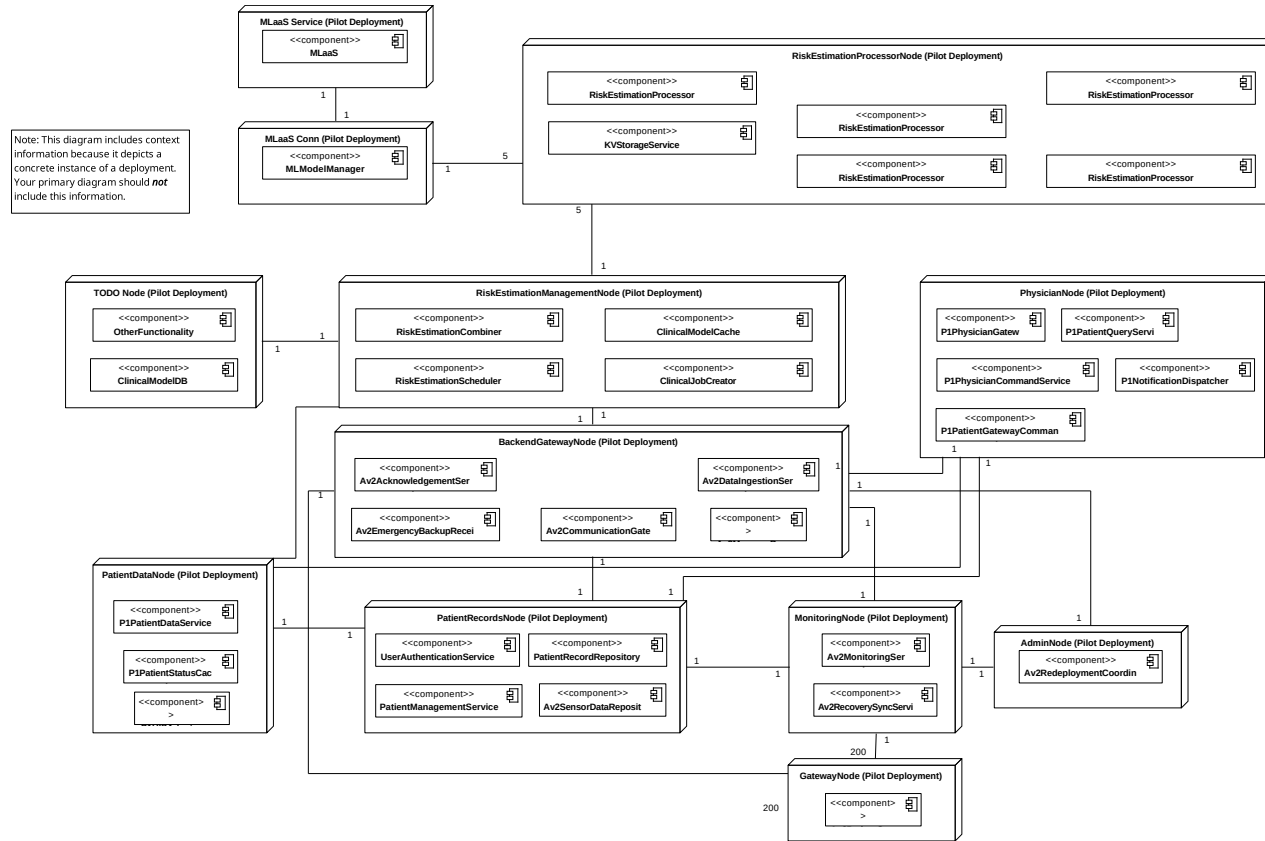


Figure C.3: Pilot Deployment (200 patients, 5 clin models)

This diagram shows a concrete initial pilot deployment for 200 patients with an average of 5 clinical models per patient. This diagram is a concrete instance of a deployment for the specified number of patients and clinical models, therefore it also includes information from the context diagram. Note that it is not correct to include this type of information in your primary deployment diagram.

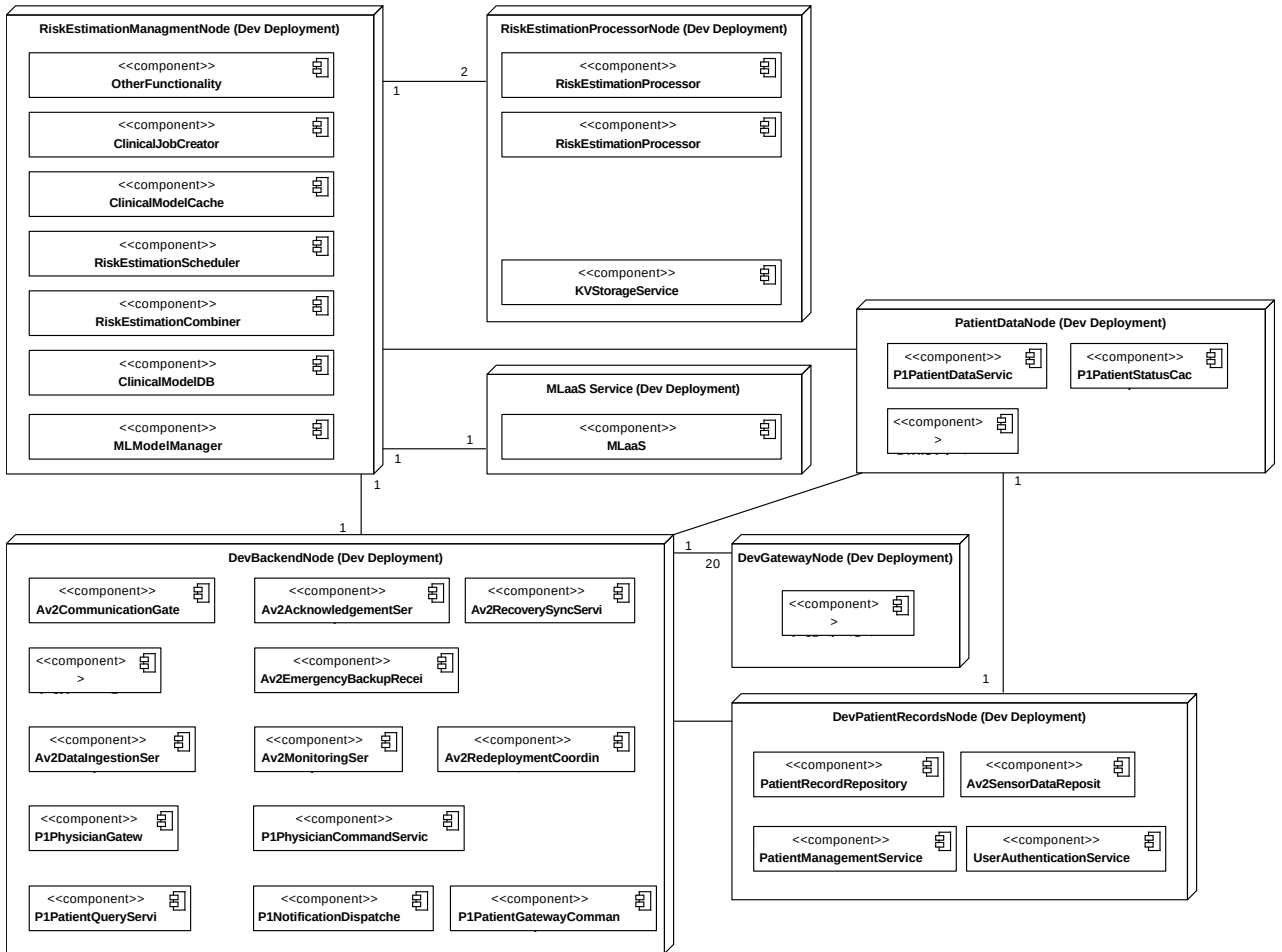


Figure C.4: Development Test Deployment (20 patients, 3 clin models)

This diagram shows a simplified deployment for local development. This diagram is a concrete instance of a deployment for the specified number of patients and clinical models, therefore it also includes information from the context diagram. Note that it is not correct to include this type of information in your primary deployment diagram.

D. Process View

Figures

D.1 Risk estimation process	20
D.2 Compute clinical model result	21
D.3 MLModel Synchronization	21
D.4 Processing MLModel updates	22
D.5 Incoming sensor data	22
D.6 Combining ClinicalModelJob results	23

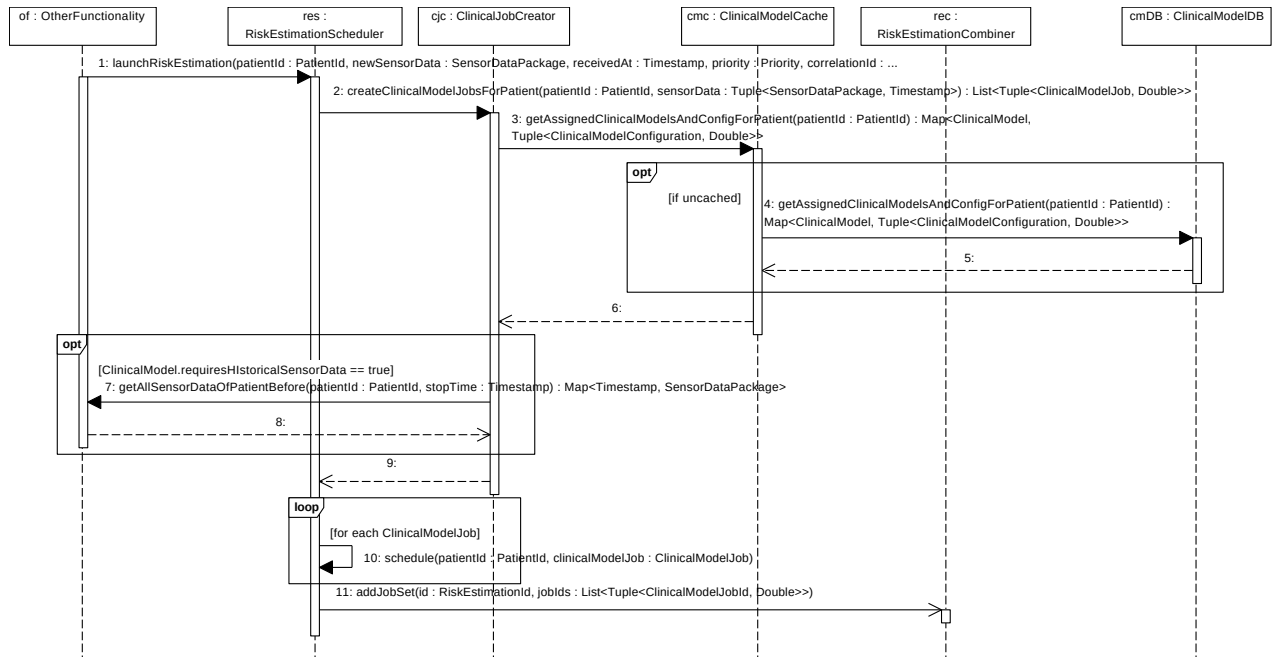


Figure D.1: Risk estimation process

The scheduling and execution of a risk estimation triggered by the arrival of new sensor data. Note that if one or more clinical models in the **ClinicalModelCache** have been invalidated, all relevant clinical models are renewed.

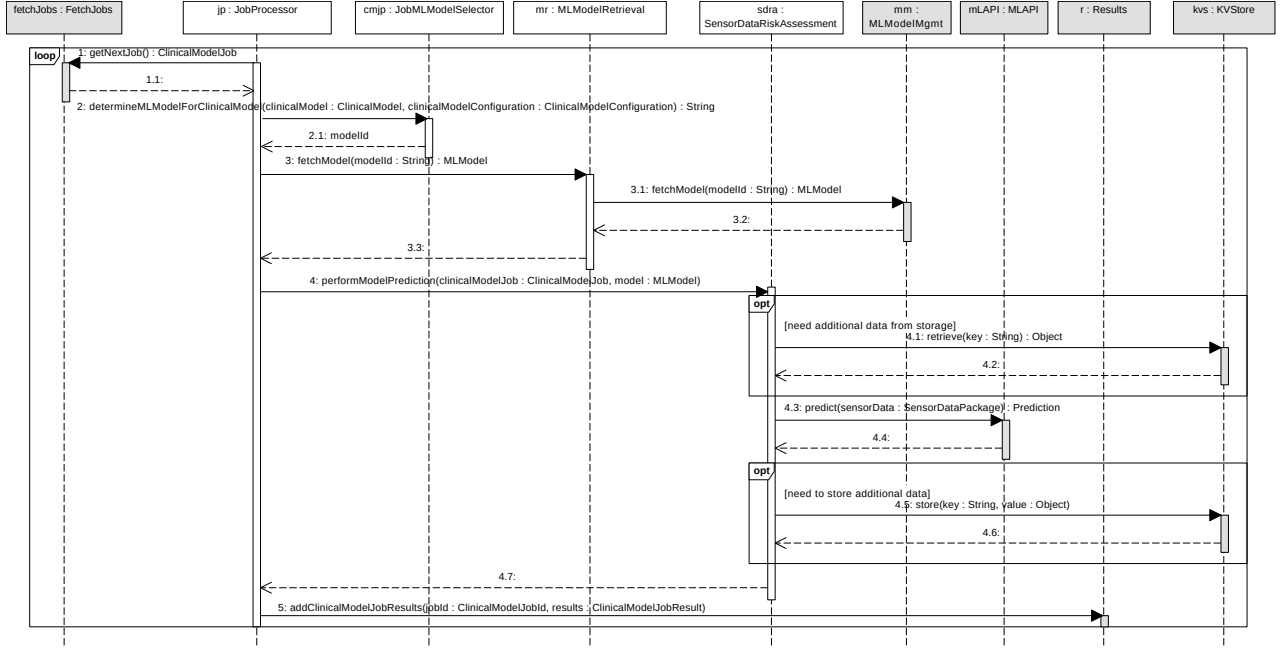


Figure D.2: Compute clinical model result

This sequence diagram shows the detailed application and computation of the clinical model. The first step involves the retrieval of the appropriate **MLModel** (specified by the **ClinicalModel**). The second step involves the calculation of the prediction using the retrieved **MLModels**. Note that, as this diagram depicts the internal flow between modules within the **RiskEstimationProcessor**, the gray lifelines depict its (external) interfaces.

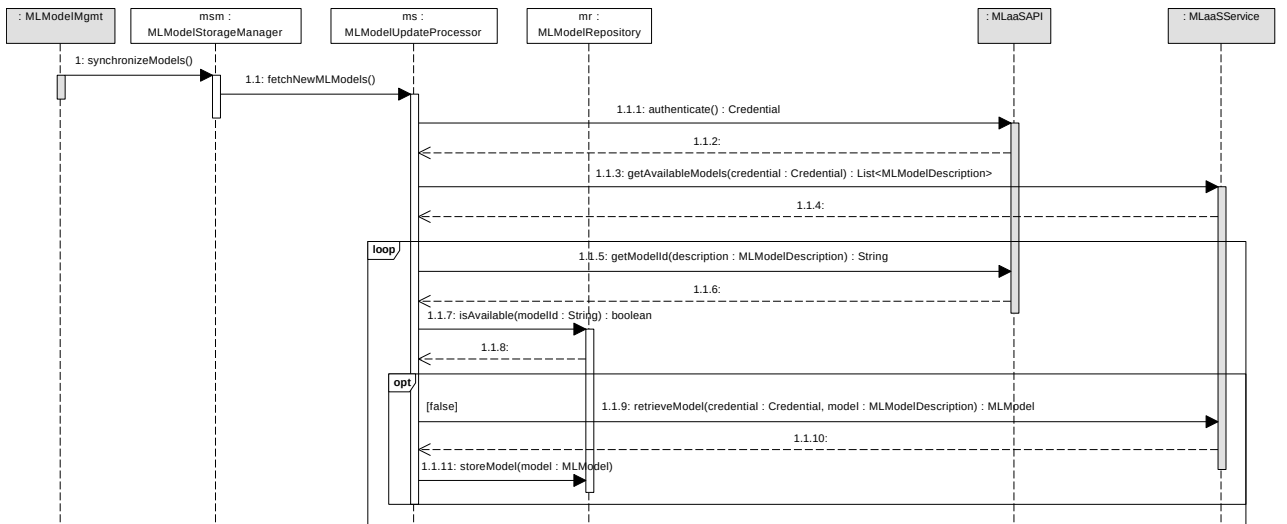


Figure D.3: MLModel Synchronization

As the system operations new **MLModels** may become available for use by the **ClinicalModels**. This diagram illustrates the synchronization logic to fetch any unavailable **MLModels** and to store a local copy of the **MLModels** in the **MLModelRepository** for faster retrieval and use in the sensor data risk assessment. Note that, as this diagram depicts the internal flow between modules within the **MLModelManager**, the gray lifelines depict its (external) interfaces.

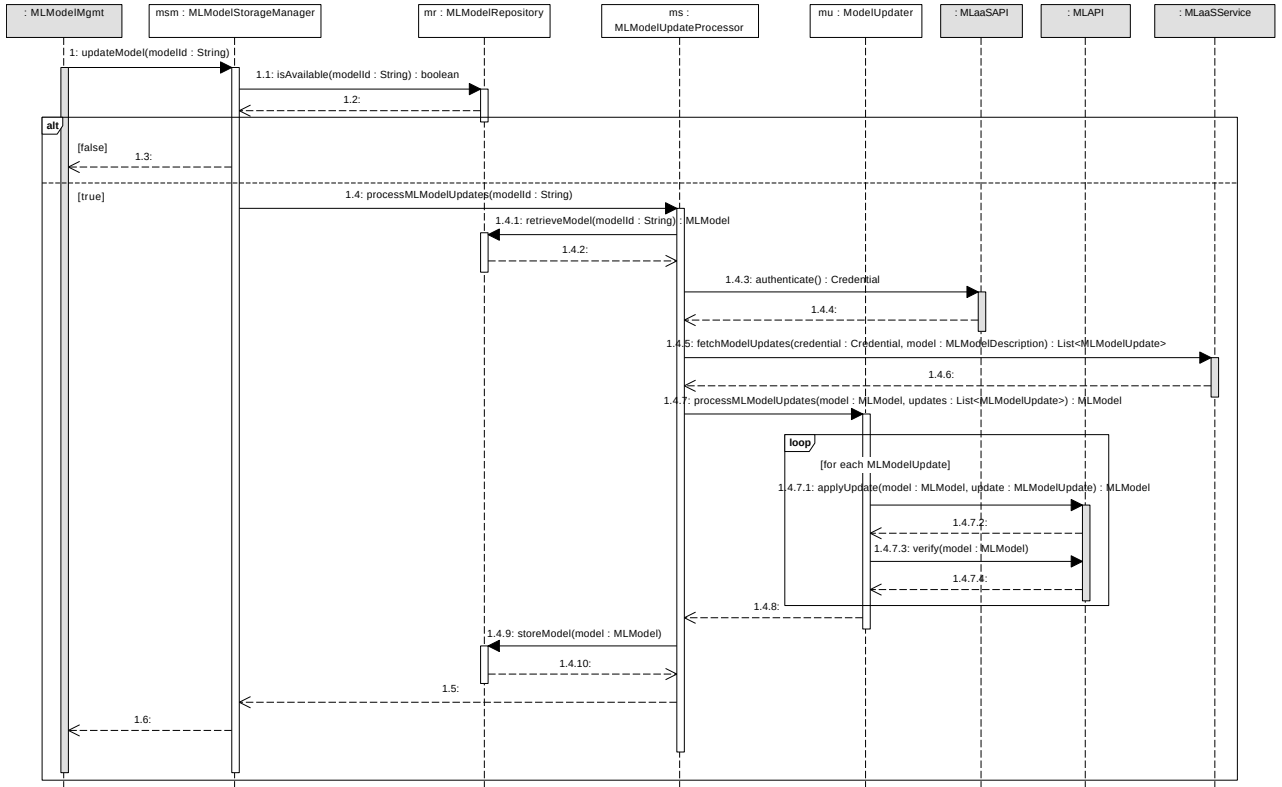


Figure D.4: Processing MLModel updates

MLModelUpdates are made available to improve the existing **MLModels**. This diagram illustrates the retrieval of these **MLModelUpdates** and their application on the local **MLModel**. As updates to these models influence the risk assessments, they need to be triggered for each individual **MLModel** that requires an update. Note that, as this diagram depicts the internal flow between modules within the **MLModelManager**, the gray lifelines depict its (external) interfaces.

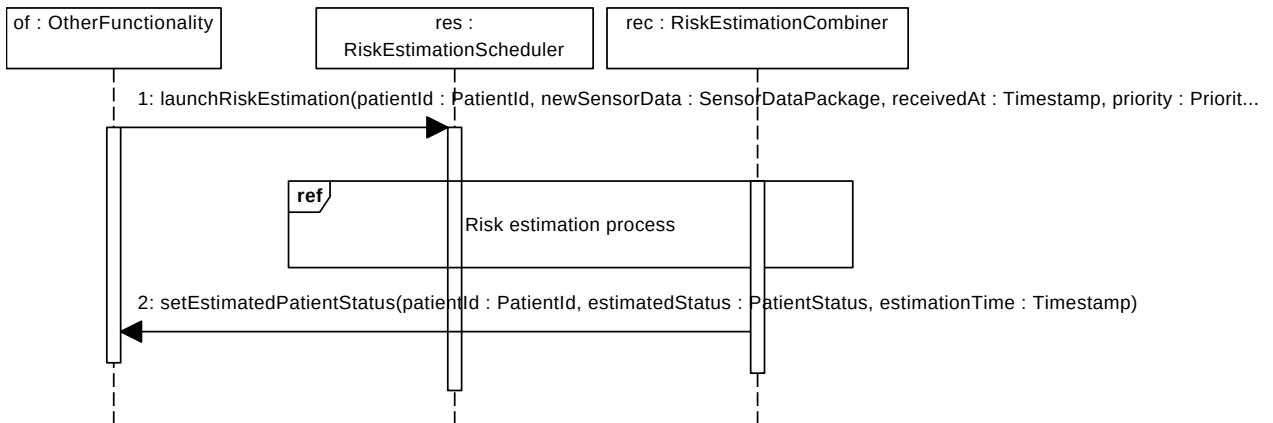


Figure D.5: Incoming sensor data

This diagram shows a small part of the processing of incoming sensor data. It does not yet depict the arrival of the sensor data, storage in a database, or any other processing, as this functionality is not yet worked out.

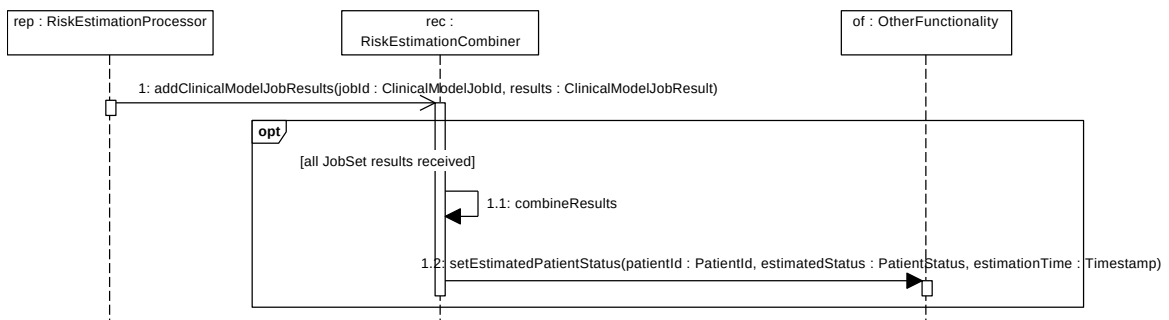


Figure D.6: Combining ClinicalModelJob results

E. Element catalog

E.1 Components

E.1.1 Av2AcknowledgementService

Responsibility: After RoutingManager confirms downstream delivery (e.g., MessageBroker accepts the message), DeliveryAcknowledger emits an acknowledgement back to the originating PatientGateway, allowing the gateway's RetryAndSynchronizationService to clear delivered entries from LocalBufferingRepository.

Super-components: None

Sub-components: None

Provided interfaces: -o Av2DeliveryAck

Required interfaces: -c Av2BrokerCommitNotification

Deployed on: ☐ eHealthPlatform, ☐ BackendGatewayNode, ☐ BackendGatewayNode (Pilot Deployment), ☐ DevBackendNode (Dev Deployment)

Visible on diagrams: figs. A.2, C.1, C.2, C.3 and C.4

E.1.2 Av2AdminPortal

Responsibility: Provides the system administrator with a web-based interface for monitoring the operational status of the PMS, receiving failure notifications, and triggering redeployment or rollback of failed internal subsystems. It consumes the RedeploymentControl interface to initiate and monitor redeployment jobs, and receives notifications from the MonitoringService via the NotificationDispatch interface. It is the only component through which a system administrator can interact with the RedeploymentCoordinator.

Super-components: None

Sub-components: None

Provided interfaces: None

Required interfaces: -c Av2NotificationDispatch, -c Av2RedeploymentControl, -c Av2SLAReporting

Deployed on: ☐ AdminWorkstation

Visible on diagrams: figs. A.2 and C.1

E.1.3 Av2AuthenticationValidator

Responsibility: Validates the authentication credentials presented by patient gateways before any data is processed further. It receives parsed transmissions from the RequestReceiver via InboundGatewayData, verifies that the gateway is registered and that its credentials are valid, and forwards authenticated transmissions to the RoutingManager via ValidatedTransmission. Transmissions that fail authentication are rejected, logged with the gateway identifier, IP address, and timestamp, and a **FailureEvent** is raised to the FailureDetector. Only transmissions from registered and authenticated gateways are allowed to proceed.

Super-components: ☒ Av2CommunicationGateway

Sub-components: None

Provided interfaces: -o Av2AuthFailureSignals, -o Av2ValidatedTransmission

Required interfaces: -c Av2InboundGatewayData

Deployed on: ☐ BackendGatewayNode, ☐ BackendGatewayNode (Pilot Deployment), ☐ DevBackendNode (Dev Deployment)

Visible on diagrams: fig. A.3

E.1.4 Av2CommunicationGapLog

Responsibility: Persistently records the start, end, and duration of every communication interruption per gateway. Provides historical query access for administrators and compliance. Updated by RecoveryTracker when a failure is opened/closed.

Super-components: ☒ Av2MonitoringService

Sub-components: None

Provided interfaces: ☒ Av2CommunicationGapHistory

Required interfaces: None

Deployed on: ☐ eHealthPlatform, ☐ MonitoringNode, ☐ DevBackendNode (Dev Deployment),
☐ MonitoringNode (Pilot Deployment)

Visible on diagrams: fig. A.4

E.1.5 Av2CommunicationGateway

Responsibility: The single backend ingress point for all primary-channel patient gateway traffic. Authenticates incoming transmissions, routes them to downstream services (MessageBroker, EmergencyNotificationService), detects communication failures, coordinates retries with exponential backoff, tracks per-gateway heartbeats for missing-data detection, and acknowledges successfully dispatched messages back to the originating gateway.

Super-components: None

Sub-components: ☒ Av2RequestReceiver, ☒ Av2AuthenticationValidator, ☒ Av2RoutingManager,
☒ Av2FailureDetector, ☒ Av2RetryCoordinator, ☒ Av2HeartbeatTracker, ☒ Av2GatewayRegistry

Provided interfaces: ☒ Av2AvailabilityMonitoring, ☒ Av2BufferedDataDispatch, ☒ Av2CommunicationGatewayHealth

Required interfaces: ☒ Av2GatewayDataIngress, ☒ Av2PatientStatusOverview

Deployed on: ☐ BackendGatewayNode, ☐ BackendGatewayNode (Pilot Deployment), ☐ DevBackendNode (Dev Deployment)

Visible on diagrams: figs. A.2, A.3, C.2, C.3 and C.4

E.1.6 Av2DataIngestionService

Responsibility: Consumes validated sensor data and emergency notifications from the MessageBroker and persists them through the SensorDataRepository. Acts as the single write path for incoming patient data into the backend persistence layer, decoupling the ingestion rate from the downstream risk-estimation pipeline.

Super-components: None

Sub-components: None

Provided interfaces: None

Required interfaces: ☒ Av2QueuedSensorDataMgmt, ☒ SensorDataMgmt

Deployed on: ☐ BackendGatewayNode, ☐ BackendGatewayNode (Pilot Deployment), ☐ DevBackendNode (Dev Deployment)

Visible on diagrams: figs. A.2, C.2, C.3 and C.4

E.1.7 Av2EmergencyBackupReceiver

Responsibility: Receives emergency notifications transmitted via the backup SMS gateway when the primary channel is unavailable. Authenticates the sending patient gateway by a pre-registered device token embedded in the SMS payload, and forwards validated emergencies to the MessageBroker for the same downstream processing as primary-channel emergencies.

Super-components: None

Sub-components: None

Provided interfaces: ☒ Av2BackupEmergencyIngress

Required interfaces: ☒ Av2QueuedSensorDataMgmt

Deployed on: ☐ eHealthPlatform, ☐ BackendGatewayNode, ☐ BackendGatewayNode (Pilot Deployment), ☐ DevBackendNode (Dev Deployment)

Visible on diagrams: figs. A.2, C.1, C.2, C.3 and C.4

E.1.8 Av2FailureClassifier

Responsibility: Receives **FailureEvent** objects from both the MissingDataDetector and the InternalFailureDetector via the DetectedFailures interface. For each failure it looks up the risk level of the affected patient via PatientStatusOverview, delegates deadline computation to the NotificationTimingPolicy, and constructs a **ClassifiedFailure** enriched with the escalation

level and notification deadline. It emits **ClassifiedFailure** objects via `EscalationEvents` to the `StakeholderNotifier`. When a failure is resolved it signals the `RecoveryTracker` via `RecoveryStatus`.

Super-components:  `Av2MonitoringService`

Sub-components: None

Provided interfaces:  `Av2EscalationEvents`,  `Av2RecoveryStatus`

Required interfaces:  `Av2DetectedFailures`,  `Av2PatientStatusOverview`

Deployed on:  `eHealthPlatform`,  `MonitoringNode`,  `DevBackendNode` (Dev Deployment),
 `MonitoringNode` (Pilot Deployment)

Visible on diagrams: fig. A.4

E.1.9 Av2FailureDetector

Responsibility: Monitors the health of the `CommunicationGateway`'s own subcomponents and detects connection-level failures. It responds to the `InternalHealthCheck` interface by reporting the current operational status of the `CommunicationGateway` and its subcomponents. It also receives authentication failure signals from the `AuthenticationValidator` and connection timeout signals, constructing **FailureEvent** objects and publishing them via the `FailureEvents` interface to the `RetryCoordinator`. Internal failures must be detectable within five seconds as required by Av2.

Super-components:  `Av2CommunicationGateway`

Sub-components: None

Provided interfaces:  `Av2CommunicationGatewayHealthCheck`,  `Av2FailureEvents`

Required interfaces:  `Av2AuthFailureSignals`,  `Av2RoutFailureSignals`

Deployed on:  `BackendGatewayNode`,  `BackendGatewayNode` (Pilot Deployment),  `DevBackendNode` (Dev Deployment)

Visible on diagrams: fig. A.3

E.1.10 Av2GatewayRegistry

Responsibility: Maintains the authoritative bidirectional mapping between **GatewayId**, **PatientId**, expected transmission interval (in seconds), and current patient risk level. It is populated during patient registration and updated whenever a patient's risk level changes via `OtherDataMgmt`. It is consulted by `HeartbeatTracker` to calibrate per-gateway expected intervals whenever risk level changes, by `AuthenticationValidator` to verify that an incoming gateway is registered, and by `FailureClassifier` to resolve a **GatewayId** to the associated **PatientId** and risk level before computing escalation deadlines. It is the only component inside `CommunicationGateway` that holds patient-to-gateway identity mappings.

Super-components:  `Av2CommunicationGateway`

Sub-components: None

Provided interfaces: None

Required interfaces:  `Av2PatientStatusOverview`

Deployed on:  `BackendGatewayNode`,  `BackendGatewayNode` (Pilot Deployment),  `DevBackendNode` (Dev Deployment)

Visible on diagrams: fig. A.3

E.1.11 Av2HeartbeatTracker

Responsibility: Maintains a per-gateway registry of the last time a transmission was received from each patient gateway and the configured expected transmission interval for that gateway. It receives heartbeat updates from the `RequestReceiver` via the `HeartbeatUpdate` interface each time a transmission arrives. It exposes the `HeartbeatRegistry` interface so the `MissingDataDetector` in the `MonitoringService` can poll for overdue gateways. It also supports gateway registration so that newly enrolled patients have their expected transmission interval configured from the start.

Super-components:  `Av2CommunicationGateway`

Sub-components: None

Provided interfaces:  `Av2AvailabilityMonitoring`

Required interfaces:  `Av2HeartbeatUpdate`

Deployed on:  `BackendGatewayNode`,  `BackendGatewayNode` (Pilot Deployment),  `DevBackendNode` (Dev Deployment)

Visible on diagrams: fig. A.3

E.1.12 Av2InternalFailureDetector

Responsibility: Polls the health check endpoints of internal communication subsystems — specifically the CommunicationGateway and the MessageBroker — via the InternalHealthCheck interface at a frequency of at least once per second. When a subsystem returns a DEGRADED or DOWN status, or fails to respond within one second, it constructs a **FailureEvent** of type INTERNAL_SUBSYSTEM_CRASH or COMMUNICATION_CHANNEL_DOWN and reports it via the DetectedFailures interface to the FailureClassifier. This polling frequency guarantees detection within five seconds as required by Av2.

Super-components:  Av2MonitoringService

Sub-components: None

Provided interfaces:  Av2DetectedFailures

Required interfaces:  Av2CommunicationGatewayHealthCheck,  Av2MessageBrokerHealthCheck

Deployed on:  eHealthPlatform,  MonitoringNode,  DevBackendNode (Dev Deployment),
 MonitoringNode (Pilot Deployment)

Visible on diagrams: fig. A.4

E.1.13 Av2MessageBroker

Responsibility: Buffers validated sensor-data packages and emergency notifications between the CommunicationGateway and the DataIngestionService. Provides durability and decoupling so that transient failures of downstream services do not lead to data loss, and exposes a health-check endpoint for the InternalFailureDetector.

Super-components: None

Sub-components: None

Provided interfaces:  Av2BrokerCommitNotification,  Av2MessageBrokerHealthCheck,  Av2QueuedSensorData

Required interfaces:  Av2BufferedDataDispatch

Deployed on:  eHealthPlatform,  BackendGatewayNode,  BackendGatewayNode (Pilot Deployment),  DevBackendNode (Dev Deployment)

Visible on diagrams: figs. A.2, C.1, C.2, C.3 and C.4

E.1.14 Av2MissingDataDetector

Responsibility: Periodically polls the HeartbeatRegistry data exposed via the AvailabilityMonitoring interface to identify patient gateways that have not sent an update within their expected transmission interval plus the five-minute tolerance defined by Av2. When an overdue gateway is detected it constructs a **FailureEvent** of type MISSING_SENSOR_DATA and reports it via the DetectedFailures interface to the FailureClassifier. The polling frequency must be high enough to guarantee detection within transmission interval plus five minutes.

Super-components:  Av2MonitoringService

Sub-components: None

Provided interfaces:  Av2DetectedFailures

Required interfaces:  Av2AvailabilityMonitoring

Deployed on:  eHealthPlatform,  MonitoringNode,  DevBackendNode (Dev Deployment),
 MonitoringNode (Pilot Deployment)

Visible on diagrams: fig. A.4

E.1.15 Av2MonitoringService

Responsibility: Central observability and recovery-orchestration component. Detects (a) missing sensor data, (b) internal subsystem crashes, (c) SLA violations of the telecom channel; classifies failures by affected patient risk level; notifies administrators and patient contacts within Av2 deadlines; records communication-gap durations; and coordinates recovery with the RecoverySyncService and RedeploymentCoordinator.

Super-components: None

Sub-components:  Av2MissingDataDetector,  Av2InternalFailureDetector,  Av2FailureClassifier,  Av2StakeholderNotifier,  Av2RecoveryTracker,  Av2SLAMonitor,  Av2CommunicationGapLog

Provided interfaces:  Av2NotificationDispatch,  Av2RecoveryCoordination,  Av2SLAReporting

Required interfaces:  Av2AvailabilityMonitoring,  Av2CommunicationGatewayHealthCheck,  Av2MessageBrokerHealthCheck,  Av2PatientContactLookup

Deployed on:  eHealthPlatform,  MonitoringNode,  DevBackendNode (Dev Deployment),  MonitoringNode (Pilot Deployment)

Visible on diagrams: figs. A.2, A.4, C.1, C.2, C.3 and C.4

E.1.16 Av2PatientGateway

Responsibility: The patient-facing edge component running on the wearable/handheld device. Acquires raw sensor readings, transmits them to the backend CommunicationGateway over the primary telecom channel, monitors device battery and network health, manages local buffering during connectivity loss, and dispatches emergency notifications via the backup channel when the primary channel is unavailable. It is the only component that physically resides with the patient.

Super-components: None

Sub-components: None

Provided interfaces:  Av2EmergencyDispatch,  Av2GatewayDataIngress

Required interfaces:  Av2BackupEmergencyIngress,  Av2DeliveryAck,  Av2GatewayResyncCommand

Deployed on:  GatewayNode,  DevGatewayNode (Dev Deployment),  GatewayNode (Pilot Deployment),  PatientGatewayDevice

Visible on diagrams: figs. A.2, B.2, C.1, C.2, C.3 and C.4

E.1.17 Av2RecoverySyncService

Responsibility: Coordinates post-failure recovery. When RecoveryTracker signals a failure has been resolved, RecoverySyncService (a) drains any sensor data queued in the MessageBroker that arrived during degraded mode, and (b) instructs the affected PatientGateway(s) to flush their LocalBufferingRepository through a GatewayResyncCommand.

Super-components: None

Sub-components: None

Provided interfaces:  Av2GatewayResyncCommand

Required interfaces:  Av2QueuedSensorDataMgmt,  Av2RecoveryCoordination

Deployed on:  eHealthPlatform,  MonitoringNode,  DevBackendNode (Dev Deployment),  MonitoringNode (Pilot Deployment)

Visible on diagrams: figs. A.2, C.1, C.2, C.3 and C.4

E.1.18 Av2RecoveryTracker

Responsibility: Tracks the lifecycle of active failures from detection through resolution. It receives **ClassifiedFailure** signals via EscalationEvents when a failure is detected and RecoveryStatus signals from the FailureClassifier when a failure is resolved. It records the failure start time, detection time, and resolution time, computes the total communication gap duration, and signals the RecoverySyncService and RedeploymentCoordinator via the RecoveryCoordination interface to begin recovery operations. It also records how long communication was interrupted per the Av2 detection requirement.

Super-components:  Av2MonitoringService

Sub-components: None

Provided interfaces:  Av2RecoveryCoordination

Required interfaces:  Av2CommunicationGapHistory,  Av2EscalationEvents,  Av2RecoveryStatus

Deployed on:  eHealthPlatform,  MonitoringNode,  DevBackendNode (Dev Deployment),  MonitoringNode (Pilot Deployment)

Visible on diagrams: fig. A.4

E.1.19 Av2RedeploymentCoordinator

Responsibility: Enables the system administrator to trigger redeployment or rollback of the CommunicationGateway within 5 minutes. Monitors rollback progress and confirms success.

Super-components: None

Sub-components: None

Provided interfaces: $\rightarrow$ Av2RedeploymentControl

Required interfaces: $\leftarrow$ Av2RecoveryCoordination

Deployed on: ☐ eHealthPlatform, ☐ AdminNode, ☐ AdminNode (Pilot Deployment), ☐ DevBackendNode (Dev Deployment)

Visible on diagrams: figs. A.2, C.1, C.2, C.3 and C.4

E.1.20 Av2RequestReceiver

Responsibility: Receives raw inbound transmissions from patient gateways via the GatewayDataIngress interface, performs initial parsing and format validation, and forwards validated inbound data to the AuthenticationValidator via the InboundGatewayData interface. It also extracts the gateway identifier and receipt timestamp from each transmission and records them via the HeartbeatUpdate interface to the HeartbeatTracker, ensuring that every received transmission is counted as a heartbeat regardless of whether it passes authentication.

Super-components: ☒ Av2CommunicationGateway

Sub-components: None

Provided interfaces: $\rightarrow$ Av2HeartbeatUpdate, $\rightarrow$ Av2InboundGatewayData

Required interfaces: $\leftarrow$ Av2GatewayDataIngress

Deployed on: ☐ BackendGatewayNode, ☐ BackendGatewayNode (Pilot Deployment), ☐ DevBackendNode (Dev Deployment)

Visible on diagrams: fig. A.3

E.1.21 Av2RetryCoordinator

Responsibility: Coordinates retry attempts for messages that the RoutingManager failed to deliver to downstream backend services. It receives **FailureEvent** objects from the FailureDetector via FailureEvents, determines whether a retry is appropriate, and schedules retries using exponential backoff. It publishes retry-ready messages via RetryDispatch to the RoutingManager. It cancels retries once delivery is confirmed or the failure has persisted long enough to require escalation to the MonitoringService.

Super-components: ☒ Av2CommunicationGateway

Sub-components: None

Provided interfaces: $\rightarrow$ Av2RetryDispatch

Required interfaces: $\leftarrow$ Av2FailureEvents

Deployed on: ☐ BackendGatewayNode, ☐ BackendGatewayNode (Pilot Deployment), ☐ DevBackendNode (Dev Deployment)

Visible on diagrams: fig. A.3

E.1.22 Av2RoutingManager

Responsibility: Routes authenticated and validated gateway transmissions to the appropriate downstream service. Sensor data packages are forwarded to the MessageBroker via BufferedDataDispatch. Emergency notifications are forwarded to the EmergencyNotificationService. It also accepts retry-ready messages from the RetryCoordinator via RetryDispatch and re-attempts delivery of previously failed transmissions. It is the only module inside the CommunicationGateway that communicates with downstream backend services.

Super-components: ☒ Av2CommunicationGateway

Sub-components: None

Provided interfaces: $\rightarrow$ Av2BufferedDataDispatch, $\rightarrow$ Av2RouteFailureSignals

Required interfaces: $\leftarrow$ Av2RetryDispatch, $\leftarrow$ Av2ValidatedTransmission

Deployed on: ☐ BackendGatewayNode, ☐ BackendGatewayNode (Pilot Deployment), ☐ DevBackendNode (Dev Deployment)

Visible on diagrams: fig. A.3

E.1.23 Av2SensorDataRepository

Responsibility: Persistent store for incoming sensor data packages and their arrival timestamps. Provides the SensorDataMgmt interface for appending new readings and for querying historical sensor data needed by clinical models that require historical context.

Super-components: None

Sub-components: None

Provided interfaces: ∅ SensorDataMgmt

Required interfaces: None

Deployed on: ☐ DevPatientRecordsNode (Dev Deployment), ☐ PatientRecordsNode (Pilot Deployment), ☐ PatientRecordsNode

Visible on diagrams: figs. A.2, C.2, C.3 and C.4

E.1.24 Av2SLAMonitor

Responsibility: Aggregates per-gateway connectivity events and computes rolling availability metrics (per year), bandwidth samples, and coverage estimates. Compares against the SLA thresholds (99.5% availability, 80% coverage, 64Kb/s bandwidth) and raises a **FailureEvent** of type SLA_VIOLATION when thresholds are breached over the measurement window.

Super-components: ☒ Av2MonitoringService

Sub-components: None

Provided interfaces: ∅ Av2DetectedFailures, ∅ Av2SLAReporting

Required interfaces: < Av2AvailabilityMonitoring

Deployed on: ☐ eHealthPlatform, ☐ MonitoringNode, ☐ DevBackendNode (Dev Deployment), ☐ MonitoringNode (Pilot Deployment)

Visible on diagrams: figs. A.4 and C.1

E.1.25 Av2StakeholderNotifier

Responsibility: Receives **ClassifiedFailure** objects via EscalationEvents and dispatches notifications to the appropriate stakeholders before the deadline encoded in each **ClassifiedFailure**. For INTERNAL_SUBSYSTEM_CRASH and COMMUNICATION_CHANNEL_DOWN failures it notifies the system administrator. For GATEWAY_UNREACHABLE and MISSING_SENSOR_DATA failures it notifies both the system administrator and initiates patient contact. It uses the NotificationDispatch interface to send the actual notifications. It must meet the notification deadline in 90% of cases as required by Av2.

Super-components: ☒ Av2MonitoringService

Sub-components: None

Provided interfaces: ∅ Av2NotificationDispatch

Required interfaces: < Av2EscalationEvents, < Av2PatientContactLookup

Deployed on: ☐ eHealthPlatform, ☐ MonitoringNode, ☐ DevBackendNode (Dev Deployment), ☐ MonitoringNode (Pilot Deployment)

Visible on diagrams: figs. A.4 and C.1

E.1.26 ClinicalJobCreator

Responsibility: The ClinicalJobCreator is responsible for constructing the **ClinicalModelJob** objects and populating them with the necessary patient health data for execution by the RiskEstimationProcessors.

Super-components: ☒ eHealth Platform

Sub-components: None

Provided interfaces: ∅ ClinicalJobMgmt

Required interfaces: < FetchClinicalModels, < OtherDataMgmt, < SensorDataMgmt

Deployed on: ☐ RiskEstimationManagementNode (Pilot Deployment), ☐ RiskEstimationManagementNode (Dev Deployment), ☐ RiskEstimationMgmtNode

Visible on diagrams: figs. A.2, C.2, C.3, C.4 and D.1

E.1.27 ClinicalModelCache

Responsibility: The `ClinicalModelCache` is a read-through cache responsible for caching the **ClinicalModels** that should be evaluated for a certain patient and their configurations. This cache is located close to the `RiskEstimationProcessor` and `RiskEstimationCombiner` in order to improve the latency of the whole risk estimation flow. The items in the `ClinicalModelCache` do not expire over time, but should be invalidated explicitly if needed. Cached models remain available for the `RiskEstimationProcessor` while updates are made in the `ClinicalModelDB`.

Super-components:  eHealth Platform

Sub-components: None

Provided interfaces:  `ClinicalModelCacheMgmt`,  `FetchClinicalModels`

Required interfaces:  `ClinicalModelStorage`,  `OtherDataMgmt`

Deployed on:  `RiskEstimationManagementNode` (Pilot Deployment),  `RiskEstimationManagementNode` (Dev Deployment),  `RiskEstimationMgmtNode`

Visible on diagrams: figs. A.2, C.2, C.3, C.4 and D.1

E.1.28 ClinicalModelDB

Responsibility: The `ClinicalModelDB` stores all the **ClinicalModels** and the configurations of these **ClinicalModels** for the different patients separately from other data. It only allows appending new **ClinicalModels**.

Super-components:  eHealth Platform

Sub-components: None

Provided interfaces:  `ClinicalModelStorage`,  `P1ClinicalConfigMgmt`

Required interfaces: None

Deployed on:  `TODO Node` (Pilot Deployment),  `RiskEstimationManagementNode` (Dev Deployment),  `TODONode`

Visible on diagrams: figs. A.2, C.2, C.3, C.4 and D.1

E.1.29 eHealth Platform

Responsibility: The parent `eHealth Platform` component that represents the context boundary in the client-server view.

Super-components: None

Sub-components:  `OtherFunctionality`,  `MLModelManager`,  `ClinicalModelCache`,  `RiskEstimationCombiner`,  `RiskEstimationProcessor`,  `RiskEstimationScheduler`,  `ClinicalJobCreator`,  `ClinicalModelDB`

Provided interfaces: None

Required interfaces:  `KVStore`,  `MLaaSService`

Deployed on:  `TODO Node` (Pilot Deployment),  `RiskEstimationManagementNode` (Pilot Deployment),  `RiskEstimationProcessorNode` (Pilot Deployment),  `MLaaS Conn` (Pilot Deployment),  `RiskEstimationManagementNode` (Dev Deployment),  `RiskEstimationProcessorNode` (Dev Deployment),  `MLaaSConnectorNode`,  `RiskEstimationMgmtNode`,  `RiskEstimationProcessorNode`,  `TODONode`

Visible on diagrams: figs. A.1 and B.1

E.1.30 HIS

Responsibility: Hospital Information System

Super-components: None

Sub-components: None

Provided interfaces:  `healthAPI`

Required interfaces: None

Deployed on:  `HISServer`

Visible on diagrams: fig. C.1

E.1.31 KVStorageService

Responsibility: Service for key-value storage of data.

Super-components: None

Sub-components: None

Provided interfaces: $\rightarrow$ KVStore

Required interfaces: None

Deployed on: ☐ RiskEstimationProcessorNode (Pilot Deployment), ☐ RiskEstimationProcessorNode (Dev Deployment), ☐ RiskEstimationProcessorNode

Visible on diagrams: figs. A.1, C.2, C.3 and C.4

E.1.32 MLaaS

Responsibility: MachineLearning-as-a-Service cloud provider. This cloud service provides APIs for managing different **MLModels** and exchanging **MLModel** updates with other parties.

Super-components: None

Sub-components: None

Provided interfaces: $\rightarrow$ MLaaSService

Required interfaces: None

Deployed on: ☐ MLaaSCloud, ☐ MLaaS Service (Pilot Deployment), ☐ MLaaS Service (Dev Deployment)

Visible on diagrams: figs. A.1, C.1, C.3 and C.4

E.1.33 MLModelManager

Responsibility: This component is responsible for managing the local **MLModels** for making predictions for malignant events based on the incoming sensor data. It keeps track of the available **MLModels** that can be used and supports exchanging **MLModel** updates with other systems using the APIs of MLaaS providers.

Super-components: ☒ eHealth Platform

Sub-components: None

Provided interfaces: $\rightarrow$ MLModelMgmt

Required interfaces: $\leftarrow$ MLaaSService

Deployed on: ☐ MLaaS Conn (Pilot Deployment), ☐ RiskEstimationManagmentNode (Dev Deployment), ☐ MLaaSConnectorNode

Visible on diagrams: figs. A.1, A.2, B.3, C.1, C.2, C.3 and C.4

E.1.34 OtherFunctionality

Responsibility: Other functionality that still has to be worked out.

Super-components: ☒ eHealth Platform

Sub-components: None

Provided interfaces: None

Required interfaces: None

Deployed on: ☐ TODO Node (Pilot Deployment), ☐ RiskEstimationManagmentNode (Dev Deployment), ☐ TODONode

Visible on diagrams: figs. C.2, C.3, C.4, D.1, D.5 and D.6

E.1.35 P1CommandRouter

Responsibility: provides IPhysicianCommand at the component boundary; thin dispatcher that routes each operation to the matching handler.

Super-components: ☒ P1PhysicianCommandService

Sub-components: None

Provided interfaces: $\rightarrow$ P1IPhysicianCommand

Required interfaces: $\leftarrow$ P1IConfigCommand, $\leftarrow$ P1IOnDemandCommand, $\leftarrow$ P1IRiskLevelCommand

Deployed on: ☐ eHealthPlatform, ☐ DevBackendNode (Dev Deployment), ☐ PhysicianNode, ☐ PhysicianNode (Pilot Deployment)

Visible on diagrams: fig. A.7

E.1.36 P1ConfigurationHandler

Responsibility: UC7, overwrites the per-patient **ClinicalModelConfiguration**, then invalidates the clinical-model cache.

Super-components:  P1PhysicianCommandService

Sub-components: None

Provided interfaces:  P1IConfigCommand

Required interfaces:  ClinicalModelCacheMgmt,  P1ClinicalConfigMgmt

Deployed on:  eHealthPlatform,  DevBackendNode (Dev Deployment),  PhysicianNode,  PhysicianNode (Pilot Deployment)

Visible on diagrams: fig. A.7

E.1.37 P1CorrelationTracker

Responsibility: mints CorrelationIds for UC9 requests and matches incoming RiskEvents back to the originating handler; the single locus an Av2 failover must preserve so UC9 result delivery survives.

Super-components:  P1PhysicianCommandService

Sub-components: None

Provided interfaces:  P1ICorrelationTracking

Required interfaces: None

Deployed on:  eHealthPlatform,  DevBackendNode (Dev Deployment),  PhysicianNode,  PhysicianNode (Pilot Deployment)

Visible on diagrams: fig. A.7

E.1.38 P1DurableNotificationLog

Responsibility: write-ahead log; red and yellow notifications are persisted here before being acked from IRiskEvents so they survive a crash.

Super-components:  P1NotificationDispatcher

Sub-components: None

Provided interfaces:  P1INotificationLog

Required interfaces: None

Deployed on:  eHealthPlatform,  DevBackendNode (Dev Deployment),  PhysicianNode,  PhysicianNode (Pilot Deployment)

Visible on diagrams: fig. A.5

E.1.39 P1EHRProxyModule

Responsibility: fronts HIS for EHR reads and writes via PatientRecordMgmt; stale-tolerant cache on reads, invalidated structurally on writes.

Super-components:  P1PatientDataService

Sub-components: None

Provided interfaces:  PatientRecordMgmt

Required interfaces:  P1IHISAccess

Deployed on:  PatientDataNode,  PatientDataNode (Dev Deployment),  PatientDataNode (Pilot Deployment)

Visible on diagrams: fig. A.6

E.1.40 P1HISAdapter

Responsibility: outbound adapter for the external HIS healthAPI; used by EHRProxyModule for UC16 reads and UC17 writes.

Super-components: None

Sub-components: None

Provided interfaces:  P1IHISAccess

Required interfaces:  healthAPI

Deployed on:  PatientDataNode,  PatientDataNode (Dev Deployment),  PatientDataNode (Pilot Deployment)

Visible on diagrams: figs. A.2, C.2, C.3 and C.4

E.1.41 P1NotificationDispatcher

Responsibility: UC5 dispatcher; subscribes to IRiskEvents; red > yellow > green priority queue; drops green first under overload; persists red/yellow so they cannot be lost.

Super-components: None

Sub-components:  P1NotificationInboxModule,  P1PriorityBuffer,  P1DurableNotificationLog,  P1RecipientRegistry,  P1RiskEventSubscriber

Provided interfaces:  P1INotificationInbox

Required interfaces:  Av2EmergencyDispatch,  P1IRiskEvents

Deployed on:  eHealthPlatform,  DevBackendNode (Dev Deployment),  PhysicianNode,  PhysicianNode (Pilot Deployment)

Visible on diagrams: figs. A.2, A.5, C.1, C.2, C.3 and C.4

E.1.42 P1NotificationInboxModule

Responsibility: provides INotificationInbox at the component boundary; owns per-physician subscription state and the pull surface used by PhysicianGateway (getPendingNotifications / acknowledgeNotification).

Super-components:  P1NotificationDispatcher

Sub-components: None

Provided interfaces:  P1INotificationInbox

Required interfaces:  P1INotificationLog,  P1IPriorityBuffer,  P1IRecipientRegistry

Deployed on:  eHealthPlatform,  DevBackendNode (Dev Deployment),  PhysicianNode,  PhysicianNode (Pilot Deployment)

Visible on diagrams: figs. A.5 and C.1

E.1.43 P1OnDemandConsultationHandler

Responsibility: UC9 orchestration, mints a **CorrelationId** via CorrelationTracker, fetches sensor data, stores it with ‘triggerEstimation=false’, launches a priority risk job carrying the **CorrelationId**, then waits for the matching IRiskEvents.

Super-components:  P1PhysicianCommandService

Sub-components: None

Provided interfaces:  P1IOnDemandCommand

Required interfaces:  LaunchRiskEstimation,  P1ICorrelationTracking,  P1IOnDemandSensorFetch,  P1IRiskEvents,  SensorDataMgmt

Deployed on:  eHealthPlatform,  DevBackendNode (Dev Deployment),  PhysicianNode,  PhysicianNode (Pilot Deployment)

Visible on diagrams: fig. A.7

E.1.44 P1PatientDataService

Responsibility: owns raw sensor data, patient status, and the HIS EHR proxy; takes over the three interfaces OtherFunctionality exposed.

TODO decomposition

Super-components: None

Sub-components:  P1SensorIngestModule,  P1PatientStatusModule,  P1EHRProxyModule

Provided interfaces:  OtherDataMgmt,  P1IRiskEvents,  PatientRecordMgmt,  SensorDataMgmt

Required interfaces:  LaunchRiskEstimation,  P1IHISAccess

Deployed on:  PatientDataNode,  PatientDataNode (Dev Deployment),  PatientDataNode (Pilot Deployment)

Visible on diagrams: figs. A.2, A.6, C.2, C.3 and C.4

E.1.45 P1PatientGatewayCommander

Responsibility: outbound channel from PMS to patient gateways for UC9 on-demand sensor-data fetches; synchronous with a bounded timeout.

Super-components: None

Sub-components: None

Provided interfaces: $\rightarrow$ P1OnDemandSensorFetch

Required interfaces: None

Deployed on: $\square$ DevBackendNode (Dev Deployment), $\square$ PhysicianNode, $\square$ PhysicianNode (Pilot Deployment)

Visible on diagrams: figs. A.2, C.2, C.3 and C.4

E.1.46 P1PatientQueryService

Responsibility: handles UC6 patient-status reads; tiered priority queue (high > medium > low) feeds the 2/5/10 s SLA tiers.

Super-components: None

Sub-components: None

Provided interfaces: $\rightarrow$ P1PatientQuery

Required interfaces: $\leftarrow$ P1PatientStatusRead

Deployed on: $\square$ eHealthPlatform, $\square$ DevBackendNode (Dev Deployment), $\square$ PhysicianNode, $\square$ PhysicianNode (Pilot Deployment)

Visible on diagrams: figs. A.2, C.1, C.2, C.3 and C.4

E.1.47 P1PatientStatusCache

Responsibility: read-through cache for patient status; pins high-risk patients to bound cross-node traffic.

Super-components: None

Sub-components: None

Provided interfaces: $\rightarrow$ P1PatientStatusRead

Required interfaces: $\leftarrow$ OtherDataMgmt

Deployed on: $\square$ PatientDataNode, $\square$ PatientDataNode (Dev Deployment), $\square$ PatientDataNode (Pilot Deployment)

Visible on diagrams: figs. A.2, C.2, C.3 and C.4

E.1.48 P1PatientStatusModule

Responsibility: owns patient-status records; fires IRiskEvents when setEstimatedPatientStatus actually changes the stored status.

Super-components: $\boxplus$ P1PatientDataService

Sub-components: None

Provided interfaces: $\rightarrow$ OtherDataMgmt, $\rightarrow$ P1IRiskEvents

Required interfaces: None

Deployed on: $\square$ PatientDataNode, $\square$ PatientDataNode (Dev Deployment), $\square$ PatientDataNode (Pilot Deployment)

Visible on diagrams: fig. A.6

E.1.49 P1PhysicianCommandService

Responsibility: handles physician write/command flows: UC7 (configure risk assessment), UC8 (update risk level), UC9 (on-demand consultation).

Super-components: None

Sub-components: $\boxplus$ P1CommandRouter, $\boxplus$ P1ConfigurationHandler, $\boxplus$ P1RiskLevelHandler, $\boxplus$ P1OnDemandConsultationHandler, $\boxplus$ P1CorrelationTracker

Provided interfaces: $\rightarrow$ P1PhysicianCommand

Required interfaces: $\leftarrow$ ClinicalModelCacheMgmt, $\leftarrow$ LaunchRiskEstimation, $\leftarrow$ P1ClinicalConfigMgmt, $\leftarrow$ P1OnDemandSensorFetch, $\leftarrow$ P1IRiskEvents, $\leftarrow$ PatientRecordMgmt, $\leftarrow$ SensorDataMgmt

Deployed on: $\square$ eHealthPlatform, $\square$ DevBackendNode (Dev Deployment), $\square$ PhysicianNode, $\square$ PhysicianNode (Pilot Deployment)

Visible on diagrams: figs. A.2, A.7, C.1, C.2, C.3 and C.4

E.1.50 P1PhysicianGateway

Responsibility: single external entry point for physicians; thin router with no business logic.

Super-components: None

Sub-components: None

Provided interfaces: $\rightarrow$ P1IPhysicianAPI

Required interfaces: $\leftarrow$ P1INotificationInbox, $\leftarrow$ P1IPatientQuery, $\leftarrow$ P1IPhysicianCommand

Deployed on: ☐ DevBackendNode (Dev Deployment), ☐ PhysicianNode, ☐ PhysicianWorkstation,
☐ PhysicianNode (Pilot Deployment)

Visible on diagrams: figs. A.2, C.1, C.2, C.3 and C.4

E.1.51 P1PriorityBuffer

Responsibility: in-memory priority queue (red > yellow > green); enforces the 100/min threshold and drops green first under overload.

Super-components: ☒ P1NotificationDispatcher

Sub-components: None

Provided interfaces: $\rightarrow$ P1IPriorityBuffer

Required interfaces: None

Deployed on: ☐ eHealthPlatform, ☐ DevBackendNode (Dev Deployment), ☐ PhysicianNode,
☐ PhysicianNode (Pilot Deployment)

Visible on diagrams: fig. A.5

E.1.52 P1RecipientRegistry

Responsibility: maps a patient-status event to the physicians who should be notified about it; UC5's "looks up the appropriate recipients" lives here. Populated via subscribeToNotifications on INotificationInbox.

Super-components: ☒ P1NotificationDispatcher

Sub-components: None

Provided interfaces: $\rightarrow$ P1IRecipientRegistry

Required interfaces: None

Deployed on: ☐ eHealthPlatform, ☐ DevBackendNode (Dev Deployment), ☐ PhysicianNode,
☐ PhysicianNode (Pilot Deployment)

Visible on diagrams: fig. A.5

E.1.53 P1RiskEventSubscriber

Responsibility: requires IRiskEvents at the component boundary; on event, queries RecipientRegistry, persists red/yellow to DurableNotificationLog, then enqueues to PriorityBuffer.

Super-components: ☒ P1NotificationDispatcher

Sub-components: None

Provided interfaces: None

Required interfaces: $\leftarrow$ P1INotificationLog, $\leftarrow$ P1IPriorityBuffer, $\leftarrow$ P1IRecipientRegistry,
 $\leftarrow$ P1IRiskEvents

Deployed on: ☐ eHealthPlatform, ☐ DevBackendNode (Dev Deployment), ☐ PhysicianNode,
☐ PhysicianNode (Pilot Deployment)

Visible on diagrams: fig. A.5

E.1.54 P1RiskLevelHandler

Responsibility: UC8, writes the updated risk level via PatientRecordMgmt; the EHR forward is structural through EHRProxyModule.

Super-components: ☒ P1PhysicianCommandService

Sub-components: None

Provided interfaces: $\rightarrow$ P1IRiskLevelCommand

Required interfaces: $\leftarrow$ PatientRecordMgmt

Deployed on: ☐ eHealthPlatform, ☐ DevBackendNode (Dev Deployment), ☐ PhysicianNode,
☐ PhysicianNode (Pilot Deployment)

Visible on diagrams: fig. A.7

E.1.55 P1SensorIngestModule

Responsibility: accepts and persists raw sensor data; triggers risk estimation unless caller opts out via ‘triggerEstimation=false’ (used by UC9).

Super-components:  P1PatientDataService

Sub-components: None

Provided interfaces:  SensorDataMgmt

Required interfaces:  LaunchRiskEstimation

Deployed on:  PatientDataNode,  PatientDataNode (Dev Deployment),  PatientDataNode (Pilot Deployment)

Visible on diagrams: fig. A.6

E.1.56 PatientManagementService

Responsibility: Provides administrative operations for managing patient records (enrolment, demographic updates, assigned clinical models, risk-level changes). It is the write-side service over the PatientRecordRepository and the source of truth for patient-related configuration consumed by other backend components.

Super-components: None

Sub-components: None

Provided interfaces: None

Required interfaces: None

Deployed on:  DevPatientRecordsNode (Dev Deployment),  PatientRecordsNode (Pilot Deployment),  PatientRecordsNode

Visible on diagrams: figs. A.2, C.2, C.3 and C.4

E.1.57 PatientRecordRepository

Responsibility: Persistent store for patient demographic data, electronic health record references, assigned clinical models, current risk level, and authentication credentials. Provides the PatientRecordMgmt, PatientStatusOverview, and UserAuthentication interfaces. Acts as the authoritative source of patient identity and risk classification within the PMS backend.

Super-components: None

Sub-components: None

Provided interfaces:  Av2PatientContactLookup,  Av2PatientStatusOverview,  UserAuthentication

Required interfaces: None

Deployed on:  DevPatientRecordsNode (Dev Deployment),  PatientRecordsNode (Pilot Deployment),  PatientRecordsNode

Visible on diagrams: figs. A.2, C.2, C.3 and C.4

E.1.58 PatientStatusService

Responsibility: Exposes read-only patient status information (current risk level, recent risk-assessment outcomes) to other services and user-facing components. Reads from the PatientRecordRepository via the PatientStatusOverview interface.

Super-components: None

Sub-components: None

Provided interfaces: None

Required interfaces:  Av2PatientStatusOverview

Deployed on:  PatientRecordsNode

Visible on diagrams: figs. A.2 and C.2

E.1.59 RiskEstimationCombiner

Responsibility: The RiskEstimationCombiner is responsible for combining the results of the ClinicalModelJobs belonging to a patient risk estimation. More precisely, the RiskEstimationScheduler passes the set of the scheduled jobs for a risk estimation to the RiskEstimationCombiner before scheduling them. The RiskEstimationCombiner then waits for

all results to arrive, combines them, and propagates the new sensor data and the results of the risk estimation to the patient record if needed.

Super-components:  eHealth Platform

Sub-components: None

Provided interfaces: $\rightarrow$ JobMgmt, $\rightarrow$ Results

Required interfaces: $\leftarrow$ OtherDataMgmt, $\leftarrow$ PatientRecordMgmt

Deployed on:  RiskEstimationManagementNode (Pilot Deployment),  RiskEstimationManagementNode (Dev Deployment),  RiskEstimationMgmtNode

Visible on diagrams: figs. A.2, C.2, C.3, C.4, D.1, D.5 and D.6

E.1.60 RiskEstimationProcessor

Responsibility: The RiskEstimationProcessor is responsible for computing **ClinicalModelJobs**. The RiskEstimationProcessor fetches new **ClinicalModelJobs** from the RiskEstimationScheduler, uses the specified **MLModel** to calculate the risk prediction, and passes the result to the RiskEstimationCombiner. Multiple instances of the RiskEstimationProcessor run in parallel to improve the throughput.

Super-components:  eHealth Platform

Sub-components: None

Provided interfaces: None

Required interfaces: $\leftarrow$ FetchJobs, $\leftarrow$ KVStore, $\leftarrow$ MLModelMgmt, $\leftarrow$ Results

Deployed on:  RiskEstimationProcessorNode (Pilot Deployment),  RiskEstimationProcessorNode (Dev Deployment),  RiskEstimationProcessorNode

Visible on diagrams: figs. A.1, A.2, B.4, C.2, C.3, C.4 and D.6

E.1.61 RiskEstimationScheduler

Responsibility: The RiskEstimationScheduler is responsible for taking in new and scheduling requests for risk estimations. It keeps track of the throughput of incoming jobs and changes the scheduling from first-in/first-out to dynamic priority earliest deadline first when going into overload modus.

Super-components:  eHealth Platform

Sub-components: None

Provided interfaces: $\rightarrow$ FetchJobs, $\rightarrow$ LaunchRiskEstimation

Required interfaces: $\leftarrow$ ClinicalJobMgmt, $\leftarrow$ FetchClinicalModels, $\leftarrow$ JobMgmt, $\leftarrow$ OtherDataMgmt, $\leftarrow$ SensorDataMgmt

Deployed on:  RiskEstimationManagementNode (Pilot Deployment),  RiskEstimationManagementNode (Dev Deployment),  RiskEstimationMgmtNode

Visible on diagrams: figs. A.2, C.2, C.3, C.4, D.1 and D.5

E.1.62 UserAuthenticationService

Responsibility: Authenticates human users (administrators, clinicians) accessing PMS interfaces. Verifies credentials against the PatientRecordRepository via the UserAuthentication interface. Distinct from the AuthenticationValidator inside the CommunicationGateway, which authenticates patient-gateway devices, not human users.

Super-components: None

Sub-components: None

Provided interfaces: None

Required interfaces: $\leftarrow$ UserAuthentication

Deployed on:  DevPatientRecordsNode (Dev Deployment),  PatientRecordsNode (Pilot Deployment),  PatientRecordsNode

Visible on diagrams: figs. A.2, C.2, C.3 and C.4

E.2 Modules

E.2.1 Av2BackupChannelManager

Responsibility: Manages the backup communication channel, currently SMS, used for emergency notifications when the primary channel is unavailable. It monitors whether the backup channel is available and dispatches emergency messages through it when the DegradeModeManager reports that the gateway is in degraded mode.

Super-components: ¶ Av2PatientGateway

Super-modules: None

Sub-modules: None

Provided interfaces: ∅ Av2BackupChannelDispatch

Required interfaces: ∅ Av2ConnectivityStatus, ∅ Av2DegradeModeStatus

Visible on diagrams: fig. B.2

E.2.2 Av2BatteryMonitor

Responsibility: Continuously monitors the battery charge level of the patient gateway device. It publishes battery status events and triggers an alert to the PatientAlertService when the charge drops to or below 15%, satisfying the Av2 prevention response measure.

Super-components: ¶ Av2PatientGateway

Super-modules: None

Sub-modules: None

Provided interfaces: ∅ Av2BatteryStatus

Required interfaces: None

Visible on diagrams: fig. B.2

E.2.3 Av2DegradeModeManager

Responsibility: Manages the degraded mode state of the patient gateway. It transitions the gateway into degraded mode when the ServerConnectivityMonitor reports the backend as unreachable, and back to normal mode upon successful reconnection. It records the timestamp at which degraded mode was entered, enabling gap duration tracking. All other modules on the gateway consult DegradeModeStatus to adapt their behavior accordingly.

Super-components: ¶ Av2PatientGateway

Super-modules: None

Sub-modules: None

Provided interfaces: ∅ Av2DegradeModeStatus

Required interfaces: ∅ Av2ServerConnectionStatus

Visible on diagrams: fig. B.2

E.2.4 Av2EmergencyNotificationService

Responsibility: Handles the dispatch of emergency notifications from the patient gateway. In normal operation it uses the primary communication channel. When the gateway is in degraded mode it delegates to the BackupChannelManager to send the emergency notification via the backup channel, ensuring no emergency alerts are lost due to primary channel failure.

Super-components: ¶ Av2PatientGateway

Super-modules: None

Sub-modules: None

Provided interfaces: ∅ Av2EmergencyDispatch

Required interfaces: ∅ Av2BackupChannelDispatch, ∅ Av2ConnectivityStatus, ∅ Av2DegradeModeStatus

Visible on diagrams: fig. B.2

E.2.5 Av2LocalBufferingRepository

Responsibility: Persistently stores sensor data packages and their timestamps on the patient gateway during periods when the primary communication channel is unavailable. It guarantees storage capacity for at least 24 hours of sensor data at the configured transmission rate, satisfying the Av2

resolution response measure. It exposes operations for writing buffered readings and reading them back for delivery upon reconnection.

Super-components: ④ Av2PatientGateway

Super-modules: None

Sub-modules: None

Provided interfaces: ⊖ Av2BufferedSensorData

Required interfaces: None

Visible on diagrams: fig. B.2

E.2.6 Av2NetworkConnectivityMonitor

Responsibility: Continuously monitors the availability and quality of the network connection on the patient gateway. It publishes connectivity status events indicating signal strength and network type, enabling the PatientAlertService to warn the patient of poor or absent connectivity.

Super-components: ④ Av2PatientGateway

Super-modules: None

Sub-modules: None

Provided interfaces: ⊖ Av2ConnectivityStatus

Required interfaces: None

Visible on diagrams: fig. B.2

E.2.7 Av2PatientAlertService

Responsibility: Presents warnings to the patient on the gateway device. It alerts the patient when the battery drops to or below 15% and when network connectivity is absent or insufficient. These warnings fulfill the Av2 prevention response measures for battery and connectivity.

Super-components: ④ Av2PatientGateway

Super-modules: None

Sub-modules: None

Provided interfaces: None

Required interfaces: ⊖ Av2BatteryStatus, ⊖ Av2ConnectivityStatus

Visible on diagrams: fig. B.2

E.2.8 Av2RetryAndSynchronizationService

Responsibility: Implements the exponential backoff retry strategy for delivering buffered sensor data to the PMS backend after a communication failure. It reads pending data from the LocalBufferingRepository and retries delivery with increasing intervals until the backend confirms receipt, at which point it clears the delivered entries from the buffer.

Super-components: ④ Av2PatientGateway

Super-modules: None

Sub-modules: None

Provided interfaces: None

Required interfaces: ⊖ Av2BufferedSensorData, ⊖ Av2DegradeModeStatus, ⊖ Av2DeliveryAck, ⊖ Av2GatewayResyncCommand, ⊖ Av2ServerConnectionStatus

Visible on diagrams: fig. B.2

E.2.9 Av2SensorCommunicator

Responsibility: Acquires raw sensor readings from the wearable sensors attached to the patient gateway via the SensorDataAcquisition interface and places them into the DataBuffer for processing and transmission.

Super-components: ④ Av2PatientGateway

Super-modules: None

Sub-modules: None

Provided interfaces: ⊖ DataBuffer

Required interfaces: ⊖ Av2SensorDataAcquisition

Visible on diagrams: fig. B.2

E.2.10 Av2SensorDataTransmitter

Responsibility: Reads sensor data from the `DataBuffer` provided by the `SensorCommunicator` and transmits it to the `CommunicationGateway`. In normal mode it transmits immediately. In degraded mode it hands data to the `LocalBufferingRepository` instead, and upon recovery delegates flushing to the `RetryAndSynchronizationService`.

Super-components:  `Av2PatientGateway`

Super-modules: None

Sub-modules: None

Provided interfaces:  `Av2GatewayDataIngress`

Required interfaces:  `Av2BufferedSensorData`,  `Av2DegradeModeStatus`,  `DataBuffer`

Visible on diagrams: fig. B.2

E.2.11 Av2ServerConnectivityMonitor

Responsibility: Monitors the reachability of the PMS backend from the patient gateway by periodically sending heartbeat probes. It publishes server connection status events that drive the `DegradeModeManager`'s decision to enter or exit degraded mode.

Super-components:  `Av2PatientGateway`

Super-modules: None

Sub-modules: None

Provided interfaces:  `Av2ServerConnectionStatus`

Required interfaces: None

Visible on diagrams: fig. B.2

E.2.12 JobMLModelSelector

Responsibility: This module is responsible for determining the appropriate **MLModel** to use for making the prediction on the provided sensor data, as there could be multiple **MLModel** candidates for making predictions.

Super-components:  `RiskEstimationProcessor`  `eHealth Platform`

Super-modules: None

Sub-modules: None

Provided interfaces:  `ApplyClinicalModel`

Required interfaces: None

Visible on diagrams: figs. B.4 and D.2

E.2.13 JobProcessor

Responsibility: The `JobProcessor` module coordinates the execution of the **ClinicalModelJobs** to ensure that the required **MLModels** are fetched, and the prediction is performed with the appropriate **MLModel**.

Super-components:  `RiskEstimationProcessor`  `eHealth Platform`

Super-modules: None

Sub-modules: None

Provided interfaces: None

Required interfaces:  `ApplyClinicalModel`,  `FetchJobs`,  `FetchMLModel`,  `Results`,  `SensorData-Analysis`

Visible on diagrams: figs. B.4 and D.2

E.2.14 MLaaSLibrary

Responsibility: MLaaS development library for working with the MLaaS providers. It supports storage & retrieval of **MLModels**, and exchanging **MLModelUpdates** with MLaaS providers.

Super-components: None

Super-modules: None

Sub-modules: None

Provided interfaces:  `MLaaSAPI`

Required interfaces: None

Visible on diagrams: fig. B.1

E.2.15 MLLibrary

Responsibility: Library for working with **MLModels** locally and using them to make predictions based on the incoming sensor data. The library can also process external updates to apply them to a specific **MLModel**.

Super-components: None

Super-modules: None

Sub-modules: None

Provided interfaces: $\rightarrow$ MLAPI

Required interfaces: None

Visible on diagrams: fig. B.1

E.2.16 MLModelRepository

Responsibility: This module stores **MLModels** locally and makes them available for use by the RiskEstimationProcessor when processing **ClinicalModelJobs**.

Super-components: $\boxtimes$ eHealth Platform $\triangleright$ $\boxtimes$ MLModelManager

Super-modules: None

Sub-modules: None

Provided interfaces: $\rightarrow$ MLModelStorage

Required interfaces: None

Visible on diagrams: figs. B.3, D.3 and D.4

E.2.17 MLModelRetrieval

Responsibility: This module is responsible for retrieving the requested **MLModels** from the MLModelManager.

Super-components: $\boxtimes$ RiskEstimationProcessor $\triangleright$ $\boxtimes$ eHealth Platform

Super-modules: None

Sub-modules: None

Provided interfaces: $\rightarrow$ FetchMLModel

Required interfaces: $\leftarrow$ MLModelMgmt

Visible on diagrams: figs. B.4 and D.2

E.2.18 MLModelStorageManager

Responsibility: This module is responsible for managing the locally stored **MLModels**, triggering the synchronization and retrieval of new **MLModels**, and coordinating the updates to the existing **MLModels**.

Super-components: $\boxtimes$ eHealth Platform $\triangleright$ $\boxtimes$ MLModelManager

Super-modules: None

Sub-modules: None

Provided interfaces: $\rightarrow$ MLModelMgmt

Required interfaces: $\leftarrow$ MLModelStorage, $\leftarrow$ MLModelSync

Visible on diagrams: figs. B.3, D.3 and D.4

E.2.19 MLModelUpdateProcessor

Responsibility: The MLModelUpdater processes and applies updates to the local **MLModels** to prevent updates to these models from requiring to retrieve the full **MLModels**.

Super-components: $\boxtimes$ eHealth Platform $\triangleright$ $\boxtimes$ MLModelManager

Super-modules: None

Sub-modules: None

Provided interfaces: $\rightarrow$ MLModelSync

Required interfaces: $\leftarrow$ MLaaSAPI, $\leftarrow$ MLaaSService, $\leftarrow$ MLModelStorage, $\leftarrow$ MLModelUpdateMgmt

Visible on diagrams: figs. B.1, B.3, D.3 and D.4

E.2.20 ModelUpdater

Responsibility: This module is responsible for processing **MLModelUpdates** and applying them to the **MLModel**.

Super-components:  eHealth Platform  MLModelManager

Super-modules: None

Sub-modules: None

Provided interfaces:  MLModelUpdateMgmt

Required interfaces:  MLAPI

Visible on diagrams: figs. B.1, B.3 and D.4

E.2.21 SensorDataRiskAssessment

Responsibility: This module performs the actual risk assessment by using the **MLModel** to make a prediction on the sensor data.

Super-components:  RiskEstimationProcessor  eHealth Platform

Super-modules: None

Sub-modules: None

Provided interfaces:  SensorDataAnalysis

Required interfaces:  KVStore,  MLAPI

Visible on diagrams: figs. B.1, B.4 and D.2

E.3 Interfaces

E.3.1 ApplyClinicalModel

Provided by:  JobMLModelSelector

Required by:  JobProcessor

Operations:

- String determineMLModelForClinicalModel(**ClinicalModel** clinicalModel, **ClinicalModelConfiguration** clinicalModelConfiguration)
 - Effect: Apply the **ClinicalModel** to obtain the most appropriate **MLModel** to use for the calculation.
 - Returns: **Identifier** of the **MLModel** to use.
 - Sequence Diagrams: fig. D.2

Diagrams: None

E.3.2 Av2AuthFailureSignals

Provided by:  Av2AuthenticationValidator

Required by:  Av2FailureDetector

Operations:

- void reportAuthFailure(**GatewayId** gatewayId, string failureReason, **Timestamp** attemptedAt)
 - Effect: Reports a single authentication failure to FailureDetector. The failureReason describes why authentication failed, for example unregistered gateway or invalid credentials. FailureDetector aggregates repeated failures from the same gateway to detect systematic attack or misconfiguration.
 - Sequence Diagrams: None

Diagrams: None

E.3.3 Av2AvailabilityMonitoring

Provided by:  Av2CommunicationGateway,  Av2HeartbeatTracker

Required by:  Av2MissingDataDetector,  Av2MonitoringService,  Av2SLAMonitor

Description : Exposes per-gateway heartbeat data so the MissingDataDetector can identify overdue gateways.

Operations:

- void getExpectedInterval(**GatewayId** gatewayId)

- Effect: Returns the configured expected transmission interval in seconds for the specified gateway. Used by MissingDataDetector to compute the detection deadline: last heartbeat plus expected interval plus five minute tolerance. Returns: The expected transmission interval in seconds as an integer.
- Sequence Diagrams: None
- **void getLastHeartbeat(**GatewayId** gatewayId)**
 - Effect: Returns the timestamp of the most recently received transmission from the specified gateway. Used by MissingDataDetector to determine how long a gateway has been silent and whether the silence exceeds the configured tolerance window. Returns: The **Timestamp** of the last received transmission from the specified gateway, or null if no transmission has ever been received.
 - Sequence Diagrams: None
- **void recordHeartBeat(**GatewayId** gatewayId, **Timestamp** timestamp)**
 - Effect: Records that a transmission was received from the specified gateway at the given timestamp. Updates the last-seen time for that gateway in the HeartbeatTracker. Called by RequestReceiver every time any transmission arrives, regardless of whether it passes authentication, so that connectivity is tracked independently of authentication status.
 - Sequence Diagrams: None
- **void registerGateway(**GatewayId** gatewayId, int expectedIntervalSeconds)**
 - Effect: Registers a new patient gateway in the HeartbeatTracker with its configured expected transmission interval in seconds. Called during patient enrolment so that the HeartbeatTracker immediately begins monitoring for overdue transmissions from that gateway. If a gateway with the given identifier is already registered its interval is updated.
 - Sequence Diagrams: None

Diagrams: None

E.3.4 Av2BackupChannelDispatch

Provided by: ¶ Av2BackupChannelManager

Required by: ¶ Av2EmergencyNotificationService

Description : Published by the BackupChannelManager. Provides operations for sending emergency messages via the backup channel and for querying whether the backup channel is currently available. Consumed by the EmergencyNotificationService module inside the PatientGateway.

Operations:

- **backupChannelType getBackupChannelType()**
 - Effect: Returns the type of backup communication channel currently configured for this patient gateway. Allows EmergencyNotificationService to log which backup channel was used when dispatching an emergency notification.
 - Returns: A **backupChannelType** enum value indicating the backup channel in use, currently always SMS.
 - Sequence Diagrams: None
- **boolean isBackupChannelAvailable()**
 - Effect: Checks whether the backup SMS channel is currently available and capable of transmitting an emergency notification. Called by EmergencyNotificationService before attempting backup channel dispatch.
 - Returns: True if the backup SMS channel is operational and can accept outbound messages. False if the backup channel is also unavailable.
 - Sequence Diagrams: None
- **BackupDeliveryResult sendViaBackupChannel(**EmergencyMessage** message, **GatewayId** gatewayId)**
 - Effect: Transmits the given emergency notification via the backup SMS channel through the telecom network to the SMSSGatewayProvider, which delivers it to the EmergencyBackupReceiver on the PMS backend. Called by EmergencyNotificationService when the gateway is in degraded mode and the primary channel is unavailable.
 - Returns: A **BackupDeliveryResult** indicating whether the SMS was successfully submitted to the telecom network, the timestamp of submission, and the channel used.
 - Sequence Diagrams: None

Diagrams: None

E.3.5 Av2BackupEmergencyIngress

Provided by: ¶ Av2EmergencyBackupReceiver

Required by: ¶ Av2PatientGateway

Operations:

- void receiveEmergencyNotification(**GatewayId** gatewayId, **EmergencyMessage** message, string deviceToken)
 - Effect: Receives an emergency notification transmitted by the patient gateway via the SMS backup channel. Authenticates the sender using the pre-registered device token embedded in the SMS payload. If authentication succeeds, forwards the validated emergency to the MessageBroker. Returns: void.
 - Sequence Diagrams: None

Diagrams: None

E.3.6 Av2BatteryStatus

Provided by: ¶ Av2BatteryMonitor

Required by: ¶ Av2PatientAlertService

Description : Published by the BatteryMonitor. Provides the current battery charge level and supports subscription to threshold-crossing alerts. Consumed by the PatientAlertService and the EmergencyNotificationService module to adapt behavior based on device power state.

Operations:

- int getBatteryLevel()
 - Effect: Returns the current battery charge level of the patient gateway device as a percentage between 0 and 100. Called periodically by PatientAlertService to determine whether the 15% warning threshold has been crossed per Av2 prevention response measures.
 - Returns: An integer between 0 and 100 representing the current battery charge percentage.
 - Sequence Diagrams: None
- boolean isBelowThreshold()
 - Effect: Returns whether the battery charge level has dropped to or below the 15% warning threshold defined by Av2 prevention response measures. PatientAlertService calls this to decide whether to display a low battery warning to the patient.
 - Returns: True if the current battery level is at or below 15%. False if the battery level is above 15%.
 - Sequence Diagrams: None

Diagrams: None

E.3.7 Av2BrokerCommitNotification

Provided by: ¶ Av2MessageBroker

Required by: ¶ Av2AcknowledgementService

Operations:

- void onMessageCommitted(**GatewayId** gatewayId, string messageId, **Timestamp** committedAt)
 - Effect: Notifies the AcknowledgementService that the MessageBroker has durably committed a message. This triggers the AcknowledgementService to emit a DeliveryAck back to the originating PatientGateway so it can clear the entry from its LocalBufferingRepository.
 - Sequence Diagrams: None

Diagrams: None

E.3.8 Av2BufferedDataDispatch

Provided by: ¶ Av2CommunicationGateway, ¶ Av2RoutingManager

Required by: ¶ Av2MessageBroker

Description : Published by the RoutingManager at the CommunicationGateway boundary. Consumed by the MessageBroker. Carries validated and routed sensor data packages from the CommunicationGateway to the MessageBroker queue.

Operations:

- void dispatchEmergencyNotification(**GatewayId** gatewayId, **EmergencyMessage** alert, **Timestamp** receivedAt)

- Effect: Forwards a validated emergency notification from the RoutingManager to the MessageBroker’s durable queue. Emergency notifications are prioritised over regular sensor data in the queue. The MessageBroker durably commits the notification before returning, ensuring it is not lost even if the DataIngestionService is momentarily unavailable.
- Sequence Diagrams: None
- void dispatchSensorData(**GatewayId** gatewayId, **SensorDataPackage** data, **Timestamp** recievedAt)
 - Effect: Forwards a validated and authenticated sensor data package from the RoutingManager to the MessageBroker’s durable queue. The recievedAt timestamp is the server-side time at which the CommunicationGateway received the transmission. The MessageBroker durably commits the package before returning, ensuring no data loss if downstream services are temporarily unavailable.
 - Sequence Diagrams: None

Diagrams: None

E.3.9 Av2BufferedSensorData

Provided by: ¶ Av2LocalBufferingRepository

Required by: ¶ Av2RetryAndSynchronizationService, ¶ Av2SensorDataTransmitter

Description : Published by the LocalBufferingRepository. Provides a to store BufferedData when in DegradMode and a way for to read the pending data when synchronizing again with the backend. Consumed by the SensorDataTransmitter and the RetryAndSynchronizationService.

Operations:

- void clearDeliveredData()
 - Effect: Removes all entries from the LocalBufferingRepository that have been confirmed as delivered by the backend via DeliveryAck. Called by RetryAndSynchronizationService after receiving acknowledgement to prevent redelivery of already-processed data.
 - Sequence Diagrams: None
- List<Tuple<**SensorDataPackage**, **Timestamp**>> getPendingData()
 - Effect: Returns all sensor data packages currently stored in the LocalBufferingRepository awaiting retransmission to the PMS backend. Called by RetryAndSynchronizationService when the gateway reconnects and begins flushing buffered data.
 - Returns: An ordered list of tuples each containing a **SensorDataPackage** and its acquisition timestamp, ordered chronologically from oldest to newest to ensure the backend receives data in the correct order.
 - Sequence Diagrams: None
- boolean isRepositoryFull()
 - Effect: Checks whether the LocalBufferingRepository has reached its maximum storage capacity.
 - Returns: True if the repository is full and cannot store additional sensor data packages without overwriting the oldest entries. False if storage capacity remains available.
 - Sequence Diagrams: None
- void storeBufferedData()
 - Effect: Persists a sensor data package and its acquisition timestamp to the LocalBufferingRepository during degraded mode when the primary channel is unavailable. The repository guarantees storage capacity for at least 24 hours of sensor data at the configured transmission rate per Av2 resolution response measures.
 - Sequence Diagrams: None

Diagrams: None

E.3.10 Av2CommunicationGapHistory

Provided by: ¶ Av2CommunicationGapLog

Required by: ¶ Av2RecoveryTracker

Operations:

- List<**CommunicationGap**> getGapHistory(**GatewayId** gatewayId)
 - Effect: Returns all recorded communication gaps for the specified gateway within the given time period. Used by administrators and compliance processes to review the history of connectivity interruptions for a specific patient gateway.
 - Returns: A list of CommunicationGap records each containing the gateway identifier, the start and end timestamps of the interruption, and the computed duration. Ordered chronologically

from oldest to newest.

– Sequence Diagrams: None

- **void recordGap(GatewayId gatewayId, Timestamp start, Timestamp end)**

– Effect: Persists a completed communication gap record for the specified gateway. Called by RecoveryTracker when a failure is resolved. The gap duration is computed from start and end and stored for compliance reporting and administrator review.

– Sequence Diagrams: None

Diagrams: None

E.3.11 Av2CommunicationGatewayHealthCheck

Provided by: [Av2CommunicationGateway](#), [Av2FailureDetector](#)

Required by: [Av2InternalFailureDetector](#), [Av2MonitoringService](#)

Description : Exposes a health check endpoint on the CommunicationGateway so that the InternalFailureDetector in MonitoringService can poll its operational status at high frequency. The CommunicationGateway aggregates the health of its internal subcomponents — RequestReceiver, AuthenticationValidator, RoutingManager, FailureDetector, RetryCoordinator, HeartbeatTracker — and returns a combined status. A DEGRADED or DOWN status or a failure to respond within one second triggers a **FailureEvent** of type INTERNAL_SUBSYSTEM_CRASH in the InternalFailureDetector.

Operations:

- **HealthStatus ping()**

– Effect: Returns the current top-level operational status of the CommunicationGateway. Returns OK if all subcomponents are functioning normally, DEGRADED if one or more subcomponents are impaired but the gateway is still processing transmissions, DOWN if the gateway cannot process transmissions.

– Returns: the current top-level operational status of the CommunicationGateway

– Sequence Diagrams: None

Diagrams: None

E.3.12 Av2ConnectivityStatus

Provided by: [Av2NetworkConnectivityMonitor](#)

Required by: [Av2BackupChannelManager](#), [Av2EmergencyNotificationService](#), [Av2PatientAlertService](#)

Description : Published by the NetworkConnectivityMonitor. Provides the current network connection state, signal strength, and network type. Consumed by the PatientAlertService, the BackupChannelManager and EmergencyNotificationService.

Operations:

- **SignalStrength getSignalStrength()**

– Effect: Returns the current signal strength of the network connection as measured by the NetworkConnectivityMonitor. Called by PatientAlertService to determine whether to warn the patient about insufficient connectivity.

– Returns: A **SignalStrength** enum value: NONE if no signal, WEAK if signal is insufficient for reliable transmission, MODERATE if functional but potentially lossy, STRONG if reliable connectivity is available.

– Sequence Diagrams: None

- **NetworkType getnetworkType()**

– Effect: Returns the type of network connection currently active on the patient gateway device. Used by SLAMonitor to determine whether the active connection is the cellular network governed by the SLA with the telecom operator.

– Returns: A **NetworkType** enum value: NONE if no connection, CELLULAR if connected via mobile network, WIFI if connected via WiFi.

– Sequence Diagrams: None

- **boolean isConnected()**

– Effect: Returns whether the patient gateway currently has any active network connection. Used by PatientAlertService to trigger a connectivity warning when the patient is completely offline, and by BackupChannelManager to decide if the backup SMS channel should be activated.

- Returns: True if a network connection of any strength is currently active. False if the device has no network connectivity.
- Sequence Diagrams: None

Diagrams: None

E.3.13 Av2DegradeModeStatus

Provided by: ¶ Av2DegradeModeManager

Required by: ¶ Av2BackupChannelManager, ¶ Av2EmergencyNotificationService, ¶ Av2RetryAndSynchronizationService, ¶ Av2SensorDataTransmitter

Description : Published by the DegradeModeManager. Reports whether the gateway is currently in degraded mode, the reason for entering degraded mode, and the timestamp at which degraded mode began. Consumed by the SensorDataTransmitter, RetryAndSynchronizationService, BackupChannelManager, and EmergencyNotificationService module.

Operations:

- void enterDegradeMode(**DegradedModeReason** reason)
 - Effect: Transitions the gateway into degraded mode, recording the reason and the current timestamp as the degraded-since time. Called by DegradeModeManager when ServerConnectivityMonitor reports the backend is unreachable. All components consulting DegradeModeStatus will subsequently receive true from isInDegradeMode.
 - Sequence Diagrams: None
- void exitDegradeMode()
 - Effect: Transitions the gateway back to normal operating mode from degraded mode. Called by DegradeModeManager when ServerConnectivityMonitor reports the backend is reachable again. Clears the degraded-since timestamp.
 - Sequence Diagrams: None
- **Timestamp** getdegradedSince()
 - Effect: Returns the timestamp at which the gateway entered its current degraded mode period. Used by RetryAndSynchronizationService to calculate the total duration of the outage and to determine which buffered readings need to be retransmitted.
 - Returns: The **Timestamp** at which degraded mode was entered. Returns null if the gateway is not currently in degraded mode.
 - Sequence Diagrams: None
- boolean isInDegradeMode()
 - Effect: Returns whether the patient gateway is currently operating in degraded mode. Consulted by SensorDataTransmitter, RetryAndSynchronizationService, BackupChannelManager, and EmergencyNotificationService to adapt their behaviour accordingly.
 - Returns: True if the gateway is currently in degraded mode. False if operating normally.
 - Sequence Diagrams: None

Diagrams: None

E.3.14 Av2DeliveryAck

Provided by: ¶ Av2AcknowledgementService

Required by: ¶ Av2PatientGateway, ¶ Av2RetryAndSynchronizationService

Operations:

- void acknowledgeDelivery(**GatewayId** gatewayId, string messageId, **Timestamp** acknowledgedAt)
 - Effect: Signals to the PatientGateway (and its RetryAndSynchronizationService) that a previously transmitted message has been durably committed by the backend. Upon receipt, the gateway clears the corresponding entry from its LocalBufferingRepository.
 - Sequence Diagrams: None

Diagrams: None

E.3.15 Av2DetectedFailures

Provided by: ¶ Av2InternalFailureDetector, ¶ Av2MissingDataDetector, ¶ Av2SLAMonitor

Required by: ¶ Av2FailureClassifier

Description : Carries **FailureEvent** objects from the MissingDataDetector and InternalFailureDetector to the FailureClassifier. Both detectors provide this interface; the FailureClassifier requires it from both.

Operations:

- List<**FailureEvent**> getActiveFailures()
 - Effect: Returns all currently active failures that have been reported but not yet resolved. Used by FailureClassifier to avoid processing duplicate failure reports for the same ongoing failure event.
 - Returns: A list of all active **FailureEvent** objects currently being tracked, each containing the failure type, affected gateway or subsystem, and detection timestamp.
 - Sequence Diagrams: None
- void reportFailure(**FailureEvent** failure)
 - Effect: Reports a newly detected failure to the FailureClassifier. Called by MissingDataDetector when a gateway exceeds its expected transmission interval plus the five minute tolerance, by InternalFailureDetector when a subsystem health check fails, and by SLAMonitor when an SLA threshold is breached. The FailureClassifier enriches the event with escalation level and notification deadline.
 - Sequence Diagrams: None

Diagrams: None

E.3.16 Av2EmergencyDispatch

Provided by:  Av2EmergencyNotificationService,  Av2PatientGateway

Required by:  P1NotificationDispatcher

Description : Published by the EmergencyNotificationService module inside the PatientGateway. Provides a single operation for sending an emergency notification, routing it via the primary channel in normal mode or the backup channel in degraded mode. Consumed by sensor-level risk detection logic on the gateway.

Operations:

- void sendEmergencyNotification(**EmergencyMessage** alert)
 - Effect: Sends an emergency
 - Sequence Diagrams: None

Diagrams: None

E.3.17 Av2EscalationEvents

Provided by:  Av2FailureClassifier

Required by:  Av2RecoveryTracker,  Av2StakeholderNotifier

Description : Carries **ClassifiedFailure** objects from the FailureClassifier to both the StakeholderNotifier and the RecoveryTracker.

Operations:

- **ClassifiedFailure** classifyFailure(**FailureEvent** failure)
 - Effect: Receives a raw **FailureEvent** from FailureClassifier and returns a **ClassifiedFailure** enriched with the escalation level and notification deadline derived from the affected patient's risk level. High-risk patients produce a deadline of 2 minutes, medium-risk 5 minutes, low-risk 10 minutes, per Av2 resolution response measures.
 - Returns: A **ClassifiedFailure** containing the original **FailureEvent**, the computed **EscalationLevel**, and the **Timestamp** by which the relevant stakeholder must be notified.
 - Sequence Diagrams: None
- **EscalationLevel** getEscalationLevel(**FailureEvent** failure)
 - Effect: Returns the escalation level that would be assigned to the given failure based on the affected patient's current risk level, without creating a full **ClassifiedFailure**. Used by StakeholderNotifier to prioritise notification dispatch when multiple failures are active simultaneously.
 - Returns: The **EscalationLevel** for the failure: HIGH for high-risk patients, MEDIUM for medium-risk, LOW for low-risk.
 - Sequence Diagrams: None

Diagrams: None

E.3.18 Av2FailureEvents

Provided by: ¶ Av2FailureDetector

Required by: ¶ Av2RetryCoordinator

Description : Published by the FailureDetector. Carries connection-level and internal failure notifications to the RetryCoordinator. Only used inside the CommunicationGateway.

Operations:

- List< **ConnectionFailure** > getActiveConnectionFailures()
 - Effect: Returns the list of currently active connection failures being tracked by FailureDetector for which retry attempts are ongoing. Used by RetryCoordinator to avoid scheduling duplicate retries for the same gateway.
 - Returns: A list of the current connection failures
 - Sequence Diagrams: None
- void reportConnectionFailure(**GatewayId** gatewayId, **Timestamp** attempt)
 - Effect: Reports a connection-level or delivery failure detected within the CommunicationGateway to the RetryCoordinator. Called by FailureDetector when it receives a RoutingFailureSignal or AuthFailureSignal that warrants a retry attempt. The attempt timestamp is used by RetryCoordinator to compute the next retry time using exponential backoff.
 - Sequence Diagrams: None

Diagrams: None

E.3.19 Av2GatewayDataIngress

Provided by: ¶ Av2PatientGateway, ¶ Av2SensorDataTransmitter

Required by: ¶ Av2CommunicationGateway, ¶ Av2RequestReceiver

Description : Published by the PatientGateway at its external boundary. Provides the entry point through which the gateway transmits sensor data and emergency notifications to the CommunicationGateway. In degraded mode outbound transmissions are buffered locally rather than sent immediately.

Operations:

- **TransmissionAck** transmitEmergencyNotification(**GatewayId** gatewayId, **EmergencyMessage** alert, **Timestamp** sendAt)
 - Effect: Transmits an emergency notification from the PatientGateway to the backend CommunicationGateway over the primary telecom channel. Used when the primary channel is available. If the primary channel is unavailable the EmergencyNotificationService delegates to BackupChannelDispatch instead and this operation is not called. Returns: A TransmissionAck confirming the backend has accepted and queued the emergency notification for downstream processing.
 - Returns: A TransmissionAck confirming the backend has accepted and queued the emergency notification for downstream processing.
 - Sequence Diagrams: None
- **TransmissionAck** transmitSensorData(**GatewayId** gatewayId, **SensorDataPackage** data, **Timestamp** sendAt)
 - Effect: Transmits a sensor data package from the PatientGateway to the backend CommunicationGateway over the primary channel. The sendAt timestamp records the moment of transmission at the gateway. The backend responds with a TransmissionAck confirming successful receipt.
 - Returns: A TransmissionAck confirming the backend has accepted the transmission. Contains the message identifier assigned by the backend and the server-side receipt timestamp.
 - Sequence Diagrams: None

Diagrams: None

E.3.20 Av2GatewayResyncCommand

Provided by: ¶ Av2RecoverySyncService

Required by: ¶ Av2PatientGateway, ¶ Av2RetryAndSynchronizationService

Operations:

- void triggerResync(**GatewayId** gatewayId, **Timestamp** fromTimestamp)

- Effect: Instructs the specified PatientGateway to flush its LocalBufferingRepository and retransmit all buffered sensor data recorded after fromTimestamp. Called by RecoverySyncService after a communication failure is resolved
- Sequence Diagrams: None

Diagrams: None

E.3.21 Av2HeartbeatUpdate

Provided by: ¶ Av2RequestReceiver

Required by: ¶ Av2HeartbeatTracker

Description : Used by the RequestReceiver to notify the HeartbeatTracker that a transmission has been received from a gateway. Only used inside the CommunicationGateway.

Operations:

- void recordTransmission(**GatewayId** gatewayId, **Timestamp** recievedAt)
 - Effect: Notifies HeartbeatTracker that a transmission was received from the specified gateway at the given server-side timestamp. Called by RequestReceiver for every inbound transmission regardless of whether it passes authentication. Updates the last-seen time for that gateway so MissingDataDetector can accurately compute how long the gateway has been silent.
 - Sequence Diagrams: None

Diagrams: None

E.3.22 Av2InboundGatewayData

Provided by: ¶ Av2RequestReceiver

Required by: ¶ Av2AuthenticationValidator

Description : Carries parsed and format-validated inbound transmissions from the RequestReceiver to the AuthenticationValidator. Only used inside the CommunicationGateway.

Operations:

- void forwardParsedTransmission(**GatewayId** gatewayId, **ParsedTransmission** transmission, **Timestamp** recievedAt)
 - Effect: Forwards a format-validated and parsed inbound transmission from RequestReceiver to AuthenticationValidator for credential verification. The recievedAt timestamp is the server-side time at which the CommunicationGateway received the raw transmission. Only transmissions that pass initial format validation are forwarded, malformed transmissions are dropped by RequestReceiver before this interface is called.
 - Sequence Diagrams: None

Diagrams: None

E.3.23 Av2MessageBrokerHealthCheck

Provided by: ¶ Av2MessageBroker

Required by: ¶ Av2InternalFailureDetector, ¶ Av2MonitoringService

Description : Exposes a health check endpoint on the MessageBroker so that the InternalFailureDetector in MonitoringService can independently poll its operational status. The MessageBroker reports its own queue depth, throughput, and availability. A failure to respond within one second or a DOWN status triggers a **FailureEvent** of type COMMUNICATION_CHANNEL_DOWN in the InternalFailureDetector, since MessageBroker failure directly breaks the data path between the CommunicationGateway and the DataIngestionService.

Operations:

- **HealthStatus** ping()
 - Effect: Returns the current operational status of the MessageBroker. Returns OK if the broker is accepting and delivering messages normally, DEGRADED if queue depth is abnormally high or throughput is reduced, DOWN if the broker is not accepting new messages.
 - Returns: the current operational status of the MessageBroker
 - Sequence Diagrams: None

Diagrams: None

E.3.24 Av2NotificationDispatch

Provided by: ¶ Av2MonitoringService, ¶ Av2StakeholderNotifier

Required by: ¶ Av2AdminPortal

Description : The outbound notification channel through which the StakeholderNotifier sends alerts to stakeholders.

Operations:

- void notifyAdministrator(**ClassifiedFailure** failure, string message)
 - Effect: Dispatches a notification to the system administrator describing the classified failure. The message is delivered via the NotificationDispatcher within the deadline encoded in the **ClassifiedFailure** — 2 minutes for HIGH escalation level, 5 minutes for MEDIUM, 10 minutes for LOW — per Av2 resolution response measures. Must succeed in 90% of cases within the deadline.
 - Sequence Diagrams: None
- void notifyPatientContact(**PatientId** patientId, **ClassifiedFailure** failure, string message)
 - Effect: Dispatches a notification to the registered contact person of the patient identified by patientId. Used for GATEWAY_UNREACHABLE and MISSING_SENSOR_DATA failures where the patient or their contact must be informed. The patient's contact information is retrieved from PatientRecordRepository via PatientContactLookup before dispatch.
 - Sequence Diagrams: None

Diagrams: None

E.3.25 Av2PatientContactLookup

Provided by: ¶ PatientRecordRepository

Required by: ¶ Av2MonitoringService, ¶ Av2StakeholderNotifier

Operations:

- **ContactComponent** getContactInfo(**PatientId** id)
 - Effect: Retrieves the contact information of the registered contact person for the specified patient from the PatientRecordRepository. Used by StakeholderNotifier to obtain the phone number or email address needed to notify the patient's contact when a GATEWAY_UNREACHABLE or MISSING_SENSOR_DATA failure is detected.
 - Returns: A ContactInfo object containing the name, phone number, and email address of the patient's registered contact person.
 - Sequence Diagrams: None

Diagrams: None

E.3.26 Av2PatientStatusOverview

Provided by: ¶ PatientRecordRepository

Required by: ¶ Av2CommunicationGateway, ¶ Av2FailureClassifier, ¶ Av2GatewayRegistry, ¶ PatientStatusService

Operations:

- **PatientStatus** getPatientStatus(**PatientId** patientId)
 - Effect: Returns the current risk level of the specified patient as stored in the PatientRecordRepository. Used by FailureClassifier to determine escalation urgency, and by GatewayRegistry to calibrate heartbeat intervals
 - Returns: The current **PatientStatus** of the specified patient including their risk level classification.
 - Sequence Diagrams: None

Diagrams: None

E.3.27 Av2QueuedSensorDataMgmt

Provided by: ¶ Av2MessageBroker

Required by: ¶ Av2DataIngestionService, ¶ Av2EmergencyBackupReceiver, ¶ Av2RecoverySyncService

Operations:

- **SensorDataPackage** consume()

- Effect: Blocks until a sensor data package is available and returns it for processing. Called by DataIngestionService and RecoverySyncService to drain the queue.
- Returns: The next **SensorDataPackage** dequeued from the MessageBroker in delivery order.
- Sequence Diagrams: None
- void enqueue(**GatewayId** gatewayId, **SensorDataPackage** data, **Timestamp** receivedAt)
 - Effect: Adds a sensor data package to the MessageBroker’s durable queue for downstream processing by the DataIngestionService. Called by the RoutingManager after a validated transmission is received.
 - Sequence Diagrams: None
- int getQueueDepth()
 - Effect: Returns the current number of messages waiting in the queue. Used by RecoverySyncService to determine when the backlog from a degraded-mode recovery has been fully drained.
 - Returns: The number of messages currently waiting in the queue as an integer.
 - Sequence Diagrams: None

Diagrams: None

E.3.28 Av2RecoveryCoordination

Provided by: ¶ Av2MonitoringService, ¶ Av2RecoveryTracker

Required by: ¶ Av2RecoverySyncService, ¶ Av2RedeploymentCoordinator

Operations:

- void initiateRecovery(**FailureEventId** id)
 - Effect: Signals RecoverySyncService and RedeploymentCoordinator to begin post-failure recovery actions for the specified failure. RecoverySyncService drains any queued sensor data that accumulated in the MessageBroker during the outage and instructs the affected PatientGateway to flush its LocalBufferingRepository. Called by RecoveryTracker when a failure is resolved.
 - Sequence Diagrams: None
- void onRecoveryComplete(**FailureEventId** id, **Timestamp** resolvedAt)
 - Effect: Notifies the MonitoringService that all recovery actions for the specified failure have been completed. Used to close the failure record, update SLA metrics with the total outage duration, and clear the failure from the active failure list.
 - Sequence Diagrams: None

Diagrams: None

E.3.29 Av2RecoveryStatus

Provided by: ¶ Av2FailureClassifier

Required by: ¶ Av2RecoveryTracker

Description : Signals the RecoveryTracker when a previously detected failure has been resolved, allowing it to close the failure record and trigger recovery coordination.

Operations:

- void onFailureResolved(**FailureEventId** failureId, **Timestamp** resolvedAt)
 - Effect: Signals RecoveryTracker that the failure identified by failureId has been resolved at the given timestamp. RecoveryTracker uses resolvedAt together with the failure’s detectedAt timestamp to compute the total communication gap duration, closes the failure record, and triggers RecoveryCoordination to begin post-failure synchronisation.
 - Sequence Diagrams: None

Diagrams: None

E.3.30 Av2RedeploymentControl

Provided by: ¶ Av2RedeploymentCoordinator

Required by: ¶ Av2AdminPortal

Operations:

- **RedeploymentStatus** getRedeploymentStatus(**RedeploymentJobId** id)
 - Effect: Returns the current status of the redeployment or rollback job identified by the given job identifier. Called periodically by AdminPortal to track job progress. If the job identifier does

not exist a `NoSuchJobException` is thrown.

- Returns: The current **RedeploymentStatus** of the job: `IN_PROGRESS` if still running, `COMPLETED` if successful, `FAILED` if the deployment failed, or `ROLLED_BACK` if a rollback completed successfully.
- Sequence Diagrams: None
- **RedeploymentJob** `triggerRedeployment(string subsystemId)`
 - Effect: Initiates a redeployment of the specified subsystem to its latest available version. Called by `AdminPortal` when the system administrator decides to redeploy a failing component. The `RedeploymentCoordinator` begins the redeployment immediately and the operation returns as soon as the job is created. The administrator must poll `getRedeploymentStatus` to track progress. The entire redeployment must complete within 5 minutes per Av2 response measures.
 - Returns: A **RedeploymentJob** object containing the assigned job identifier, the target subsystem, the start timestamp, and the initial status of `IN_PROGRESS`.
 - Sequence Diagrams: None
- **RedeploymentJob** `triggerRollback(string subsystemId, string targetVersion)`
 - Effect: Initiates a rollback of the specified subsystem to the given previously working version. Called by `AdminPortal` when redeployment fails or when the administrator decides to revert to a stable state. Like `triggerRedeployment`, the operation returns immediately with a job handle and the administrator polls for progress. Must complete within 5 minutes per Av2 response measures.
 - Returns: A **RedeploymentJob** object containing the assigned job identifier, the target subsystem, the target version, the start timestamp, and the initial status of `IN_PROGRESS`.
 - Sequence Diagrams: None

Diagrams: None

E.3.31 Av2RetryDispatch

Provided by: ¶ Av2RetryCoordinator

Required by: ¶ Av2RoutingManager

Description : Published by the `RetryCoordinator`. Provides retry-ready messages to the `RoutingManager` for re-delivery. Only used inside the `CommunicationGateway`.

Operations:

- `void cancelRetry(GatewayId gatewayId)`
 - Effect: Cancels all pending retry attempts for the specified gateway. Called by `RetryCoordinator` when delivery is confirmed via `BrokerCommitNotification`, or when the failure has persisted long enough to require escalation to `MonitoringService` rather than further retrying.
 - Sequence Diagrams: None
- **RetryState** `getRetryState(GatewayId gatewayId)`
 - Effect: Returns the current retry state for the specified gateway including the number of attempts made, the next scheduled retry time, and the current backoff interval. Used by `RetryCoordinator` to determine whether to continue retrying or escalate.
 - Returns: A **RetryState** object containing the gateway identifier, current attempt number, timestamp of the next scheduled retry, and the current backoff interval duration.
 - Sequence Diagrams: None
- `void scheduleRetry(GatewayId gatewayId, int attempt)`
 - Effect: Schedules a retry delivery attempt for the specified gateway. The `attemptNumber` determines the backoff interval — each successive attempt doubles the wait time before the next retry. Called by `RetryCoordinator` after receiving a connection failure from `FailureDetector`. `RoutingManager` picks up the retry signal and reattempts delivery to the `MessageBroker`.
 - Sequence Diagrams: None

Diagrams: None

E.3.32 Av2RoutFailureSignals

Provided by: ¶ Av2RoutingManager

Required by: ¶ Av2FailureDetector

Operations:

Diagrams: None

E.3.33 Av2SensorDataAcquisition

Provided by: None

Required by: ¶ Av2SensorCommunicator

Description : Published by the external monitoring device. Provides the sensor data to the PatientGatewayApplication. Consumed by the SensorCommunicator

Operations:

- boolean isSensorAvailable()
 - Effect: Returns true if the monitoring device is connected and producing readings. Used by SensorCommunicator to detect sensor disconnection and alert PatientAlertService accordingly.
 - Returns: True if the monitoring device is connected and producing readings. False if the device is disconnected or not responding.
 - Sequence Diagrams: None
- **SensorDataPackage** readLatestReading()
 - Effect: Returns the most recent sensor data package produced by the monitoring device, containing one or more SensorDataElements covering all active sensor types (e.g. heart rate, blood pressure, ECG). Called periodically by SensorCommunicator at the configured transmission rate.
 - Returns: sensor data package
 - Sequence Diagrams: None

Diagrams: None

E.3.34 Av2ServerConnectionStatus

Provided by: ¶ Av2ServerConnectivityMonitor

Required by: ¶ Av2DegradeModeManager, ¶ Av2RetryAndSynchronizationService

Description : Published by the ServerConnectivityMonitor. Reports whether the PMS backend is currently reachable from the patient gateway and the timestamp of the last successful contact. Consumed by the DegradeModeManager to decide when to enter or exit degraded mode.

Operations:

- **Timestamp** getLastSuccessfulContactTime()
 - Effect: Returns the timestamp of the most recent successful heartbeat probe response from the PMS backend. Used by DegradeModeManager and RetryAndSynchronizationService to calculate how long the gateway has been operating without backend connectivity.
 - Returns: The **Timestamp** of the last successful contact with the PMS backend. If no successful contact has ever been made, returns null.
 - Sequence Diagrams: None
- boolean isServerReachable()
 - Effect: Returns whether the PMS backend is currently reachable from the patient gateway based on the most recent heartbeat probe sent by ServerConnectivityMonitor. Used by DegradeModeManager to decide whether to enter or exit degraded mode.
 - Returns: True if the most recent heartbeat probe received a response from the backend within the expected timeout. False if the backend did not respond, indicating the primary channel or backend is unavailable.
 - Sequence Diagrams: None

Diagrams: None

E.3.35 Av2SLAReporting

Provided by: ¶ Av2MonitoringService, ¶ Av2SLAMonitor

Required by: ¶ Av2AdminPortal

Operations:

- **SLAReport** getSLAReport(**Period** period)
 - Effect: Returns a compiled SLA compliance report for the given time period covering all monitored gateways. The report includes rolling availability percentage, mobile network coverage ratio, and average bandwidth measurements, compared against the SLA thresholds of 99.5% availability, 80% coverage, and 64Kb/s bandwidth. Used by AdminPortal to display compliance status to the system administrator.

- Returns: An SLAReport containing per-gateway and aggregate SLA metrics for the requested period, including any threshold violations detected during that period.
- Sequence Diagrams: None

Diagrams: None

E.3.36 Av2ValidatedTransmission

Provided by: ¶ Av2AuthenticationValidator

Required by: ¶ Av2RoutingManager

Description : Carries transmissions that have passed authentication validation from the AuthenticationValidator to the RoutingManager. Only used inside the CommunicationGateway.

Operations:

- void forwardValidatedTransmission(**GatewayId** gatewayId, **ParsedTransmission** transmission, **Timestamp** receivedAt)
 - Effect: Forwards a transmission that has passed authentication validation from AuthenticationValidator to RoutingManager for downstream dispatch. Only transmissions from registered gateways with valid credentials reach this interface. The RoutingManager uses the transmission type to decide whether to route it as sensor data or as an emergency notification.
 - Sequence Diagrams: None

Diagrams: None

E.3.37 ClinicalJobMgmt

Provided by: ¶ ClinicalJobCreator

Required by: ¶ RiskEstimationScheduler

Operations:

- List<Tuple<**ClinicalModelJob**, Double>> createClinicalModelJobsForPatient(**PatientId** patientId, Tuple<**SensorDataPackage**, **Timestamp**> sensorData)
 - Effect: Construct the list of **ClinicalModelJobs** for a specified patient. These jobs are populated with the necessary data for their execution by the **RiskEstimationProcessor**.
 - Returns: A list of tuples containing the **ClinicalModelJob** and its weight.
 - Sequence Diagrams: fig. D.1

Diagrams: None

E.3.38 ClinicalModelCacheMgmt

Provided by: ¶ ClinicalModelCache

Required by: ¶ P1ConfigurationHandler, ¶ P1PhysicianCommandService

Operations:

- void invalidateCacheEntries(**PatientId** patientId)
 - Effect: The **ClinicalModelCache** will invalidate (i.e., remove) all items in its cache for the patient identified by patientId. If the cache does not contain any items for this patient, nothing is changed. After invalidating the cached items for a certain patient, the next request for them will lead to fetching them from the database and storing them in the cache again.
 - Sequence Diagrams: None

Diagrams: None

E.3.39 ClinicalModelStorage

Provided by: ¶ ClinicalModelDB

Required by: ¶ ClinicalModelCache

Operations:

- Map<**ClinicalModel**, Tuple<**ClinicalModelConfiguration**, Double>> getAssignedClinicalModelsAndConfigForPatient(**PatientId** patientId) throws *NoSuchPatientException*
 - Effect: Fetch and return the ids of the **ClinicalModels** assigned to the patient identified by patientId and their configurations for this patient. The configurations are returned as a **ClinicalModelConfiguration** containing a map of configuration parameters and their value.

- Returns: A map with for every applicable **ClinicalModel**, a tuple containing the corresponding **ClinicalModelConfiguration** for the specified patient and the corresponding weight factors used for combining the results.
- Sequence Diagrams: fig. D.1

Diagrams: None

E.3.40 DataBuffer

Provided by: ¶ Av2SensorCommunicator

Required by: ¶ Av2SensorDataTransmitter

Description : Published by the SensorCommunicator. Provides a buffer of acquired sensor readings awaiting transmission. Consumed by the SensorDataTransmitter

Operations:

- void bufferReading(**SensorDataPackage** reading, **Timestamp** time)
 - Effect: Adds the given sensor data package and its acquisition timestamp to the in-memory buffer maintained by SensorCommunicator. Called each time a new reading is acquired from the wearable sensor hardware. If the buffer is full, the oldest entry is overwritten to ensure the most recent data is always preserved.
 - Sequence Diagrams: None
- void clearDeliveredReadings(**Timestamp** upTo)
 - Effect: Removes all buffered readings with an acquisition timestamp at or before the given timestamp from the buffer. Called by SensorDataTransmitter after the CommunicationGateway confirms receipt of a batch of readings via TransmissionAck.
 - Sequence Diagrams: None
- List<Tuple<**SensorDataPackage**, **Timestamp**>> getBufferedReading()
 - Effect: Returns all sensor data packages currently held in the buffer awaiting transmission. Called by SensorDataTransmitter to retrieve readings for delivery to the CommunicationGateway.
 - Returns: An ordered list of tuples each containing a **SensorDataPackage** and the timestamp at which it was acquired. Ordered from oldest to newest so that SensorDataTransmitter delivers data in chronological order.
 - Sequence Diagrams: None
- boolean isBufferFull()
 - Effect: Checks whether the buffer has reached its maximum capacity.
 - Returns: True if the buffer is at maximum capacity and cannot accept new readings without overwriting existing ones. False if space remains available.
 - Sequence Diagrams: None

Diagrams: None

E.3.41 FetchClinicalModels

Provided by: ¶ ClinicalModelCache

Required by: ¶ ClinicalJobCreator, ¶ RiskEstimationScheduler

Operations:

- Map<**ClinicalModel**, Tuple<**ClinicalModelConfiguration**, Double>> getAssignedClinicalModelsAndConfig-ForPatient(**PatientId** patientId)
 - Effect: The ClinicalModelCache will return the clinical models assigned to the patient identified by patientId and their configurations for this patient. The configurations are a map of configuration parameters and their value. If the cache contains data for the patient with given id, the data from the cache is returned. If not, the ClinicalModelCache will fetch all clinical models assigned to the patient with given patientId and their configurations for this patient, store these in the cache and return them.
 - Returns: A map with for every applicable **ClinicalModel**, a tuple containing the corresponding **ClinicalModelConfiguration** for the specified patient and the corresponding weight factors used for combining the results.
 - Sequence Diagrams: fig. D.1

Diagrams: None

E.3.42 FetchJobs

Provided by: [¶ RiskEstimationScheduler](#)

Required by: [¶ JobProcessor](#), [¶ RiskEstimationProcessor](#)

Operations:

- **ClinicalModelJob** getNextJob()
 - Effect: Returns the next clinical model computation job that must be performed (i.e., the first job in the queue).
 - Returns: The next clinical model computation job that must be performed.
 - Sequence Diagrams: [fig. D.2](#)

Diagrams: None

E.3.43 FetchMLModel

Provided by: [¶ MLModelRetrieval](#)

Required by: [¶ JobProcessor](#)

Operations:

- **MLModel** fetchModel(String modelId) throws *NoSuchMLModelException*
 - Effect: Fetch the **MLModel** corresponding with the provided modelId.
 - Returns: The requested **MLModel**.
 - Sequence Diagrams: [fig. D.2](#)

Diagrams: None

E.3.44 healthAPI

Provided by: [¶ HIS](#)

Required by: [¶ P1HISAdapter](#)

Description : This interface provides a reduced set of simplified operations that are sufficient in the context of the patient monitoring service that is being developed. It is derived from the HL7 FHIR standard for healthcare interoperability, based on the HAPI FHIR implementation of the FHIR specification in Java. The relevant OpenAPIs are described at: <https://hapi.fhir.org/baseR4/swagger-ui/>, while the FHIR standard is available at: <https://hl7.org/fhir/>. The descriptions from the data types used in this interface follow directly from the API documentation (<https://hapifhir.io/hapi-fhir/apidocs/hapi-fhir-structures-r5/org/hl7/fhir/r5/model/package-summary.html>).

Operations:

- void deleteObservation(string id) throws *AuthorizationException2*, *AuthenticationException2*
 - Effect: Delete an **Observation** instance with the specified id.
 - Sequence Diagrams: None
- void deletePatient(string id) throws *AuthorizationException2*, *AuthenticationException2*
 - Effect: Delete a **Patient** instance with the specified id.
 - Sequence Diagrams: None
- void deleteRiskAssessment(string id) throws *AuthorizationException2*, *AuthenticationException2*
 - Effect: Delete a **RiskAssessment** instance with the specified id.
 - Sequence Diagrams: None
- **Observation** getObservation(string id) throws *AuthorizationException2*, *AuthenticationException2*
 - Effect: Retrieve an observation instance
 - Returns: The **Observation** with the specified identifier.
 - Sequence Diagrams: None
- **Patient** getPatient(string id) throws *AuthorizationException2*, *AuthenticationException2*
 - Effect: Retrieve a patient record with the specified identifier.
 - Returns: The **Patient** with the specified identifier.
 - Sequence Diagrams: None
- **RiskAssessment** getRiskAssessment(string id) throws *AuthorizationException2*
 - Effect: Retrieve a **RiskAssessment** instance with the specified id.
 - Returns: The **RiskAssessment** with the specified identifier.
 - Sequence Diagrams: None
- **Observation** saveObservation(**Observation** observation) throws *AuthorizationException2*, *AuthenticationException2*

- Effect: Create a new **Observation** instance or in case an existing **Observation** instance exists with the same identifier, update this **Observation** instance.
- Returns: The **Observation** object.
- Sequence Diagrams: None
- **Patient** savePatient(**Patient** patient) throws *AuthorizationException2*
 - Effect: Create a new **Patient** instance or update an existing **Patient** instance.
 - Returns: The updated **Patient** object.
 - Sequence Diagrams: None
- **RiskAssessment** saveRiskAssessment(**RiskAssessment** riskAssessment) throws *AuthorizationException2*, *AuthenticationException2*
 - Effect: Create a new **RiskAssessment** instance or update an existing **RiskAssessment** instance.
 - Returns: The updated **RiskAssessment** object.
 - Sequence Diagrams: None
- List<**Observation**> searchObservation(**Query**<**Date**> date, **Query**<**String**> dataAbsentReason, **Query**<**Patient**> subject, **Query**<**String**> valueConcept, **Query**<**Date**> valueDate, **Query**<**Observation**> derivedFrom, **Query**<**Patient**> patient, **Query**<**Quantity**> valueQuantity, **Query**<**String**> identifier, **Query**<**String**> performer, **Query**<**String**> method, **Query**<**String**> category, **Query**<**String**> device, **Query**<**ObservationStatus**> status) throws *AuthorizationException2*
 - Effect: Search for **Observation** instances matching the specified query criteria.
 - Returns: The list of **Observations** matching the specified search criteria.
 - Sequence Diagrams: None
- List<**Patient**> searchPatient(**Query**<**Date**> birthDate, **Query**<**Boolean**> deceased, **Query**<**String**> addressState, **Query**<**String**> administrativeGender, **Query**<**String**> link, **Query**<**String**> language, **Query**<**String**> addressCountry, **Query**<**Date**> deathDate, **Query**<**String**> phonetic, **Query**<**String**> telecom, **Query**<**String**> addressCity, **Query**<**String**> email, **Query**<**String**> given, **Query**<**String**> identifier, **Query**<**String**> address, **Query**<**String**> generalPractitioner, **Query**<**Boolean**> active, **Query**<**String**> addressPostalCode, **Query**<**String**> phone, **Query**<**String**> organizationCustodian, **Query**<**String**> name, **Query**<**String**> family) throws *AuthorizationException2*, *AuthenticationException2*
 - Effect: Search for **Patient** instances matching the specified query criteria.
 - Returns: The list of **Patients** matching the specified search criteria.
 - Sequence Diagrams: None
- List<**RiskAssessment**> searchRiskAssessment(**Query**<**Date**> date, **Query**<**String**> identifier, **Query**<**String**> performer, **Query**<**String**> method, **Query**<**Range**> probability, **Query**<**Patient**> subject, **Query**<**String**> condition, **Query**<**Patient**> patient, **Query**<**BigDecimal**> risk) throws *AuthenticationException2*, *AuthorizationException2*
 - Effect: Search for a **RiskAssessment** instance matching the provided query criteria.
 - Returns: The list of **RiskAssessments** matching the specified search criteria.
 - Sequence Diagrams: None

Diagrams: None

E.3.45 JobMgmt

Provided by:  RiskEstimationCombiner

Required by:  RiskEstimationScheduler

Operations:

- void addJobSet(**RiskEstimationId** id, List<Tuple<**ClinicalModelJobId**, Double>> jobIds)
 - Effect: Adds a set of identifiers for jobs belonging to a single risk estimation identified by id. The partial results of each clinical model computation job identified by an element in jobIds have to be combined in order to find the final result of the risk estimation as a whole.
 - Sequence Diagrams: fig. D.1

Diagrams: None

E.3.46 KVStore

Provided by:  KVStorageService

Required by:  RiskEstimationProcessor,  SensorDataRiskAssessment,  eHealth Platform

Operations:

- **Object** retrieve(**String** key) throws *IOException*

- Effect: Retrieve the object with the provided key from the Key-Value storage.
- Returns: The object retrieved from storage.
- Sequence Diagrams: fig. D.2
- void store(String key, **Object** value) throws *IOException*
 - Effect: Store the provided object in the Key-Value storage under the provided key.
 - Sequence Diagrams: fig. D.2

Diagrams: None

E.3.47 LaunchRiskEstimation

Provided by: RiskEstimationScheduler

Required by: P1OnDemandConsultationHandler, P1PatientDataService, P1PhysicianCommandService, P1SensorIngestModule

Operations:

- void launchRiskEstimation(**PatientId** patientId, **SensorDataPackage** newSensorData, **Timestamp** receivedAt, **Priority** priority, **CorrelationId** correlationId)
 - Effect: The **RiskEstimationScheduler** will fetch the clinical models and their configurations associated to the patient identified by patientId from storage using the **ClinicalModelCache**, notify the **RiskEstimationCombiner** of the different jobs that will be performed for a single risk estimation and schedule the individual jobs in its queue. In normal modus, queued jobs are returned in FIFO order. In overload modus, the system switches to dynamic priority: earliest deadline first and enqueues jobs of patient with a high risk level with an earlier deadline (2 instead of 5 minutes) to prioritize them over patients with lower risk levels. The **SensorDataPackage** newSensorData is passed because its arrival triggered the risk estimation. A risk level is estimated based on the computation of these clinical models. For the computation of the clinical models, the given sensor data newSensorData (which is the new sensor data that was received) is used and other required data is fetched from the respective databases if needed. The given time-stamp receivedAt is used in order to avoid fetching the new sensor data from the database. This time-stamp represents the time at which the new sensor data was received.
 - Sequence Diagrams: figs. D.1 and D.5

Diagrams: None

E.3.48 MLaaSAPI

Provided by: MLaaSLibrary

Required by: MLModelUpdateProcessor

Description : This API provides a number of operations for managing the models deployed on the MLaaS cloud.

Operations:

- **Credential** authenticate() throws *InvalidAPIKeyException*
 - Effect: Construct a **Credential** object for authenticating requests with the MLaaS service provider. This operation uses the previously configured API key to construct the **Credential** object. If such an API key is not configured, an exception will be thrown.
 - Returns: Authentication token for subsequent requests.
 - Sequence Diagrams: figs. D.3 and D.4
- void delete(**MLModel** model, **Credential** token) throws *AuthorizationException*
 - Effect: Delete a model from the online MLaaS service.
 - Sequence Diagrams: None
- **MLModelDescription** getModelDescription(List<**MLModelDescription**> descriptions, String modelId)
 - Effect: This method retrieves the correct **MLModelDescription** that corresponds with the **MLModel** identified by the provided modelId.
 - Returns: The **MLModelDescription** corresponding with the **MLModel** identified by the modelId.
 - Sequence Diagrams: None
- String getModelId(**MLModelDescription** description)
 - Effect: Extract the modelId from the **MLModelDescription**.
 - Returns: String with the modelId of the **MLModel** specified in the **MLModelDescription**.
 - Sequence Diagrams: fig. D.3
- void register(**MLModel** model, **Credential** token) throws *AuthorizationException*

- Effect: Register a new model with the MLaaS service.
- Sequence Diagrams: None
- void setAPIKey(String apiKey) throws *AuthenticationException*
 - Effect: Set the credentials for authenticating with the ML-as-a-Service provider.
 - Sequence Diagrams: None

Diagrams: None

E.3.49 MLaaSService

Provided by: ¶ MLaaS

Required by: ¶ MLModelManager, ¶ MLModelUpdateProcessor, ¶ eHealth Platform

Description : This interface offers operations for interacting with MLaaS service providers.

Operations:

- List<**MLModelUpdate**> fetchModelUpdates(**Credential** credential, **MLModelDescription** model)
 - Effect: Fetch a list of model updates to apply to the local **MLModel**.
 - Returns: The list of MLModelUpdates to apply locally.
 - Sequence Diagrams: fig. D.4
- List<**MLModelDescription**> getAvailableModels(**Credential** credential)
 - Effect: Get the list of available ML models to use for making predictions on incoming sensor data.
 - Returns: The list of available MLModels.
 - Sequence Diagrams: fig. D.3
- void pushModelUpdates(**Credential** credential, **MLModelDescription** model, List<**MLModelUpdate**> updates)
 - Effect: Send a list of MLModelUpdates to the online service.
 - Sequence Diagrams: None
- **MLModel** retrieveModel(**Credential** credential, **MLModelDescription** model)
 - Effect: Retrieve a specific **MLModel** corresponding with the provided **MLModelDescription**.
 - Returns: The requested **MLModel**.
 - Sequence Diagrams: fig. D.3

Diagrams: None

E.3.50 MLAPI

Provided by: ¶ MLLibrary

Required by: ¶ ModelUpdater, ¶ SensorDataRiskAssessment

Description : This is the API provided by the MLLibrary to use **MLModels** locally for making predictions and processing updates to these models.

Operations:

- **MLModel** applyUpdate(**MLModel** model, **MLModelUpdate** update)
 - Effect: Applies the provided **MLModelUpdate** on the provided **MLModel** and returns the resulting **MLModel**.
 - Returns: The resulting **MLModel** after applying the requested **MLModelUpdate**.
 - Sequence Diagrams: fig. D.4
- **Prediction** predict(**SensorDataPackage** sensorData)
 - Effect: Make a prediction with the active **MLModel** on the provided sensordata.
 - Returns: The prediction based on the provided sensor data.
 - Sequence Diagrams: fig. D.2
- void verify(**MLModel** model) throws *InvalidMLModelException*
 - Effect: Verifies that a provided **MLModel** is working correctly after applying **MLModelUpdates**.
 - Sequence Diagrams: fig. D.4

Diagrams: None

E.3.51 MLModelMgmt

Provided by: ¶ MLModelManager, ¶ MLModelStorageManager

Required by: ¶ MLModelRetrieval, ¶ RiskEstimationProcessor

Description : The interface provides operations for managing different **MLModels** locally, querying available models for making predictions on newly incoming sensor data.

Operations:

- **MLModel** fetchModel(String modelId) throws *NoSuchMLModelException*
 - Effect: Fetch the **MLModel** corresponding with the provided modelId.
 - Returns: The requested **MLModel**.
 - Sequence Diagrams: fig. D.2
- List<String> getAvailableModels()
 - Effect: Get the list of available **MLModels** that can be used for making **Predictions** locally.
 - Returns: The list of identifiers of the available **MLModels** that can be used.
 - Sequence Diagrams: None
- void synchronizeModels()
 - Effect: Synchronize the list of locally-stored **MLModels** by checking for newly-available **MLModels** that are not stored in the local **MLModelRepository** and retrieving these **MLModels** from the **MLaaS** provider.
 - Sequence Diagrams: fig. D.3
- void updateModel(String modelId) throws *NoSuchMLModelException*
 - Effect: Update the model with the specified id by fetching **MLModelUpdates** and applying them to the model.
 - Sequence Diagrams: fig. D.4

Diagrams: None

E.3.52 MLModelStorage

Provided by:  **MLModelRepository**

Required by:  **MLModelStorageManager**,  **MLModelUpdateProcessor**

Operations:

- boolean isAvailable(String modelId)
 - Effect: Check whether an **MLModel** (identified by the modelId) is available in the local **MLModelRepository**.
 - Returns: Boolean indicating if the specified model is present in the repository.
 - Sequence Diagrams: figs. D.3 and D.4
- **MLModel** retrieveModel(String modelId)
 - Effect: Retrieve the **MLModel** with the specified id.
 - Returns: The requested **MLModel**.
 - Sequence Diagrams: fig. D.4
- void storeModel(**MLModel** model)
 - Effect: Store an **MLModel** in the local **MLModelRepository**
 - Sequence Diagrams: figs. D.3 and D.4

Diagrams: None

E.3.53 MLModelSync

Provided by:  **MLModelUpdateProcessor**

Required by:  **MLModelStorageManager**

Operations:

- void fetchNewMLModels()
 - Effect: Fetches newly available **MLModels** and store them in the **MLModelRepository**.
 - Sequence Diagrams: fig. D.3
- void processMLModelUpdates(String modelId) throws *NoSuchMLModelException*
 - Effect: Retrieve and process the updates for the specified **MLModel** and store the updated **MLModel** in the **MLModelRepository**.
 - Sequence Diagrams: fig. D.4

Diagrams: None

E.3.54 MLModelUpdateMgmt

Provided by:  **ModelUpdater**

Required by: [MLModelUpdateProcessor](#)

Operations:

- **MLModel** processMLModelUpdates(**MLModel** model, List<**MLModelUpdate**> updates)
 - Effect: Requests the **ModelUpdater** to apply the list of **MLModelUpdates** on the provided **MLModel**.
 - Returns: The **MLModel** resulting from the application of the list of **MLModelUpdates** on the initial **MLModel**.
 - Sequence Diagrams: [fig. D.4](#)

Diagrams: None

E.3.55 OtherDataMgmt

Provided by: [P1PatientDataService](#), [P1PatientStatusModule](#)

Required by: [ClinicalJobCreator](#), [ClinicalModelCache](#), [P1PatientStatusCache](#), [RiskEstimation-Combiner](#), [RiskEstimationScheduler](#)

Operations:

- **PatientStatus** getPatientStatus(**PatientId** patientId) throws *NoSuchPatientException*
 - Effect: Fetch and return the status of the patient identified by the patientId.
 - Returns: The status of the patient.
 - Sequence Diagrams: None
- void setEstimatedPatientStatus(**PatientId** patientId, **PatientStatus** estimatedStatus, **Timestamp** estimationTime) throws *NoSuchPatientException*
 - Effect: Update the patient status estimation of the patient identified by patientId to the given value estimatedStatus and update the time of estimation to estimationTime. The time of estimation is the time at which the corresponding estimation job for this patient was completed. If patient's estimated risk changed the appropriate parties are notified.
 - Sequence Diagrams: [figs. D.5 and D.6](#)

Diagrams: None

E.3.56 P1ClinicalConfigMgmt

Provided by: [ClinicalModelDB](#)

Required by: [P1ConfigurationHandler](#), [P1PhysicianCommandService](#)

Operations:

- void setClinicalModelConfigForPatient(**PatientId** patientId, **ClinicalModelConfiguration** clinicalModelConfiguration)
 - Effect: Updates the configuration of the specified clinical model for the given patient in the **ClinicalModelDB**. Triggers invalidation of the corresponding entry in **ClinicalModelCache** so that the next risk estimation for this patient uses the updated configuration. Called by **ConfigWriter** when a cardiologist reconfigures a patient's risk assessment via UC7.
 - Sequence Diagrams: None

Diagrams: None

E.3.57 P1IConfigCommand

Provided by: [P1ConfigurationHandler](#)

Required by: [P1CommandRouter](#)

Operations:

- void configurePatientRiskAssessment(**PatientId** patientId, **ClinicalModelConfiguration** clinicalModelConfiguration)
 - Effect: Internal delegate — **CommandRouter** forwards the UC7 request to **ConfigurationHandler**, which writes the config and triggers the cache invalidation.
 - Sequence Diagrams: None

Diagrams: None

E.3.58 P1ICorrelationTracking

Provided by: [P1CorrelationTracker](#)

Required by: ¶ P1OnDemandConsultationHandler

Operations:

- boolean consumeCorrelation(**CorrelationId** correlationId)
 - Effect: Resolves a **CorrelationId** by removing it from the pending set if present; reports whether the event matches a tracked UC9 request.
 - Returns: True if the **CorrelationId** was pending (and has now been consumed); false otherwise.
 - Sequence Diagrams: None
- **CorrelationId** newCorrelationId()
 - Effect: Mints a fresh **CorrelationId** and records it in the pending set so a future IRiskEvents can be matched back to this UC9 request.
 - Returns: The freshly minted **CorrelationId**.
 - Sequence Diagrams: None

Diagrams: None

E.3.59 P1IHISAccess

Provided by: ¶ P1HISAdapter

Required by: ¶ P1EHRProxyModule, ¶ P1PatientDataService

Operations:

- **PatientRecord** getPatientRecord(**PatientId** patientId)
 - Effect: Retrieves the full EHR record of the specified patient from the HIS via the FHIRTranslator and HealthAPIClient. If the HIS is temporarily unavailable the HISAdapter returns a cached copy if one exists. If no cached copy is available a NoSuchPatientRecordException is thrown.
 - Returns: The aggregated **PatientRecord** (demographics + observations + risk assessments per FHIR) retrieved from the HIS.
 - Sequence Diagrams: None
- void updatePatientRecord(**PatientId** patientId, **PatientRecord** patientRecord)
 - Effect: Persists an updated patient record to the HIS via the FHIRTranslator and HealthAPIClient. Called by HISDispatcher when a physician updates a patient's risk level or when a completed risk assessment result must be written back to the HIS. The update must be forwarded to the HIS within 1 minute of the originating command per P1 response measures.
 - Sequence Diagrams: None

Diagrams: None

E.3.60 P1INotificationInbox

Provided by: ¶ P1NotificationDispatcher, ¶ P1NotificationInboxModule

Required by: ¶ P1PhysicianGateway

Operations:

- void acknowledgeNotification(**NotificationId** notificationId)
 - Effect: Confirms delivery of the notification so the dispatcher can remove it from the priority buffer and the durable log.
 - Sequence Diagrams: None
- List<**Notification**> getPendingNotifications(**PhysicianId** physicianId)
 - Effect: Returns the physician's queued notifications in priority order (red > yellow > green) without removing them from the buffer.
 - Returns: The list of pending Notifications for the physician, ordered by severity then arrival.
 - Sequence Diagrams: None
- void subscribeToNotifications(**PhysicianId** physicianId)
 - Effect: Adds the physician to the recipient registry so future patient-status events fan out to them.
 - Sequence Diagrams: None
- void unsubscribeFromNotifications(**PhysicianId** physicianId)
 - Effect: Removes the physician from the recipient registry; notifications already queued are still delivered.
 - Sequence Diagrams: None

Diagrams: None

E.3.61 P1INotificationLog

Provided by: ¶ P1DurableNotificationLog

Required by: ¶ P1NotificationInboxModule, ¶ P1RiskEventSubscriber

Operations:

- void markDelivered(**NotificationId** notificationId)
 - Effect: Removes the notification from the durable log once the physician has acked delivery, bounding storage growth.
 - Sequence Diagrams: None
- void persist(**Notification** notification)
 - Effect: Writes the notification to the durable log before it enters the in-memory buffer so a crash cannot lose a red or yellow notification.
 - Sequence Diagrams: None
- List<**Notification**> recoverPending()
 - Effect: On startup, returns notifications that were persisted but never acked so NotificationInboxModule can replay them into the priority buffer.
 - Returns: The list of unacked Notifications recovered from the durable log.
 - Sequence Diagrams: None

Diagrams: None

E.3.62 P1IONdemandCommand

Provided by: ¶ P1OnDemandConsultationHandler

Required by: ¶ P1CommandRouter

Operations:

- **ConsultationId** requestOnDemandConsultation(**PatientId** patientId)
 - Effect: Internal delegate, CommandRouter forwards the UC9 request to OnDemandConsultationHandler, which mints a **CorrelationId**, fetches sensor data, launches a priority risk job, and awaits the matching event.
 - Returns: The **ConsultationId** minted for this UC9 request.
 - Sequence Diagrams: None

Diagrams: None

E.3.63 P1IONdemandSensorFetch

Provided by: ¶ P1PatientGatewayCommander

Required by: ¶ P1OnDemandConsultationHandler, ¶ P1PhysicianCommandService

Operations:

- **SensorDataPackage** requestCurrentSensorData(**PatientId** patientId, **CorrelationId** correlationId)
 - Effect: Synchronously asks the patient gateway to push the current sensor reading; bounded by the commander timeout sized inside the 3-min UC9 initiation budget.
 - Returns: The fresh **SensorDataPackage** supplied by the patient gateway for this UC9 request.
 - Sequence Diagrams: None

Diagrams: None

E.3.64 P1IPatientQuery

Provided by: ¶ P1PatientQueryService

Required by: ¶ P1PhysicianGateway

Operations:

- **PatientStatus** consultPatientStatus(**PatientId** patientId)
 - Effect: Returns the current risk level and latest estimated status of the specified patient. Served from HotCache if a fresh entry exists within the staleness threshold defined by StalenessPolicy. If the cache entry is stale or absent, CacheLoader fetches fresh data from PatientRecordRepository before responding. Response time is bounded per P1: within 2 seconds for high-risk patients, 5 seconds for medium-risk, 10 seconds for low-risk.
 - Returns: The patient's most recent status.
 - Sequence Diagrams: None

Diagrams: None

E.3.65 P1PatientStatusRead

Provided by: ¶ P1PatientStatusCache

Required by: ¶ P1PatientQueryService

Operations:

- **PatientStatus** getPatientStatus(**PatientId** patientId)
 - Effect: Cache-first read of the patient’s status; on miss the cache fetches via OtherDataMgmt and populates itself for subsequent reads.
 - Returns: The patient’s most recent status.
 - Sequence Diagrams: None

Diagrams: None

E.3.66 P1PhysicianAPI

Provided by: ¶ P1PhysicianGateway

Required by: None

Operations:

- void configurePatientRiskAssessment(**PatientId** patientId, **ClinicalModelConfiguration** clinicalModelConfiguration)
 - Effect: Overwrites the per-patient **ClinicalModelConfiguration** and invalidates the clinical-model cache.
 - Sequence Diagrams: None
- **PatientStatus** consultPatientStatus(**PatientId** patientId)
 - Effect: Returns the named patient’s current status; SLA tier (2/5/10 s) is picked from the patient’s risk level.
 - Returns: The patient’s most recent status, risk level, supporting data, and timestamp.
 - Sequence Diagrams: None
- **ConsultationId** requestOnDemandConsultation(**PatientId** patientId)
 - Effect: Initiates an on-demand consultation, fetches fresh sensor data, launches a priority risk estimation, and returns an id the caller uses to correlate the eventual result.
 - Returns: The **ConsultationId** minted for this request, used to match the result notification back to the caller.
 - Sequence Diagrams: None
- void subscribeToNotifications(**PhysicianId** physicianId)
 - Effect: Registers a physician to receive notifications about subsequent patient-status changes.
 - Sequence Diagrams: None
- void updatePatientRiskLevel(**PatientId** patientId, **PatientStatus** patientStatus)
 - Effect: Updates the patient’s risk level and forwards the change to the EHR via the HIS proxy.
 - Sequence Diagrams: None

Diagrams: None

E.3.67 P1PhysicianCommand

Provided by: ¶ P1CommandRouter, ¶ P1PhysicianCommandService

Required by: ¶ P1PhysicianGateway

Operations:

- void configurePatientRiskAssessment(**PatientId** patientId, **ClinicalModelConfiguration** clinicalModelConfiguration)
 - Effect: Overwrites the per-patient **ClinicalModelConfiguration** and invalidates the clinical-model cache.
 - Sequence Diagrams: None
- **ConsultationId** requestOnDemandConsultation(**PatientId** patientId)
 - Effect: Initiates an on-demand consultation, fetches fresh sensor data, launches a priority risk estimation, and returns an id correlating the eventual result.
 - Returns: The **ConsultationId** minted for this request, used to match the result back to the caller.
 - Sequence Diagrams: None
- void updatePatientRiskLevel(**PatientId** patientId, **PatientStatus** patientStatus)
 - Effect: Updates the patient’s risk level and forwards the change to the EHR via the HIS proxy.
 - Sequence Diagrams: None

Diagrams: None

E.3.68 P1PriorityBuffer

Provided by: ¶ P1PriorityBuffer

Required by: ¶ P1NotificationInboxModule, ¶ P1RiskEventSubscriber

Operations:

- void enqueue(**PhysicianId** physicianId, **Notification** notification)
 - Effect: Adds the notification to the physician’s pending set; severity drives priority ordering and the drop-green-first policy when the 100/min threshold is exceeded.
 - Sequence Diagrams: None
- List<**Notification**> peekPending(**PhysicianId** physicianId)
 - Effect: Returns the physician’s pending notifications in priority order (red > yellow > green) without removing them from the buffer.
 - Returns: The list of pending Notifications for the physician, ordered by severity then arrival.
 - Sequence Diagrams: None
- void removePending(**NotificationId** notificationId)
 - Effect: Removes the notification from the buffer after the physician acks it through INotificationInbox.
 - Sequence Diagrams: None

Diagrams: None

E.3.69 P1RecipientRegistry

Provided by: ¶ P1RecipientRegistry

Required by: ¶ P1NotificationInboxModule, ¶ P1RiskEventSubscriber

Operations:

- List<**PhysicianId**> findRecipients(**PatientId** patientId)
 - Effect: Looks up the physicians who should be notified about events for the patient; backs UC5 recipient resolution.
 - Returns: The list of PhysicianIds subscribed to events for this patient.
 - Sequence Diagrams: None
- void subscribe(**PhysicianId** physicianId)
 - Effect: Adds the physician to the notification recipient set.
 - Sequence Diagrams: None
- void unsubscribe(**PhysicianId** physicianId)
 - Effect: Removes the physician from the notification recipient set.
 - Sequence Diagrams: None

Diagrams: None

E.3.70 P1RiskEvents

Provided by: ¶ P1PatientDataService, ¶ P1PatientStatusModule

Required by: ¶ P1NotificationDispatcher, ¶ P1OnDemandConsultationHandler, ¶ P1PhysicianCommandService, ¶ P1RiskEventSubscriber

Operations:

- **SubscriptionId** subscribe(**SubscriberId** subscriberId, **FilterCriteria** filterCriteria)
 - Effect: Registers a subscriber to receive **RiskEvent** payloads matching the filter; an empty filter receives every event.
 - Returns: A **SubscriptionId** the caller uses to unsubscribe later.
 - Sequence Diagrams: None
- void unsubscribe(**SubscriptionId** subscriptionId)
 - Effect: Removes the named subscription so no further RiskEvents are pushed to it.
 - Sequence Diagrams: None

Diagrams: None

E.3.71 P1RiskLevelCommand

Provided by: ¶ P1RiskLevelHandler

Required by: ¶ P1CommandRouter

Operations:

- void updatePatientRiskLevel(**PatientId** patientId, **PatientStatus** patientStatus)
 - Effect: Internal delegate, CommandRouter forwards the UC8 request to RiskLevelHandler, which writes the new risk level via PatientRecordMgmt.
 - Sequence Diagrams: None

Diagrams: None

E.3.72 PatientRecordMgmt

Provided by: ¶ P1EHRProxyModule, ¶ P1PatientDataService

Required by: ¶ P1PhysicianCommandService, ¶ P1RiskLevelHandler, ¶ RiskEstimationCombiner

Operations:

- **PatientRecord** getPatientRecord(**PatientId** patientId) throws *NoSuchPatientRecordException*
 - Effect: This will fetch the EHR record of the patient with given id from the HIS and return it. If the HIS is not available, an older (cached) copy of the patient record is returned if possible.
 - Returns: The requested patient record.
 - Sequence Diagrams: None
- void updatePatientRecord(**PatientId** patientId, **PatientRecord** patientRecord)
 - Effect: Writes the patient record back to the HIS by dispatching the appropriate healthAPI save operations.
 - Sequence Diagrams: None

Diagrams: None

E.3.73 Results

Provided by: ¶ RiskEstimationCombiner

Required by: ¶ JobProcessor, ¶ RiskEstimationProcessor

Operations:

- void addClinicalModelJobResults(**ClinicalModelJobId** jobId, **ClinicalModelJobResult** results)
 - Effect: Sends the result of the performed clinical model computation (identified by the jobId) to the RiskEstimationCombiner for combining with the other partial results belonging to the same risk estimation.
 - Sequence Diagrams: figs. D.2 and D.6

Diagrams: None

E.3.74 SensorDataAnalysis

Provided by: ¶ SensorDataRiskAssessment

Required by: ¶ JobProcessor

Operations:

- void performModelPrediction(**ClinicalModelJob** clinicalModelJob, **MLModel** model)
 - Effect: Perform a prediction using the provided **MLModel** and the data provided in the clinicalModelJob definition. Afterwards the results are submitted for combination by the RiskEstimationCombiner.
 - Sequence Diagrams: fig. D.2

Diagrams: None

E.3.75 SensorDataMgmt

Provided by: ¶ Av2SensorDataRepository, ¶ P1PatientDataService, ¶ P1SensorIngestModule

Required by: ¶ Av2DataIngestionService, ¶ ClinicalJobCreator, ¶ P1OnDemandConsultationHandler, ¶ P1PhysicianCommandService, ¶ RiskEstimationScheduler

Operations:

- void addSensorData(**PatientId** patientId, **SensorDataPackage** sensorData, **Timestamp** receivedAt)
 - Effect: Store the given sensor data and meta-data.
 - Sequence Diagrams: None
- Map<**Timestamp**, **SensorDataPackage**> getAllSensorDataOfPatient(**PatientId** patientId)

- Effect: This will fetch and return all sensor data belonging to the patient identified by `patientId`
- Returns: The sensor data and the timestamp of their arrival in the system.
- Sequence Diagrams: None
- **Map<Timestamp, SensorDataPackage> getAllSensorDataOfPatientBefore(PatientId patientId, Timestamp stopTime)**
 - Effect: Fetch and return all sensor data belonging to the patient identified by `patientId` which was received before the specified time `stopTime`.
 - Returns: The sensor data and the timestamp of their arrival in the system.
 - Sequence Diagrams: fig. D.1

Diagrams: None

E.3.76 UserAuthentication

Provided by: `⌘ PatientRecordRepository`

Required by: `⌘ UserAuthenticationService`

Operations:

- **boolean authenticate(string userId, string credentials)**
 - Effect: Verifies the provided credentials against the stored record in `PatientRecordRepository`. Returns true if authentication succeeds. Throws `AuthenticationException` if credentials are missing or malformed.
 - Returns: True if the provided credentials match the stored credentials for the specified user in the `PatientRecordRepository`. False is never returned — if credentials do not match, `AuthenticationException` is thrown instead, so a true return always means authentication succeeded.
 - Sequence Diagrams: None
- **void revokeSession(string userId, string sessionToken)**
 - Effect: Invalidates the specified session token for the given user immediately. Any subsequent request carrying the revoked token is rejected with an authentication error. Called on explicit logout or when an administrative session revocation is triggered. The token is added to a revocation list consulted by `UserAuthenticationService` on every incoming request.
 - Sequence Diagrams: None

Diagrams: None

E.4 Nodes

E.4.1 AdminNode

Responsibility: Server node within the PMS backend that hosts administrative coordination components, in particular the `RedeploymentCoordinator`. Provides the control plane the `AdminPortal` connects to in order to trigger redeployment or rollback of failed subsystems within the 5-minute Av2 deadline.

Visible on diagrams: fig. C.2

E.4.2 AdminNode (Pilot Deployment)

Responsibility: Pilot-deployment instance of the administrative server node hosting the `RedeploymentCoordinator`. Single instance shared across all pilot patients and clinical models.

Visible on diagrams: fig. C.3

E.4.3 AdminWorkstation

Responsibility: Client device (workstation, laptop) used by the system administrator. Hosts the `AdminPortal` web client through which administrators receive notifications and trigger redeployments or rollbacks. Outside the PMS server-side deployment boundary.

Visible on diagrams: fig. C.1

E.4.4 BackendGatewayNode

Responsibility: Server node hosting all backend-side gateway components: CommunicationGateway, MessageBroker, DataIngestionService, AcknowledgementService, and EmergencyBackupReceiver. Acts as the single backend ingress point for all primary-channel and SMS-backup patient-gateway traffic.

Visible on diagrams: fig. C.2

E.4.5 BackendGatewayNode (Pilot Deployment)

Responsibility: Pilot-deployment instance of the backend gateway tier hosting CommunicationGateway, MessageBroker, DataIngestionService, AcknowledgementService, and EmergencyBackupReceiver. Single ingress point for all 200 pilot patient gateways over both the primary and SMS-backup channels.

Visible on diagrams: fig. C.3

E.4.6 DevBackendNode (Dev Deployment)

Responsibility: Single-machine development node that collapses all backend-side responsibilities (gateway ingress, broker, ingestion, acknowledgement, SMS receiver, monitoring, recovery sync, redeployment coordination) onto one host to simplify local testing of the 20-patient dev deployment.

Visible on diagrams: fig. C.4

E.4.7 DevGatewayNode (Dev Deployment)

Responsibility: Development-deployment instance of the patient-side device hosting the PatientGateway. Replicated 20 times in the dev deployment to simulate the dev-scale patient population.

Visible on diagrams: fig. C.4

E.4.8 DevPatientRecordsNode (Dev Deployment)

Responsibility: Development-deployment node hosting the patient-data persistence components (PatientRecordRepository, SensorDataRepository, and related management/status/authentication services) used by the local dev environment.

Visible on diagrams: fig. C.4

E.4.9 eHealthPlatform

Responsibility: Container node representing the deployment context boundary for the eHealth Platform. Hosts NotificationDispatcher, PhysicianCommandService, PatientDataService, PatientQueryService, PatientStatusCache.

Visible on diagrams: fig. C.1

E.4.10 GatewayNode

Responsibility: **Patient**-side device node hosting the PatientGateway component. One node instance is deployed per enrolled patient; the multiplicity is captured on the deployment-association line to BackendGatewayNode, not on the node itself.

Visible on diagrams: fig. C.2

E.4.11 GatewayNode (Pilot Deployment)

Responsibility: Pilot-deployment instance of the patient-side device hosting the PatientGateway. Replicated 200 times in the pilot deployment, once per enrolled patient.

Visible on diagrams: fig. C.3

E.4.12 HISServer

Responsibility: External infrastructure hosting the Hospital Information System. Exposes the healthAPI (FHIR-based) consumed by PMS backend components that need to read or write patient records, observations, and risk assessments. Outside the PMS deployment boundary.

Visible on diagrams: fig. C.1

E.4.13 MLaaS Conn (Pilot Deployment)

Responsibility: This node is responsible for the connection with the MLaaS Service.

Visible on diagrams: fig. C.3

E.4.14 MLaaS Service (Dev Deployment)

Responsibility: This represents the MLaaS cloud service.

Visible on diagrams: fig. C.4

E.4.15 MLaaS Service (Pilot Deployment)

Responsibility: This represents the cloud MLaaS service.

Visible on diagrams: fig. C.3

E.4.16 MLaaS Cloud

Responsibility: Cloud infrastructure of the MLaaS Service provider.

Visible on diagrams: fig. C.1

E.4.17 MLaaSConnectorNode

Responsibility: Node responsible for handling connections with external MLaaS providers.

Visible on diagrams: figs. C.1 and C.2

E.4.18 MonitoringNode

Responsibility: Server node hosting the monitoring and recovery tier of the PMS backend: MonitoringService and RecoverySyncService. Responsible for detecting missing sensor data and internal failures within the Av2 detection deadlines, classifying failures by patient risk, dispatching notifications, recording communication gaps, and coordinating post-failure recovery.

Visible on diagrams: fig. C.2

E.4.19 MonitoringNode (Pilot Deployment)

Responsibility: Pilot-deployment instance of the monitoring tier hosting MonitoringService and RecoverySyncService. Responsible for failure detection, classification, notification, and post-failure recovery coordination for the 200-patient pilot population.

Visible on diagrams: fig. C.3

E.4.20 PatientDataNode

Responsibility: Server node hosting the patient data service layer: PatientDataService (owns raw sensor data, patient status, and HIS EHR proxy), PatientStatusCache (read-through cache for fast status lookups), and HISAdapter (outbound adapter for the external HIS healthAPI). Bridges the backend ingestion tier and the external Hospital Information System.

Visible on diagrams: fig. C.2

E.4.21 PatientDataNode (Dev Deployment)

Responsibility: Development-deployment instance of the patient data tier. HISAdapter points to a mock HIS stub; no real HIS connection.

Visible on diagrams: fig. C.4

E.4.22 PatientDataNode (Pilot Deployment)

Responsibility: Pilot-deployment instance of the patient data tier.

Visible on diagrams: fig. C.3

E.4.23 PatientGatewayDevice

Responsibility: This node represents the physical wearable/handheld device carried by the patient.

Visible on diagrams: fig. C.1

E.4.24 PatientRecordsNode

Responsibility: Server node hosting all patient-data persistence and identity components: `PatientRecordRepository`, `PatientManagementService`, `PatientStatusService`, `UserAuthenticationService`, and `SensorDataRepository`. Acts as the central data tier of the PMS backend.

Visible on diagrams: fig. C.2

E.4.25 PatientRecordsNode (Pilot Deployment)

Responsibility: Pilot-deployment instance of the patient-data persistence tier. Single instance hosting `PatientRecordRepository` and `SensorDataRepository` (plus the management/status/authentication services) for the 200-patient pilot.

Visible on diagrams: fig. C.3

E.4.26 PhysicianNode

Responsibility: Server node hosting all physician-facing interface components: `PhysicianGateway` (the single external entry point for all physician API calls, with no business logic), `PhysicianCommandService` (handles physician write and command flows for UC7 configure risk assessment, UC8 update risk level, and UC9 on-demand consultation), `PatientQueryService` (handles UC6 patient-status reads with tiered SLA priority queue for high, medium, and low risk patients), and `NotificationDispatcher` (UC5 dispatcher that subscribes to risk events, maintains a red-yellow-green priority queue, and fans notifications out to registered physicians). Communicates with `BackendGatewayNode` to reach sensor data and gateway services, with `PatientRecordsNode` for patient status and record reads, and with `RiskEstimationMgmtNode` to launch risk estimations.

Visible on diagrams: fig. C.2

E.4.27 PhysicianNode (Pilot Deployment)

Responsibility: Pilot-deployment instance of the physician interface tier. Single instance serving all physicians accessing the 200-patient pilot.

Visible on diagrams: fig. C.3

E.4.28 PhysicianWorkstation

Responsibility: Client device used by physicians to access the PMS. Hosts the `PhysicianGateway` web or desktop client through which cardiologists and other registered physicians consult patient status (UC6), perform on-demand consultations (UC9), configure risk assessments (UC7), update risk levels (UC8), and receive notifications about patient status changes (UC5). Outside the PMS server-side deployment boundary.

Visible on diagrams: fig. C.1

E.4.29 RiskEstimationManagementNode (Pilot Deployment)

Responsibility: The `RiskEstimationManagement` node hosts the creation, scheduling and combining of the clinical jobs.

Visible on diagrams: fig. C.3

E.4.30 RiskEstimationManagmentNode (Dev Deployment)

Responsibility: The development deployment is a single RiskEstimationManagement node that contains all other functionality, job creation, scheduling, and combination.

Visible on diagrams: fig. C.4

E.4.31 RiskEstimationMgmtNode

Responsibility: The RiskEstimationMgmtNode keeps track of the risk estimation jobs, and schedules and combines the results.

Visible on diagrams: fig. C.2

E.4.32 RiskEstimationProcessorNode

Responsibility: Multiple RiskEstimationProcessor nodes enable the processing of multiple sensorData inputs in parallel.

Visible on diagrams: fig. C.2

E.4.33 RiskEstimationProcessorNode (Dev Deployment)

Responsibility: The development deployment only uses two RiskEstimationProcessorNodes to run **ClinicalModelJobs** in parallel.

Visible on diagrams: fig. C.4

E.4.34 RiskEstimationProcessorNode (Pilot Deployment)

Responsibility: There are five instances of the RiskEstimationProcessorNode to enable the processing of multiple **ClinicalModelJobs** in parallel.

Visible on diagrams: fig. C.3

E.4.35 SMSGatewayProvider

Responsibility: Represents the third-party SMS service used as the backup channel.

Visible on diagrams: fig. C.1

E.4.36 TelecomNetwork

Responsibility: represents the cellular network governed by the SLA (99.5% / 80% / 64Kb/s).

Visible on diagrams: fig. C.1

E.4.37 TODO Node (Pilot Deployment)

Responsibility: This TODO node for the pilot deployment contains the **OtherFunctionality** of the system that has not been worked out in detail yet.

Visible on diagrams: fig. C.3

E.4.38 TODONode

Responsibility: Temporary node that requires further decomposition.

Visible on diagrams: fig. C.2

E.5 Exceptions

- *AuthenticationException* (Δ Exception):
Signals a problem with the credentials used for authentication. These may be invalid or expired.
- *AuthenticationException2* (Δ SecurityException):
Missing or invalid credentials.
- *AuthorizationException* (Δ Exception):

- Signals that the principal has insufficient access rights to perform the operation.
- *AuthorizationException2* (Δ *SecurityException*):
The requestor is not authorized to perform the requested action.
- *Exception*:
Subtypes: Δ *AuthenticationException*, Δ *AuthorizationException*, Δ *InvalidAPIKeyException*, Δ *InvalidMLModelException*, Δ *IOException*, Δ *NoSuchMLModelException*, Δ *NoSuchPatientException*, Δ *NoSuchPatientRecordException*
Base exception class.
- *InvalidAPIKeyException* (Δ *Exception*):
Thrown when no correctly formatted API key is configured for constructing authentication Credential[s].
- *InvalidMLModelException* (Δ *Exception*):
Thrown when a provided MLModel is invalid.
- *IOException* (Δ *Exception*):
Thrown when an IO operation fails.
- *NoSuchMLModelException* (Δ *Exception*):
Thrown when the requested MLModel cannot be found.
- *NoSuchPatientException* (Δ *Exception*):
Thrown if no patient with the specified identifier exists.
- *NoSuchPatientRecordException* (Δ *Exception*):
Thrown if no patient record for the patient with the given identifier exists in the HIS.
- *SecurityException*:
Subtypes: Δ *AuthenticationException2*, Δ *AuthorizationException2*
Parent class for security exceptions.

E.6 Data types

- **Address**:
Attributes: **AddressUse** use, **AddressType** type, String text, List<String> line, String city, String district, String state, String postalCode, String country, **Period** period, long serialVersionUID
Base StructureDefinition for Address Type: An address expressed using postal conventions (as opposed to GPS or other location definition formats). This data type may be used to convey addresses for use in delivering mail as well as for visiting locations which might not be valid for mail delivery. There are a variety of postal address formats defined around the world. The line attribute contains the house number, apartment number, street name, street direction, P.O. The text attribute specifies the entire address as it should be displayed e.g. on a postal label. This may be provided instead of or as well as the specific parts.
- **AddressType**:
Enum: POSTAL, PHYSICAL, BOTH, NULL.
Distinguishes between physical addresses (those you can visit) and mailing addresses (e.g. PO Boxes and care-of addresses). Most addresses are both.
- **AddressUse**:
Enum: HOME, WORK, TEMP, OLD, BILLING, NULL.
The purpose of the address.
- **Annotation**:
Attributes: **Date** time, String text, long serialVersionUID, String author
Base StructureDefinition for Annotation Type: A text note which also contains information about who made the statement and when.
- **backupChannelType**:
Enum: SMS. The type of backup communication channel available to the patient gateway when the primary channel is unavailable.
- **BackupDeliveryResult**:
Attributes: boolean succes, **Timestamp** sendAt, string channelUsed
The result of an attempt to send a message via the backup communication channel.
- **BigDecimal**:
Immutable, arbitrary-precision signed decimal numbers. A BigDecimal consists of an arbitrary

- precision integer unscaled value and a 32-bit integer scale
- ClassifiedFailure:**
 Attributes: **FailureEvent** failure, **EscalationLevel** level, **Timestamp** notificationDeadline
 A failure event enriched with its escalation level and the deadline by which the relevant stakeholder must be notified, derived from the affected patient risk level per Av2 response measures.
 - ClinicalModel:**
 Attributes: String id, String name, String description, List<String> requiredParameters, List<String> optionalParameters, List<String> applicableMLModelIds, boolean requiresHistoricalSensorData
 A clinical model. This model has an id, name, description. It contains a list of parameters for using the model and a set of MLModelIds to specify which types of MLModels can be used in the context of the ClinicalModel.
 - ClinicalModelConfiguration:**
 Attributes: Map<String, String> modelParameters
 The configuration for a clinical model. Represented as a set key/value pair for the parameters of the clinical model.
 - ClinicalModelJob:**
 Attributes: **ClinicalModel** clinicalModel, **ClinicalModelJobId** jobId, **SensorDataPackage** sensorData, **PatientId** patientId, **ClinicalModelConfiguration** clinicalModelConfiguration, **PatientRecord** patientRecord, Map<**Timestamp**, **SensorDataPackage**> historicalSensorData
 A single job for the computation of a clinical model. This contains a ClinicalModelJobId, ClinicalModelId and PatientId respectively identifying the job itself, the corresponding clinical model and the patient. If the corresponding risk estimation the job belongs to is triggered by the arrival of a sensor data package, this sensor data is also contained.
 - ClinicalModelJobId:**
 A piece of data uniquely identifying a certain clinical model job in the system. This architecture does not specify the exact format of this identifier, but possibilities are a long integer, a string, a URI etc.
 - ClinicalModelJobResult:**
 The result of the computation of a clinical model containing the estimated risk level for the patient. The resulting risk score for ClinicalModels are on the same unit scale so the the results of multiple ClinicalModels can be combined.
 - CodeableConcept:**
 Attributes: List<**Coding**> coding, String text, long serialVersionUID
 Base StructureDefinition for CodeableConcept Type: A concept that may be defined by a formal reference to a terminology or ontology or may be provided by text.
 - CodeType:**
 Attributes: long serialVersionUID, String system, String value
 Primitive type "code" in FHIR, when not bound to an enumerated list of codes
 - Coding:**
 Attributes: String system, String version, **CodeType** code, String display, boolean userSelected, long serialVersionUID
 Base StructureDefinition for Coding Type: A reference to a code defined by a terminology system.
 - ConsultationId:**
 TODO maybe delete/merge into CorrelationId
 - ContactComponent:**
 Attributes: List<**CodeableConcept**> relationship, String name, List<**ContactPoint**> telecom, **Address** address, String administrativeGender, String organization, **Period** period, long serialVersionUID
 A contact party (e.g. guardian, partner, friend) for the patient. The relationship attribute species the nature of the relationship between the patient and the contact person. It is a CodeableConcept (which can be formal reference to terminology or an ontology). For example, in the system <http://hl7.org/fhir/ValueSet/relatedperson-relationshiptype>, it can be coded as CHILD for a child, or MGRMTH for maternal grandmother, etc.
 - ContactPoint:**
 Attributes: **ContactPointSystem** system, String value, **ContactPointUse** use, int rank, long

serialVersionUID, **Period** period

Base StructureDefinition for ContactPoint Type: Details for all kinds of technology mediated contact points for a person or organization, including telephone, email, etc. The value attribute contains the actual contact point details, in a form that is meaningful to the designated communication system (i.e. phone number or email address). Rank specifies a preferred order in which to use a set of contacts. ContactPoints with lower rank values are more preferred than those with higher rank values. Period specifies the time period when the contact point was/is in use.

- **ContactPointSystem:**

Enum: PHONE, FAX, EMAIL, PAGER, URL, SMS, OTHER, NULL.

Enum denoting the type of communication channel for the ContactPoint.

- **ContactPointUse:**

Enum: HOME, WORK, TEMP, OLD, MOBILE, NULL.

Enum denoting the usage type of the ContactPoint.

- **CorrelationId:**

Cross-component tag that ties a UC9 on-demand consultation to its eventual risk-estimation result.

- **Credential:**

Authentication object for making authenticated requests to the MLaaS service provider.

- **Date:**

The class Date represents a specific instant in time, with millisecond precision.

- **DegradedModeReason:**

Enum: NO_NETWORK, SERVER_UNREACHABLE, INTERNAL_FAILURE. The reason the patient gateway entered degraded mode.

- **Duration:**

Represents a length of time.

- **EmergencyMessage:**

Attributes: **PatientId** patientId, **GatewayId** gatewayId, **EmergencyType** type, **Timestamp** detectedAt, string payload

An emergency alert to be dispatched via the backup channel when the primary communication channel is unavailable.

- **EscalationLevel:**

Enum: HIGH, MEDIUM, LOW. The urgency level of a detected failure, derived from the risk level of the affected patient. HIGH corresponds to high-risk patients (notify within 2 minutes), MEDIUM to medium-risk patients (notify within 5 minutes), LOW to low-risk patients (notify within 10 minutes).

- **FailureEvent:**

Attributes: **FailureEventId** id, **FailureType** type, **GatewayId** gatewayId, int subsystemId, **Timestamp** detectedAt, **PatientId** patientId

Represents a detected failure in the system. The gatewayId is present when the failure concerns a specific patient gateway, and may be null for internal subsystem failures. Similarly patientId is null when the failure does not concern a specific patient. The detectedAt timestamp records when the system became aware of the failure, while startedAt estimates when the failure actually began.

- **FailureEventId:**

A unique identifier for a failure event.

- **FailureType:**

Enum: MISSING_SENSOR_DATA, INTERNAL_SUBSYSTEM_CRASH, COMMUNICATION_CHANNEL_DOWN, GATEWAY_UNREACHABLE. Classifies the kind of failure detected by the MonitoringService.

- **FilterCriteria:**

Attributes: **CorrelationId** correlationId

Optional filter passed to IRiskEvents.subscribe. Empty for NotificationDispatcher (wants every event); carries a correlationId for PhysicianCommandService UC9 result lookup.

- **GatewayId:**

A piece of data uniquely identifying a patient gateway device.

- **HealthStatus:**

Enum: OK, DEGRADED, DOWN. The operational status of an internal subsystem as

reported by its health check endpoint.

- **Identifier:**

Attributes: **IdentifierUse** use, String type, String system, String value, **Period** period, String assigner, long serialVersionUID

Base StructureDefinition for Identifier Type: An identifier - identifies some entity uniquely and unambiguously. Typically this is used for business identifiers. The type is a coded type for the identifier that can be used to determine which identifier to use for a specific purpose. The system attribute establishes the namespace for the value - that is, a URL that describes a set of values that are unique. The value attribute specifies the portion of the identifier typically relevant to the user and which is unique within the context of the system. The period specifies the time period during which identifier is/was valid for use. The assigner specifies the organization that issued/manages the identifier.

- **IdentifierUse:**

Enum: USUAL, OFFICIAL, TEMP, SECONDARY, OLD, NULL.

Enum denoting the usage type of a particular Identifier.

- **MLModel:**

Attributes: String modelId, **MLModelDescription** modelDescription

Machine learning model for making predictions based on sensordata. The actual internal encoding of the model itself is left open.

- **MLModelDescription:**

Attributes: String uuid, String name, String description, String providerName

Object providing the MLModel meta data. It does not contain the model itself.

- **MLModelUpdate:**

Attributes: **MLModelDescription** modelDescription

Represents a model update that can be applied on an MLModel. These model updates enable exchanging changes to the MLModel between different parties without sending the actual MLModel to the other parties.

- **NetworkType:**

NetworkType: Enum: NONE, CELLULAR, WIFI. The type of network connection currently available to the patient gateway. CELLULAR refers to mobile network connectivity governed by the SLA with the telecom operator.

- **Notification:**

Attributes: **NotificationId** id, **PatientId** patientId, **PatientStatus** status, **NotificationSeverity** severity, **Timestamp** ts

UC5 payload returned to physicians via INotificationInbox.getPendingNotifications.

- **NotificationId:**

Identifies one queued notification (used for ack/drop).

- **NotificationSeverity:**

Enum: RED, YELLOW, GREEN, LITERAL.

Severity of a Notification. Drives the dispatcher's priority queue + drop policy (Response measure 5: red > yellow > green; green dropped first under overload).

- **Object:**

Root class for objects that can be stored in the Key-Value storage service. Data types that need to be saved need to inherit from this class.

- **Observation:**

Attributes: List<**Identifier**> identifier, List<**Identifier**> partOf, **ObservationStatus** status, List<**CodeableConcept**> category, **CodeableConcept** code, **Patient** subject, **Period** effective, **Date** issued, List<**Identifier**> performer, String value, **CodeableConcept** dataAbsentReason, **CodeableConcept** interpretation, List<**Annotation**> note, **CodeableConcept** bodySite, **CodeableConcept** method, String device, List<**Observation**> hasMember, List<**Observation**> derivedFrom, long serialVersionUID, **ObservationReferenceRangeComponent** referenceRange, **ObservationComponentComponent** component

Measurements and simple assertions made about a patient, device or other subject. The partOf attribute specifies the larger event of which this particular Observation is a component or step. For example, an observation as part of a procedure.

The category specifies a code that classifies the general type of observation being made. For example, in the system <http://terminology.hl7.org/CodeSystem/observation-category>, vital-signs is used for clinical observations such as blood pressure, heart rate, etc.

The code attribute describes what was observed. Sometimes this is called the observation 'name'. For example, in the system <http://loinc.org>, code '8867-4' is used for expressing the heart rate, '85354-9' for a blood pressure panel (which must have components '8480-6' and '8462-4' for, respectively, systolic and diastolic pressure).

The performer is who was responsible for asserting the observed value as 'true'. This can refer to a Patient, Practitioner, organization, etc. The value is the information determined as a result of making the observation, if the information has a simple value.

The dataAbsentReason provides a reason why the expected value in the element Observation.value[x] is missing. For example, in the system <http://terminology.hl7.org/CodeSystem/data-absent-reason>, 'not-applicable'.

The interpretation is a categorical assessment of an observation value. For example, high, low, normal. For example, in the system <http://terminology.hl7.org/CodeSystem/v3-ObservationInterpretation>, 'H' for high, or 'ND' for not detected.

The bodySite indicates the site on the subject's body where the observation was made (i.e. the target site). For example, in the system <http://snomed.info/sct>, code '344001' indicates the ankle.

The method indicates the mechanism used to perform the observation. For example, in the system <http://snomed.info/sct>, code '46973005' indicates blood pressure taking procedure.

The device attribute specifies an identifier of the device used to generate the observation data.

- **ObservationComponentComponent:**

Attributes: String code, **Range** value, String dataAbsentReason, List<String> interpretation, long serialVersionUID, **ObservationReferenceRangeComponent** referenceRange

Some observations have multiple component observations. These component observations are expressed as separate code value pairs that share the same attributes. Examples include systolic and diastolic component observations for blood pressure measurement and multiple component observations for genetics observations. The code describes what was observed. Sometimes this is called the observation 'code'. For example, in the system <http://loinc.org>, '8480-6' is used for the systolic component of the blood pressure.,

The dataAbsentReason provides a reason why the expected value in the element Observation.component.value[x] is missing. For example, in the system <http://terminology.hl7.org/CodeSystem/data-absent-reason>, 'not-applicable'.

The interpretation provides a categorical assessment of an observation value. For example, high, low, normal. For example, in the system <http://terminology.hl7.org/CodeSystem/v3-ObservationInterpretation>, 'H' for high, or 'ND' for not detected.

- **ObservationReferenceRangeComponent:**

Attributes: **Quantity** low, **Quantity** high, **CodeableConcept** type, List<**CodeableConcept**> appliesTo, **Range** age, String text, long serialVersionUID

Guidance on how to interpret the value by comparison to a normal or recommended range. Multiple reference ranges are interpreted as an "OR". In other words, to represent two distinct target populations, two 'referenceRange' elements would be used.

The appliesTo specifies codes to indicate the target population this reference range applies to. For example, in the system <http://snomed.info/sct>, code '248152002' indicates female.

The type specifies codes to indicate the what part of the targeted reference population it applies to. For example, in the system <http://terminology.hl7.org/CodeSystem/referencrange-meaning>, 'normal' for the 95% normal range, or 'recommended' for the recommended range by a relevant professional body.

The text attribute contains a text based reference range in an observation which may be used when a quantitative range is not appropriate for an observation.

- **ObservationStatus:**

Enum: REGISTERED, PRELIMINARY, FINAL, AMENDED, CORRECTED, CANCELLED, ENTEREDINERROR, UNKNOWN, NULL.

The status of the observation result value or the RiskAssessment.

- **Patient:**

Attributes: List<**Identifier**> identifier, boolean active, List<String> name, String administrativeGender, **Date** birthDate, **Date** deceased, List<**Address**> address, boolean multipleBirth, List<**Identifier**> generalPractitioner, **Identifier** managingOrganization, long serialVersionUID, **ContactComponent** contact

Demographics and other administrative information about an individual or animal receiving care

- or other health-related services.
- **PatientId:**
A piece of data uniquely identifying a patient in the system. This architecture does not specify the exact format of this identifier, but possibilities are a long integer, a string, a URI, etc.
- **PatientRecord:**
The structured contents of the EHR record of a certain patient, including medication history, recent treatments, allergies, etc.
- **PatientStatus:**
The status of a patient, i.e., their risk level.
- **Period:**
Attributes: **Date** start, **Date** end, long serialVersionUID
Base StructureDefinition for Period Type: A time period defined by a start and end date and optionally time.
- **PhysicianId:**
Identifies a physician for notification routing.
- **Prediction:**
A prediction from the MLModel.
- **Priority:**
Enum: HIGH, NORMAL.
Priority of a launched risk-estimation job. UC9 uses 'HIGH'; 'NORMAL' is the default for legacy schedule-driven jobs.
- **Quantity:**
Attributes: **BigDecimal** value, String unit, String system, **CodeType** code, long serialVersionUID
Base StructureDefinition for Quantity Type: A measured amount (or an amount that can potentially be measured). Note that measured amounts include amounts that are not precisely quantified, including amounts involving arbitrary units and floating currencies. Unit specifies a human-readable form of the unit. System contains the identification of the system that provides the coded form of the unit. The code contains a computer processable form of the unit in some unit representation system. For example, blood pressure can be expressed in the system <http://unitsofmeasure.org> and the unit mm[Hg].
- **Query<T>:**
Attributes: List<T> values, List<String> qualifier
The Query object is used to express search restrictions in a string-based format for interacting with the REST-based health API. For example, before a specified date is encoded as 'ltyyyy-MM-dd'. The list of qualifiers specify how the corresponding value should be used (e.g., lt for less than a date, matches if it should be present in a string, exact for an exact string match).
- **Range:**
Attributes: long serialVersionUID, **Quantity** low, **Quantity** high
Base StructureDefinition for Range Type: A set of ordered Quantities defined by a low and high limit.
- **RedeploymentJob:**
Attributes: int subsystemId, **RedeploymentJobId** id, **Timestamp** startedAt, **RedeploymentStatus** status
Represents an active or completed redeployment or rollback operation. Must complete within 5 minutes per Av2 response measures.
- **RedeploymentJobId:**
A unique identifier for a redeployment job.
- **RedeploymentStatus:**
Enum: IN_PROGRESS, COMPLETED, FAILED, ROLLED_BACK. The current status of a redeployment or rollback operation initiated by the system administrator.
- **RetryState:**
Attributes: **GatewayId** gatewayId, int attemptNumber, **Timestamp** nextRetryAt, **Duration** currentBackoffInterval
Tracks the current exponential backoff retry state for a patient gateway attempting to re-establish communication. The currentBackoffInterval doubles with each failed attempt.
- **RiskAssessment:**

Attributes: List<**Identifier**> identifier, **ObservationStatus** status, **CodeableConcept** method, **CodeableConcept** code, **Patient** subject, **Date** occurrence, String condition, **Identifier** performer, List<**CodeableConcept**> reason, List<**Observation**> basis, String mitigation, List<**Annotation**> note, long serialVersionUID, **RiskAssessmentPredictionComponent** prediction

An assessment of the likely outcome(s) for a patient or other subject as well as the likelihood of each outcome. Method specifies the algorithm, process or mechanism used to evaluate the risk. Code specifies the type of the risk assessment performed. For example, in the system <http://snomed.info/sct>, '709510001' represents Assessment of risk for disease (procedure). For assessments or prognosis specific to a particular condition, the condition attribute indicates the condition being assessed. It should be a reference to an existing condition but is a simplified as a string here. Reason specifies the reason the risk assessment was performed. Mitigation contains a description of the steps that might be taken to reduce the identified risk(s).

- **RiskAssessmentPredictionComponent:**

Attributes: String outcome, **Range** probability, String qualitativeRisk, **BigDecimal** relativeRisk, **Date** when, String rationale, long serialVersionUID

Describes the expected outcome for the subject. Outcome specifies one of the potential outcomes for the patient (e.g. remission, death, a particular condition). QualitativeRisk indicates how likely the outcome is (in the specified timeframe), expressed as a qualitative value (e.g. low, medium, or high). Rationale contains additional information explaining the basis for the prediction. Both outcome and qualitativeRisk are CodeableConcepts (which can be formal reference to terminology or an ontology), but simplified as a string.

- **RiskEstimationId:**

A piece of data uniquely identifying a risk estimation performed by the PMS. This architecture does not specify the exact format of this identifier, but possibilities are a long integer, a string, a URL etc.

- **RiskEvent:**

Attributes: **PatientId** patientId, **PatientStatus** status, **Timestamp** ts, **CorrelationId** correlationId

Payload pushed to IRiskEvents subscribers when setEstimatedPatientStatus actually changes the stored status.

- **SensorDataElement:**

Attributes: String sensorId, String sensorType, String measurementType, String measurementUnit, **BigDecimal** measurementValue, String measurementValueString

Each sub-package lists the id of the sensor, the type of the sensor, the type of the measurements and the measurements itself. These measurements range from a single value (e.g., in case of the blood pressure) to a complex data structure (e.g., in case of an ECG). Non-numerical types of measurements can be captured in the measurementValueString attribute.

- **SensorDataPackage:**

Attributes: List<**SensorDataElement**> sensorDataElements

A package of sensor data, i.e., a list of sensorDataElements.

- **SignalStrength:**

Enum: NONE, WEAK, MODERATE, STRONG. Represents the quality of the network signal available to the patient gateway. NONE indicates no signal, WEAK indicates signal insufficient for reliable data transmission, MODERATE indicates functional but potentially lossy connectivity, STRONG indicates reliable connectivity.

- **SubscriberId:**

Identifies a component-level subscriber (NotificationDispatcher, PhysicianCommandService). Distinct from PhysicianId.

- **SubscriptionId:**

Handle for an active subscription.

- **Timestamp:**

The representation of a time (i.e., date and time of the day) in the system.

E.7 Unresolved issues

SA Plugin v10.0.0 (VP OpenAPI v17.2)

- [Comp.02] : 1 (The required interface x::x is not required by any child component)

- [Comp.03] : 3 (*The provided interface $x::x$ is not required by any sibling components, but is also not provided by its parent component x*)
- [Comp.04] : 8 (*The required interface $x::x$ is not provided by any sibling components, but is also not required by its parent component x*)
- [Comp.05] : 2 (*Component x has no provided or required interfaces*)
- [Comp.08] : 1 (*The required interface $x::x$ is not required by any child module*)
- [Diag.comp.06] : 24 (*The component x may be missing on the C/S context diagram.*)
- [Diag.depl.04] : 3 (*Component x and its ancestor x have both been deployed*)
- [Diag.depl.05] : 30 (*Component x at x cannot communicate with component providing x through a deployment connection*)
- [Diag.depl.06] : 13 (*The deployment connection between x and x is not used by any of the components deployed in the two nodes.*)
- [Diag.seq.01] : 2 (*x : x (x) calls x but does not require its interface.*)
- [If.02] : 1 (*The interface x is not provided by any component or module*)
- [If.03] : 1 (*The interface x is not required by any component or module*)
- [Op.01] : 1 (*The interface x has no operations*)
- [Op.03] : 2 (*Operation $x::x$ uses non-existing data type x .*)